Handbook de Programación Competitiva



McDuck Team

November 1, 2026

Contents

1 Base de competencia **2**
- 1.1 Template C++17 . . . 2
- 1.2 Includes estándar sin bits/stdc++.h . . . 2
- 1.3 Matemática cotidiana en C++ . . . 2
- 1.4 Template Python 3 . . . 5
- 1.5 Imports y utilidades de Python . . . 5
- 1.6 Tipos, límites y overflow . . . 6
- 1.7 Sintaxis C++ más usada . . . 6
- 1.8 STL esencial . . . 6
- 1.9 Contenedores comunes . . . 6
- 1.10 Algoritmos STL comunes . . . 7
- 1.11 Comparadores, bits y aleatoriedad . . . 7
- 1.12 Entrada y salida . . . 7
- 1.13 Parsing seguro . . . 8
- 1.14 Letras a posiciones . . . 8

2 Técnicas **8**
- 2.1 Búsqueda binaria . . . 8
- 2.2 Sumas parciales . . . 8
- 2.3 Two pointers y sliding window . . . 8
- 2.4 Máscaras de bits . . . 9
- 2.5 Conversión a base arbitraria . . . 9
- 2.6 Compresión de coordenadas . . . 9
- 2.7 Descomposición por raíz: frecuentes y poco frecuentes . . . 9
- 2.8 Meet-in-the-middle: suma de subconjuntos . . . 9

3 Estructuras de datos **10**
- 3.1 Disjoint Set Union (DSU) . . . 10
- 3.2 Fenwick Tree (BIT) . . . 10
- 3.3 Segment Tree iterativo . . . 10
- 3.4 Lazy Segment Tree . . . 11
- 3.5 Segment Tree persistente . . . 12
- 3.6 Sparse Table . . . 12
- 3.7 Disjoint Sparse Table . . . 13
- 3.8 Stack y queue monótonas . . . 13
- 3.9 Ordered Set de PBDS . . . 13
- 3.10 Fenwick 2D offline . . . 13
- 3.11 Wavelet Tree . . . 14
- 3.12 Treap implícito . . . 14
- 3.13 Convex Hull Trick y Li Chao Tree . . . 15
- 3.14 Intervalos disjuntos (ODT) . . . 16

4 Grafos **16**
- 4.1 BFS, DFS y bipartición . . . 16
- 4.2 BFS multiorigen . . . 16
- 4.3 Orden topológico . . . 17
- 4.4 Componentes fuertemente conexas (Tarjan) . . . 17
- 4.5 2-SAT . . . 17
- 4.6 Dijkstra . . . 18
- 4.7 0–1 BFS . . . 18
- 4.8 Bellman–Ford . . . 18
- 4.9 Floyd–Warshall . . . 19
- 4.10 Árbol de expansión mínima (Kruskal) . . . 19
- 4.11 Puentes y puntos de articulación . . . 19
- 4.12 Componentes biconexas por vértices . . . 20
- 4.13 Ancestro común más bajo (LCA) . . . 20
- 4.14 LCA con binary lifting . . . 21
- 4.15 LCA $O(1)$ con Euler tour y RMQ . . . 21
- 4.16 Virtual Tree . . . 21
- 4.17 Heavy-Light Decomposition . . . 22
- 4.18 Descomposición por centroides . . . 22
- 4.19 Rerooting DP . . . 23
- 4.20 Diámetro de un àrbol y camino . . . 23
- 4.21 Camino euleriano . . . 23
- 4.22 Flujo máximo (Ford–Fulkerson) . . . 24
- 4.23 Flujo máximo (Dinic) . . . 24
- 4.24 Flujo de costo mínimo . . . 25
- 4.25 Matching bipartito: Hopcroft–Karp . . . 25
- 4.26 Asignación de costo mínimo: Húngaro . . . 26
- 4.27 Cobertura mínima bipartita: König . . . 26

5 Strings **26**
- 5.1 Rolling hash . . . 26
- 5.2 Prefix function, KMP . . . 27
- 5.3 Z-function . . . 27
- 5.4 Trie . . . 27
- 5.5 Manacher . . . 28
- 5.6 Aho–Corasick . . . 28
- 5.7 Suffix Array y LCP . . . 28

6 Matemática **29**
- 6.1 Teoremas e identidades para reconocer soluciones 29
- 6.2 GCD, LCM y Euclides extendido . . . 31
- 6.3 Aritmética modular . . . 31
- 6.4 Teorema chino del resto para dos congruencias . 31
- 6.5 Combinatoria módulo primo . . . 31
- 6.6 Teorema de Lucas . . . 32
- 6.7 Criba lineal y menor factor primo . . . 32
- 6.8 Criba segmentada: primos en un intervalo . . . 32
- 6.9 Factorización por división por tentativa . . . 32
- 6.10 Listar divisores . . . 32
- 6.11 Pollard–Rho con Brent . . . 33
- 6.12 Pollard–Rho con Brent optimizado . . . 33
- 6.13 Exponenciación de matrices . . . 34
- 6.14 Eliminación gaussiana y determinante . . . 34
- 6.15 Base XOR de enteros de 64 bits . . . 35
- 6.16 Dependencia XOR de bitsets con reconstrucción 35
- 6.17 NTT y convolución . . . 35

7 Geometría **36**
- 7.1 Geometría 2D entera . . . 36
- 7.2 Área y convex hull . . . 36
- 7.3 Rectas con punto flotante . . . 36
- 7.4 Círculos . . . 37
- 7.5 Punto en polígono . . . 37
- 7.6 Rotating calipers: diámetro convexo . . . 37
- 7.7 Geometría 3D y planos . . . 37
- 7.8 Camino mínimo rodeando un polígono convexo . 38

8 Programación dinámica **38**
- 8.1 Longest Increasing Subsequence . . . 38
- 8.2 LCS con reconstrucción . . . 38
- 8.3 Distancia de edición con operaciones . . . 38
- 8.4 Distancia al palíndromo más cercano . . . 39
- 8.5 Knapsack 0/1 . . . 39
- 8.6 Bitmask DP: camino hamiltoniano mínimo . . . 39
- 8.7 Mochila ilimitada . . . 39
- 8.8 Suma exacta 0/1 con mínima cantidad de objetos 40
- 8.9 Subset sum con bitset . . . 40
- 8.10 Digit DP: contar enteros por sus dígitos . . . 40
- 8.11 Divide and Conquer DP Optimization . . . 40
- 8.12 DP $O(n\sqrt{n})$ por cota triangular . . . 41

9 Algoritmos offline **41**
- 9.1 Mo's Algorithm . . . 41

1. Base de competencia

1.1 Template C++17

Base común para los snippets del handbook. Incluye el template clásico del equipo; las macros son opcionales, pero permiten pegar código viejo sin traducir nombres durante una competencia.

```
#include <bits/stdc++.h>
#define pb push_back
#define fst first
#define snd second
#define fore(i,a,b) for (int i = (a), ThxDem = (b); i < ThxDem; ++i)
#define SZ(x) ((int)(x).size())
#define ALL(x) (x).begin(), (x).end()
#define mset(a,v) memset((a), (v), sizeof(a))
#define FIN ios::sync_with_stdio(false); cin.tie(nullptr); cout.tie(
     nullptr)
using namespace std;

using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;
using u128 = __uint128_t;
using ii = pair<int,int>;
using pii = pair<int,int>;
using pll = pair<ll,ll>;

const int INF = numeric_limits<int>::max() / 4;
const ll LINF = numeric_limits<ll>::max() / 4;

int main() {FIN;

    return 0;
}
```

Para llenar memoria byte a byte, usar `memset` solamente con valores seguros como cero o -1. Para otros valores, usar `fill`.

Cuidado: `fore` guarda el límite una sola vez: no usarlo si ese límite debe cambiar dentro del loop. (Es decir, no se recalcula con cada iteración como si lo haría i < SZ(v) `pb/fst/snd/ALL/SZ` son abreviaturas; ante un error de tipos, expandirlas mentalmente antes de depurar.

1.2 Includes estándar sin bits/stdc++.h

`bits/stdc++.h` es la opción rápida con GNU C++; no es parte del estándar. Si el juez usa Clang/MSVC o exige portabilidad, incluir uno por uno:

Necesidad	Header
entrada/salida	`<iostream>`, `<iomanip>`, `<sstream>`
vector/arreglos/tuplas	`<vector>`, `<array>`, `<tuple>`, `<utility>`
strings	`<string>`, `<cstring>`
sort/búsquedas/numéricos	`<algorithm>`, `<numeric>`
stack/colas	`<stack>`, `<queue>`, `<deque>`
set/map/hash	`<set>`, `<map>`, `<unordered_set>`, `<unord`
matemática	`<cmath>`, `<complex>`, `<limits>`, `<bitset`
random/tiempo	`<random>`, `<chrono>`
funciones/opcionales	`<functional>`, `<optional>`
assert/tipos enteros	`<cassert>`, `<cstdint>`

Template portable mínimo:

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <iostream>
#include <limits>
#include <map>
#include <numeric>
#include <queue>
#include <set>
#include <string>
#include <tuple>
#include <unordered_map>
#include <utility>
#include <vector>
#include <climits>
```

Agregar headers sólo cuando se usen. Para ICPC con GCC, `bits/stdc++.h` sigue siendo la alternativa más práctica.

1.3 Matemática cotidiana en C++

1.3.1 Raíces, potencias y logaritmos

Operación	C++
$\sqrt{x}$ real	`sqrt(x)`, `sqrtl(x)`
x^y real	`pow(x,y)`, `powl(x,y)`
$e^x, \ln x$	`exp(x)`, `log(x)`
$\log_2 x, \log_{10} x$	`log2(x)`, `log10(x)`
redondeos	`floor`, `ceil`, `round`, `trunc`
hipotenusa estable	`hypot(x,y)`
valor absoluto	`abs(x)`, `fabs(x)`

Cuidado: No usar `pow(a,b)` para calcular potencias enteras exactas, ya que trabaja con punto flotante y puede introducir errores de redondeo. En su lugar, usar exponenciación binaria.

Tampoco confiar en convertir directamente `sqrt(n)` a entero cuando `n` está cerca de cuadrados perfectos grandes, debido a posibles errores de precisión.

Para calcular raíces cuadradas enteras, se recomienda utilizar la función `integer_sqrt`.

Para realizar una división entera redondeando el resultado hacia arriba, utilizar `ceil_division`.

Para potencias enteras usar `binary_power`: requiere exponente no negativo y que la respuesta entre en `long long`. La sobrecarga real admite exponentes negativos. Usar `2LL` o `2.0L` para elegir explícitamente la versión entera o real.

```
ll integer_sqrt(ll value) {
    assert(value >= 0);
    ll root = sqrtl((long double)value);
    while ((i128)(root + 1) * (root + 1) <= value) ++root;
    while ((i128)root * root > value) --root;
    return root; // floor(sqrt(value))
}

ll ceil_division(ll a, ll b) {
    assert(b && !(a == numeric_limits<ll>::min() && b == -1));
    ll q = a / b, r = a % b;
    return q + (r != 0 && ((r > 0) == (b > 0)));
}

ll floor_division(ll a, ll b) {
    assert(b && !(a == numeric_limits<ll>::min() && b == -1));
    ll q = a / b, r = a % b;
    return q - (r != 0 && ((r > 0) != (b > 0)));
}

ll binary_power(ll base, ll exponent) {
    assert(exponent >= 0);
    ll result = 1;
    for (; exponent; exponent >>= 1) {
        if (exponent & 1) result *= base;
        if (exponent > 1) base *= base; // Evita el cuadrado final
     innecesario.
    }
    return result;
}

long double binary_power(long double base, ll exponent) {
    assert(base != 0.0L || exponent >= 0);
    // Convierte |exponent| sin desbordar cuando exponent ==
     LLONG_MIN.
    unsigned long long e = exponent < 0 ? -(unsigned long long)
     exponent
                                        : (unsigned long long)exponent
     ;
    long double result = 1.0L;
    for (; e; e >>= 1) {
        if (e & 1) result *= base;
        if (e > 1) base *= base;
    }
    return exponent < 0 ? 1.0L / result : result;
}

ll modular_power(ll base, ll exponent, ll modulus) {
    assert(exponent >= 0 && modulus > 0);
    base %= modulus;
    if (base < 0) base += modulus;
    ll result = 1 % modulus;
    for (; exponent; exponent >>= 1) {
        if (exponent & 1) result = (i128)result * base % modulus;
        if (exponent > 1) base = (i128)base * base % modulus;
    }
    return result;
}
```

`integer_sqrt` ya usa una aproximación rápida con `sqrtl`; las correcciones enteras conservan exactitud cerca de cuadrados grandes. Las divisiones requieren divisor no nulo y cociente representable: excluir `LLONG_MIN / -1`. Las potencias usan $O(\log e)$ multiplicaciones y $O(1)$ memoria; para $e = 0$ devuelven la identidad.

Potencia de 2

Para potencias de dos, usar desplazamientos. **Cuidado:** El tipo de la izquierda se determina antes de asignar: usar `1LL << k` para `long long`, con $0 \le k \le 62$ para un resultado positivo. Para `unsigned long long`, usar `1ULL << k` con $0 \le k \le 63$. Nunca desplazar por un valor negativo ni por el ancho del tipo o más.

```
    // Potencia 2 elevado a la k; requiere 0 <= k <= 62.
    int k = 40;
    ll x = 1LL << k;

    // Maxima potencia positiva para tipos de 32, 64 y 128 bits.
    int p_int = 1 << 30;
    unsigned int p_uint = 1U << 31;
    ll p_ll = 1LL << 62;
    unsigned long long p_ull = 1ULL << 63;
    i128 p_i128 = (i128)1 << 126;
```

```
u128 p_u128 = (u128)1 << 127;
```

1.3.2 Punto flotante

Comparar con error absoluto y relativo: Ej: 0.1 + 02 == 03?

```
bool almost_equal(long double a, long double b,
                  long double epsilon = 1e-12L) {
    return fabsl(a - b) <= epsilon * max({1.0L, fabsl(a), fabsl(b)});
}
```

- `double`: unas 15 cifras decimales; `long double`: más en GNU/Linux x86, pero no necesariamente en todas las plataformas.
- Para $a \cdot b$ antes de módulo, convertir primero a `__int128` si puede exceder `long long`.
- Para $\lceil a/b \rceil$ con positivos sirve `(a+b-1)/b`; con negativos usar la función anterior.

1.3.3 Trabajo con ángulos y trigonometría

Usar cuando aparecen triángulos, rotaciones, pendientes, direcciones o coordenadas polares. C++ recibe y devuelve ángulos en **radianes**. Para un triángulo rectángulo:

$$\sin(\theta) = \frac{CO}{H}, \qquad \cos(\theta) = \frac{CA}{H}, \qquad \tan(\theta) = \frac{CO}{CA} = \frac{\sin(\theta)}{\cos(\theta)}.$$

El signo depende del cuadrante:

Cuadrante	rango	x	y	signos positivos
I	$0 < \theta < \pi/2$	$+$	$+$	$\sin, \cos, \tan$
II	$\pi/2 < \theta < \pi$	$-$	$+$	$\sin$
III	$\pi < \theta < 3\pi/2$	$-$	$-$	$\tan$
IV	$3\pi/2 < \theta < 2\pi$	$+$	$-$	$\cos$

Ángulos notables

Grados	Radianes	sin	cos	tan
$0°$	0	0	1	0
$30°$	$\pi/6$	$1/2$	$\sqrt{3}/2$	$1/\sqrt{3}$
$45°$	$\pi/4$	$\sqrt{2}/2$	$\sqrt{2}/2$	1
$60°$	$\pi/3$	$\sqrt{3}/2$	$1/2$	$\sqrt{3}$
$90°$	$\pi/2$	1	0	no definida
$180°$	π	0	-1	0
$270°$	$3\pi/2$	-1	0	no definida
$360°$	2π	0	1	0

Identidades frecuentes

$$\begin{aligned}
\sin^2\theta + \cos^2\theta &= 1, \\
\sin(-\theta) &= -\sin\theta, \qquad & \cos(-\theta) &= \cos\theta, \\
\sin(a+b) &= \sin a\cos b + \cos a\sin b, \\
\cos(a+b) &= \cos a\cos b - \sin a\sin b, \\
\sin(2a) &= 2\sin a\cos a, \\
\cos(2a) &= \cos^2 a - \sin^2 a = 1 - 2\sin^2 a.
\end{aligned}$$

La periodicidad es 2π para seno y coseno, y π para tangente.

Operación	`double`	`long double`
seno, coseno, tangente	`sin/cos/tan`	`sinl/cosl/tanl`
inversas	`asin/acos/atan`	`asinl/acosl/atanl`
ángulo de (x, y)	`atan2(y,x)`	`atan2l(y,x)`
hipotenusa	`hypot(x,y)`	`hypotl(x,y)`

```
const long double PI = acosl(-1.0L);

long double degrees_to_radians(long double degrees) {
    return degrees * PI / 180.0L;
}

long double radians_to_degrees(long double radians) {
    return radians * 180.0L / PI;
}

long double normalize_angle(long double angle) { // [0, 2*pi)
    angle = fmodl(angle, 2.0L * PI);
    if (angle < 0) angle += 2.0L * PI;
    return angle;
}

long double signed_angle(long double angle) { // [-pi, pi)
    angle = normalize_angle(angle);
    if (angle >= PI) angle -= 2.0L * PI;
    return angle;
}

long double safe_acos(long double value) {
    return acosl(clamp(value, -1.0L, 1.0L));
}

long double safe_asin(long double value) {
    return asinl(clamp(value, -1.0L, 1.0L));
}

void trigonometry_example(long double degrees) {
    long double angle = degrees_to_radians(degrees);
    long double sine = sinl(angle);
    long double cosine = cosl(angle);
    long double tangent = tanl(angle);
    long double direction = atan2l(sine, cosine); // primero y, luego
      x
    (void)tangent; (void)direction;
}
```

Conversión entre grados y radianes

Una vuelta completa cumple

$$360° = 2\pi \text{ rad}, \qquad 180° = \pi \text{ rad}.$$

Por lo tanto:

$$rad = grados\frac{\pi}{180}, \qquad grados = rad\frac{180}{\pi}.$$

En C++17 no existe `std::numbers::pi`; la forma portable es `acosl(-1.0L)`. `M_PI` no es estándar y puede no estar definido.

Normalización y distancia angular

El mismo rayo puede representarse con infinitos valores: θ, $\theta + 2\pi$, $\theta - 4\pi$, etc. Normalizar evita errores al comparar direcciones. Para saber cuánto girar, no restar ángulos sin considerar el salto entre 0 y 2π.

```
// Giro antihorario desde 'from' hasta 'to', en [0, 2*pi).
long double counterclockwise_angle(long double from, long double to)
     {
    return normalize_angle(to - from);
}

// Menor distancia entre dos direcciones, en [0, pi].
long double angle_distance(long double first, long double second) {
    return fabsl(signed_angle(second - first));
}

// Giro firmado mas corto desde 'from' hasta 'to', en [-pi, pi).
long double signed_angle_difference(long double from, long double to)
     {
    return signed_angle(to - from);
}

// target pertenece al arco antihorario cerrado start -> finish.
bool angle_between_ccw(long double start, long double target,
                       long double finish, long double epsilon = 1e
     -12L) {
    return counterclockwise_angle(start, target)
        <= counterclockwise_angle(start, finish) + epsilon;
}
```

Ejemplo: entre 350° y 10° hay solamente 20°, no 340°.

`atan2`: dirección y cuadrantes

Para un vector (x, y):

$$r = \sqrt{x^2 + y^2}, \qquad \theta = \operatorname{atan2}(y, x).$$

Siempre escribir `atan2(y,x)`: el argumento vertical va primero. `atan(y/x)` pierde el cuadrante, divide por cero cuando $x = 0$ y no puede distinguir $(1, 1)$ de $(-1, -1)$.

Vector	`atan2(y,x)`	grados
$(1, 0)$	0	$0°$
$(0, 1)$	$\pi/2$	$90°$
$(-1, 0)$	π	$180°$
$(0, -1)$	$-\pi/2$	$-90°$

Cuidado: La dirección del vector $(0, 0)$ no está definida. Algunas implementaciones devuelven cero para `atan2(0,0)`, pero no debe usarse como un ángulo geométrico válido.

Coordenadas polares y rotaciones

Conversión polar a cartesiana:

$$x = r\cos\theta, \qquad y = r\sin\theta.$$

Rotar (x, y) un ángulo θ en sentido antihorario:

$$x' = x\cos\theta - y\sin\theta, \qquad y' = x\sin\theta + y\cos\theta.$$

```
pair<long double,long double> polar_to_cartesian(
        long double radius, long double angle) {
    return {radius * cosl(angle), radius * sinl(angle)};
}

pair<long double,long double> rotate_coordinates(
        long double x, long double y, long double angle) {
    long double cosine = cosl(angle), sine = sinl(angle);
    return {x * cosine - y * sine,
            x * sine + y * cosine};
}

pair<long double,long double> cartesian_to_polar(
        long double x, long double y) {
    return {hypotl(x, y), atan2l(y, x)}; // {radio, angulo}
}
```

Para rotar alrededor de un centro C, trasladar $P - C$, rotar y volver a sumar C. Varias rotaciones suman sus ángulos; una rotación no cambia distancias.

Ángulo entre vectores

Para vectores u y v no nulos:

$$\cos\theta = \frac{u \cdot v}{|u||v|}, \qquad \theta = \operatorname{atan2}(|u \times v|, u \cdot v).$$

La versión con `atan2` suele ser más estable y conserva mejor ángulos cercanos a 0 o π.

```
long double angle_between_vectors(long double ux, long double uy,
                                  long double vx, long double vy) {
    long double dot = ux * vx + uy * vy;
    long double cross_value = ux * vy - uy * vx;
    assert(hypotl(ux, uy) > 0 && hypotl(vx, vy) > 0);
    return atan2l(fabsl(cross_value), dot); // [0, pi]
}

// Angulo orientado u -> v: positivo antihorario, en [-pi, pi].
long double oriented_angle(long double ux, long double uy,
                           long double vx, long double vy) {
    return atan2l(ux * vy - uy * vx, ux * vx + uy * vy);
}
```

Si sólo importa comparar ángulos o saber si son agudos/obtusos, evitar la trigonometría: $u \cdot v > 0$ indica ángulo agudo, $= 0$ recto y < 0 obtuso.

Triángulos cualesquiera

Si los lados a, b, c están opuestos a los ángulos A, B, C:

Ley de cosenos:

$$c^2 = a^2 + b^2 - 2ab\cos C, \qquad \cos C = \frac{a^2 + b^2 - c^2}{2ab}.$$

Ley de senos:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R,$$

donde R es el radio de la circunferencia circunscrita.

Área:

$$s = \frac{a+b+c}{2}, \qquad K = \sqrt{s(s-a)(s-b)(s-c)}.$$

```
long double angle_opposite_side(long double a, long double b,
                                long double opposite) {
    assert(a > 0 && b > 0);
    long double cosine = (a * a + b * b - opposite * opposite)
                       / (2.0L * a * b);
    return safe_acos(cosine);
}

long double opposite_side(long double a, long double b,
                          long double included_angle) {
    long double squared = a * a + b * b
                        - 2.0L * a * b * cosl(included_angle);
    return sqrtl(max(0.0L, squared));
}

bool valid_triangle(long double a, long double b, long double c,
                    long double epsilon = 1e-12L) {
    return a > 0 && b > 0 && c > 0
        && a + b > c + epsilon
        && a + c > b + epsilon
        && b + c > a + epsilon;
}

long double triangle_area_heron(long double a, long double b,
                                long double c) {
    assert(valid_triangle(a, b, c));
    long double semiperimeter = (a + b + c) / 2.0L;
    long double product = semiperimeter * (semiperimeter - a)
                        * (semiperimeter - b) * (semiperimeter - c);
    return sqrtl(max(0.0L, product));
}
```

Cuidado: La ley de senos puede tener dos soluciones cuando se conocen dos lados y un ángulo no comprendido. La ley de cosenos determina directamente el lado opuesto al ángulo comprendido. Herón puede perder precisión en triángulos casi degenerados; con coordenadas, preferir producto cruzado.

Circunferencias, arcos y sectores

Con radio r y ángulo central θ en radianes:

$$\text{arco} = r\theta, \qquad \text{sector} = \frac{1}{2}r^2\theta, \qquad \text{cuerda} = 2r\sin(\theta/2).$$

```
long double arc_length(long double radius, long double angle) {
    return radius * fabsl(angle);
}

long double sector_area(long double radius, long double angle) {
    return radius * radius * fabsl(angle) / 2.0L;
}

long double chord_length(long double radius, long double angle) {
    return 2.0L * radius * sinl(fabsl(angle) / 2.0L);
}
```

Polígonos regulares

Para un polígono regular de $n \geq 3$ lados:

$$\sum \text{interiores} = (n-2)\pi, \quad \text{interior} = \frac{(n-2)\pi}{n}, \quad \text{exterior} = \frac{2\pi}{n}.$$

Si R es el circunradio y s el lado:

$$s = 2R\sin(\pi/n), \qquad \text{área} = \frac{nR^2\sin(2\pi/n)}{2}.$$

Convenciones: matemática, brújula y pantalla

Antes de usar un ángulo, identificar la convención del problema:

- matemática: 0 apunta a $+x$ y crece antihorario;
- rumbo de brújula: 0 apunta al norte y crece horario;
- matrices/pantallas: la fila o coordenada y suele crecer hacia abajo, invirtiendo visualmente la orientación.

```
long double bearing_to_math_angle(long double bearing) {
    return normalize_angle(PI / 2.0L - bearing);
}

long double math_angle_to_bearing(long double angle) {
    return normalize_angle(PI / 2.0L - angle);
}

// Direccion en una grilla donde row crece hacia abajo.
long double screen_direction(long double delta_column,
                             long double delta_row) {
    return atan2l(-delta_row, delta_column);
}

// Una recta no orientada es igual tras girar pi: resultado [0, pi).
long double normalize_line_angle(long double angle) {
    angle = fmodl(angle, PI);
    if (angle < 0) angle += PI;
    return angle;
}
```

Orden polar exacto sin funciones trigonométricas

Para ordenar muchos vectores por ángulo, calcular `atan2` para cada uno es más lento y agrega error flotante. Se puede separar el plano en dos mitades y comparar con producto cruzado. El siguiente orden comienza en $+x$ y avanza antihorario; los colineales se ordenan por longitud.

```
int polar_half(ll x, ll y) {
    return (y < 0 || (y == 0 && x < 0)) ? 1 : 0;
}

bool polar_less(pair<ll,ll> first, pair<ll,ll> second) {
    auto [ax, ay] = first;
    auto [bx, by] = second;
    int half_a = polar_half(ax, ay), half_b = polar_half(bx, by);
    if (half_a != half_b) return half_a < half_b;
    i128 cross_value = (i128)ax * by - (i128)ay * bx;
    if (cross_value != 0) return cross_value > 0;
    i128 length_a = (i128)ax * ax + (i128)ay * ay;
    i128 length_b = (i128)bx * bx + (i128)by * by;
    return length_a < length_b;
}
```

Cuidado: Excluir el vector $(0, 0)$ o decidir explícitamente dónde ubicarlo: no posee dirección. El comparador debe ser estricto; no introducir epsilon en un comparador de `sort`, `set` o `map`.

Proyección, reflexión y bisectriz

La proyección del vector v sobre una dirección no nula d es

$$\text{proj}_d(v) = d\frac{v \cdot d}{d \cdot d}.$$

Si n es una normal de una recta/espejo, la reflexión es

$$\text{reflect}(v) = v - 2\,\text{proj}_n(v).$$

La bisectriz interna de dos vectores no opuestos apunta en la suma de sus versiones unitarias.

```
pair<long double,long double> project_vector(
        long double vx, long double vy,
        long double dx, long double dy) {
    long double norm_squared = dx * dx + dy * dy;
    assert(norm_squared > 0);
    long double scale = (vx * dx + vy * dy) / norm_squared;
    return {dx * scale, dy * scale};
}

pair<long double,long double> reflect_vector(
        long double vx, long double vy,
        long double normal_x, long double normal_y) {
    auto projection = project_vector(vx, vy, normal_x, normal_y);
    return {vx - 2.0L * projection.first,
            vy - 2.0L * projection.second};
}

optional<pair<long double,long double>> internal_angle_bisector(
        long double ax, long double ay,
        long double bx, long double by) {
    long double norm_a = hypotl(ax, ay), norm_b = hypotl(bx, by);
    if (norm_a == 0 || norm_b == 0) return nullopt;
    long double x = ax / norm_a + bx / norm_b;
    long double y = ay / norm_a + by / norm_b;
    long double norm = hypotl(x, y);
    if (norm < 1e-18L) return nullopt; // direcciones opuestas
    return pair<long double,long double>{x / norm, y / norm};
}
```

Para reflejar un punto respecto de una recta, proyectar el vector desde un punto de la recta hasta el punto original y usar la misma identidad. Para la bisectriz externa, restar los vectores unitarios en lugar de sumarlos.

Funciones relacionadas de <cmath>

- `sinh/cosh/tanh`: funciones hiperbólicas; no describen ángulos circulares y aparecen ocasionalmente en fórmulas analíticas.
- `fmod(a,b)` conserva el signo de a; para normalizar ángulos negativos hay que sumar el período, como hace `normalize_angle`.
- `remainder(a,b)` devuelve el residuo respecto del múltiplo más cercano y puede ser útil para obtener un ángulo firmado, aunque su tratamiento de empates suele ser menos intuitivo en competencia.
- `copysign(magnitude,sign)` copia el signo sin bifurcar; `isfinite/isnan/isinf` permiten detectar resultados inválidos.

Rangos de retorno:

- `asin`: $[-\pi/2, \pi/2]$; `acos`: $[0, \pi]$.
- `atan`: $(-\pi/2, \pi/2)$; `atan2(y,x)`: $[-\pi, \pi]$.
- `atan2` distingue los cuatro cuadrantes y funciona cuando $x = 0$; para direcciones casi siempre es mejor que `atan(y/x)`.

Cuidado: Por errores de redondeo, un coseno calculado puede quedar apenas fuera de $[-1, 1]$ y hacer que `acos` devuelva NaN; aplicar `clamp`. La tangente no está definida cuando $\cos(\theta) = 0$.

Checklist de competencia:

- confirmar si la entrada está en grados o radianes;
- usar `atan2(y,x)`, no `atan(y/x)`;
- normalizar antes de comparar direcciones que cruzan $0/2\pi$;
- aplicar `clamp` antes de `asin/acos`;
- evitar trigonometría si dot/cross permiten decidir exactamente;
- usar `long double` y un epsilon acorde a las coordenadas;
- no comparar resultados flotantes con `==`.

Cada función trigonométrica y cada helper de esta sección cuesta $O(1)$ tiempo y $O(1)$ memoria.

1.4 Template Python 3

Usar Python cuando la complejidad tiene margen, predominan enteros grandes, strings, diccionarios o implementación corta. En problemas ajustados de $O(n \log n)$ pesado, grafos enormes o DP con millones de estados, preferir C++.

```
import sys
from collections import Counter, defaultdict, deque
from functools import lru_cache
from heapq import heappop, heappush
from bisect import bisect_left, bisect_right
from math import gcd, isqrt, lcm, inf

input = sys.stdin.buffer.readline

def ints():
    return map(int, input().split())

def solve():
    n = int(input())
    values = list(ints())
    assert len(values) == n
    print(sum(values))

if __name__ == "__main__":
    solve()
```

Atajos importantes:

- `math.isqrt(n)` da $\lfloor\sqrt{n}\rfloor$ exactamente.
- `pow(a,b,mod)` hace potencia modular y `pow(a,-1,mod)` calcula el inverso si existe.
- `sys.setrecursionlimit(...)` sólo amplía el límite lógico; para DFS muy profundo suele ser más seguro usar una pila iterativa.
- Evitar `pop(0)` en listas; usar `deque.popleft()`.
- Usar input().decode() si se toma como b antes del input

1.5 Imports y utilidades de Python

```
import sys
from collections import Counter, defaultdict, deque
from functools import lru_cache
from heapq import heappop, heappush
from bisect import bisect_left, bisect_right
from math import gcd, isqrt, lcm, inf
# sys.setrecursionlimit(1_000_000) no se limite
input = sys.stdin.buffer.readline
```

1.5.1 Entrada rapida

```
n = int(input())
values = list(map(int, input().split()))
```

Lee una linea en $O(L)$, donde L es su cantidad de bytes.

1.5.2 Counter

Cuenta frecuencias.

```
frequency = Counter([2, 1, 2, 3, 2])
print(frequency[2])       # 3
print(frequency.most_common(1)) # [(2, 3)]
```

Construirlo cuesta $O(n)$; consultar una frecuencia cuesta $O(1)$ promedio.

1.5.3 defaultdict

Crea un valor inicial automaticamente para claves nuevas.

```
graph = defaultdict(list)
graph[1].append(4)
graph[1].append(7)
print(graph[1]) # [4, 7]

count = defaultdict(int)
count[5] += 1
print(count[5]) # 1
```

Acceso e insercion: $O(1)$ promedio.

1.5.4 deque

Cola doble: insertar o quitar en ambos extremos.

```
q = deque([2, 3])
q.append(4)
q.appendleft(1)
print(q.popleft()) # 1
print(q.pop())     # 4
```

`append`, `appendleft`, `pop` y `popleft` cuestan $O(1)$.

1.5.5 lru_cache

Memoiza resultados de una funcion recursiva.

```
@lru_cache(None) # None almacena y si pongo nro ultimos x
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

print(fibonacci(10)) # 55
```

Cada estado se calcula una vez. En este ejemplo: $O(n)$ tiempo y $O(n)$ memoria.

1.5.6 heapq

Heap minimo: permite obtener siempre el menor elemento.

```
heap = []
heappush(heap, 5)
heappush(heap, 2)
heappush(heap, 7)

print(heappop(heap)) # 2
print(heappop(heap)) # 5
```

`heappush` y `heappop` cuestan $O(\log n)$. Para un max-heap, guardar los valores negados.

1.5.7 bisect

Busca posiciones en un vector ordenado.

```
values = [1, 2, 2, 2, 5]

print(bisect_left(values, 2))  # 1
print(bisect_right(values, 2)) # 4
print(bisect_left(values, 4))  # 4
```

`bisect_left` busca la primera posicion posible; `bisect_right`, la posterior a las repeticiones. Ambos cuestan $O(\log n)$. Insertar en una lista sigue costando $O(n)$.

1.5.8 Matematica

```
print(gcd(18, 24)) # 6
print(lcm(6, 8))   # 24
print(isqrt(20))   # 4
print(inf > 10**100) # True
a = int("ABCDEF", 16)
b = int("123456", 16)

result = a * b
print(format(result, "X"))
```

`gcd` cuesta $O(\log(\min(a,b)))$; `lcm` tiene costo equivalente. `isqrt(n)` devuelve $\lfloor\sqrt{n}\rfloor$ sin errores de punto flotante. `inf` representa infinito para inicializar minimos o distancias.

1.6 Tipos, límites y overflow

- `int`: aproximadamente $[-2 \cdot 10^9, 2 \cdot 10^9]$.
- `long long`: aproximadamente $[-9 \cdot 10^{18}, 9 \cdot 10^{18}]$.
- `double`: unas 15 cifras decimales de precisión.
- `long double`: precisión dependiente de la plataforma.
- `__int128`: entero no estándar disponible en GCC/Clang.

Promover *antes* de operar:

```
ll promoted_product(int a, int b) { return 1LL * a * b; }
bool square_leq(ll x, ll n) { return x <= n / x; } // x > 0

void print_i128(i128 x) {
    if (x == 0) { cout << 0; return; }
    bool negative = x < 0;
    u128 value = negative ? (u128)(-(x + 1)) + 1 : (u128)x;
    if (negative) cout << '-';
    string s;
    while (value) {
        s.push_back(char('0' + value % 10));
        value /= 10;
    }
    reverse(s.begin(), s.end());
    cout << s;
}
```

El overflow de enteros con signo es comportamiento indefinido en C++.

1.7 Sintaxis C++ más usada

1.7.1 Funciones, referencias y lambdas

Pasar objetos grandes por referencia constante si no se modifican; usar referencia normal cuando la función debe modificarlos.

```
ll sum_values(const vector<int> &values) {
    ll result = 0;
    for (int value : values) result += value;
    return result;
}

void normalize(vector<int> &values) {
    sort(values.begin(), values.end());
    values.erase(unique(values.begin(), values.end()), values.end());
}

int apply_operation(int a, int b, const function<int(int,int)> &
     operation) {
    return operation(a, b);
}

void lambda_example() {
    int offset = 5;
    auto add_offset = [offset](int value) { return value + offset; };
    assert(add_offset(3) == 8);
}
```

1.7.2 Pair, tuple, array y optional

```
void aggregate_types_example() {
    pair<int,string> person{20, "Ada"};
    auto [age, name] = person;

    tuple<int,int,ll> edge{0, 1, 15};
    auto [from, to, weight] = edge;

    array<int, 4> fixed{1, 2, 3, 4};
    optional<int> answer;
    if (!fixed.empty()) answer = fixed.front();
    if (answer) cout << *answer << '\n';

    (void)age; (void)name; (void)from; (void)to; (void)weight;
}
```

Cuidado: Comparar `int` con `size()` mezcla tipos firmados y sin signo. En competencia suele ser más simple convertir el tamaño una vez a `int`.

1.8 STL esencial

Estructura	Operaciones principales	Complejidad
`vector`	acceso / push al final	$O(1)$ / amortizado
`deque`	push/pop en ambos extremos	$O(1)$
`stack, queue`	push/pop/top/front	$O(1)$
`priority_queue`	push/pop/top	$O(\log n)$ / $O(1)$
`set, map`	find/insert/erase por clave	$O(\log n)$
`unordered_*`	find/insert/erase	promedio $O(1)$

```
void stl_examples(vector<ll> &v, ll x) {
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());

    auto lo = lower_bound(v.begin(), v.end(), x); // primero >= x
    auto hi = upper_bound(v.begin(), v.end(), x); // primero > x
    if (lo != v.end()) { ll value = *lo; (void)value; }
    (void)hi;

    priority_queue<int> max_heap;
    priority_queue<int, vector<int>, greater<int>> min_heap;

    ll total = accumulate(v.begin(), v.end(), 0LL);
    (void)max_heap; (void)min_heap; (void)total;
}
```

Cuidado: `reserve(n)` reserva capacidad pero no crea elementos. Un arreglo local suele vivir en el stack; uno global o `static`, no.

1.9 Contenedores comunes

1.9.1 Vector y string

```
void vector_and_string_examples() {
    vector<int> values(5, 0);       // cinco ceros
    values.push_back(7);
    values.pop_back();
    values.resize(10, -1);          // crea elementos
    values.reserve(100);            // solo reserva capacidad
    values.insert(values.begin() + 2, 9); // O(n)
    values.erase(values.begin() + 2);     // O(n)

    string text = "competitive";
    text += " programming";
    string part = text.substr(0, 11);
    size_t position = text.find("program");
    if (position != string::npos) text.replace(position, 7, "contest"
     );
    (void)part;
}
```

1.9.2 Set, multiset, map y unordered

```
void associative_container_examples() {
    set<int> ordered{5, 1, 5}; // {1,5}
    ordered.insert(3);
    bool exists = ordered.count(5) != 0;
    auto first_at_least = ordered.lower_bound(4);
    if (first_at_least != ordered.end()) ordered.erase(first_at_least
     );

    multiset<int> repeated{2, 2, 3};
    auto occurrence = repeated.find(2);
    if (occurrence != repeated.end()) repeated.erase(occurrence); //
     una copia
    repeated.erase(2); // todas las copias restantes de 2

    map<string,int> frequency;
    ++frequency["duck"];
    auto found = frequency.find("duck");
    if (found != frequency.end()) cout << found->second << '\n';

    unordered_map<int,int> fast_frequency;
    fast_frequency.reserve(1000);
    fast_frequency.max_load_factor(0.7);
    ++fast_frequency[10];
    (void)exists;
}
```

1.9.3 Stack, queue, deque y priority queue

```
void queue_examples() {
    stack<int> stack_values;
    stack_values.push(1); stack_values.push(2);
    int top = stack_values.top(); stack_values.pop();

    queue<int> pending;
    pending.push(1); pending.push(2);
    int front = pending.front(); pending.pop();

    deque<int> double_ended;
    double_ended.push_front(1); double_ended.push_back(2);
    double_ended.pop_front(); double_ended.pop_back();

    priority_queue<int> maximum_heap;
```

```
    priority_queue<int, vector<int>, greater<int>> minimum_heap;
    maximum_heap.push(5); minimum_heap.push(5);
    (void)top; (void)front;
}
```

Contenedor	Acceso/búsqueda	Inserción/borrado
`vector` final	$O(1)$	amortizado $O(1)$
`deque` extremos	$O(1)$	$O(1)$
`set/map`	$O(\log n)$	$O(\log n)$
`unordered_*`	promedio $O(1)$	promedio $O(1)$
`priority_queue` top	$O(1)$	$O(\log n)$

1.10 Algoritmos STL comunes

```
void common_algorithms(vector<int> values) {
    sort(values.begin(), values.end());
    sort(values.rbegin(), values.rend());
    stable_sort(values.begin(), values.end()); // conserva el orden
     relativo de elementos que empatan (noo mas rapido si mas memo)

    int minimum = *min_element(values.begin(), values.end());
    int maximum = *max_element(values.begin(), values.end());
    auto [minimum_it, maximum_it] = minmax_element(values.begin(),
     values.end());

    ll sum = accumulate(values.begin(), values.end(), 0LL);
    int amount = count(values.begin(), values.end(), 5); // Cuantas
     veces aparece 5
    int positives = count_if(values.begin(), values.end(),
                             [](int value) { return value > 0; });
    bool all_positive = all_of(values.begin(), values.end(),
                               [](int value) { return value > 0; });

    reverse(values.begin(), values.end());
    rotate(values.begin(), values.begin() + values.size() / 2, values
     .end()); // {1, 2, 3, 4, 5, 6} -> {4, 5, 6, 1, 2, 3}
    (void)minimum; (void)maximum; (void)minimum_it; (void)maximum_it;
    (void)sum; (void)amount; (void)positives; (void)all_positive; //:
      no la uso; es intencional
}
```

1.10.1 Bounds, duplicados y selección

```
void ordered_range_algorithms(vector<int> values, int target) {
    sort(values.begin(), values.end());
    auto first_equal = lower_bound(values.begin(), values.end(),
     target);
    auto after_equal = upper_bound(values.begin(), values.end(),
     target);
    int occurrences = after_equal - first_equal;
    bool exists = binary_search(values.begin(), values.end(), target)
     ;

    values.erase(unique(values.begin(), values.end()), values.end());
    if (!values.empty()) {
        int middle = values.size() / 2;
        nth_element(values.begin(), values.begin() + middle, values.
     end());
        int median = values[middle];
        (void)median;
    }
    (void)occurrences; (void)exists;
}
```

1.10.2 Permutaciones y sumas acumuladas

```
void sequence_algorithms() {
    vector<int> permutation{1, 2, 3};
    do {
        // procesar permutation
    } while (next_permutation(permutation.begin(), permutation.end())
     );

    // partial_sum acumula con el tipo de entrada, no con el de
     prefix.
    // Usar ll; cada suma parcial debe entrar en ese tipo.
    vector<ll> values(5);
    iota(values.begin(), values.end(), 10LL); // 10,11,12,13,14
    vector<ll> prefix(values.size());
    partial_sum(values.begin(), values.end(), prefix.begin());
}
```

Cuidado: `lower_bound`, `upper_bound` y `binary_search` requieren el rango ordenado con el mismo comparador.

1.11 Comparadores, bits y aleatoriedad

1.11.1 Orden personalizado

El comparador debe ser estricto: nunca puede decir que $a < a$. No usar una tolerancia EPS como comparador directo de `set` o `sort`.

```
struct BySecondThenFirst {
    bool operator()(const pair<int,int> &a, const pair<int,int> &b)
     const {
        return tie(a.second, a.first) < tie(b.second, b.first);
    }
};

void comparator_example(vector<pair<int,int>> &items) {
    sort(items.begin(), items.end(), BySecondThenFirst{});
    sort(items.begin(), items.end(), [](const auto &a, const auto &b)
      {
        if (a.first != b.first) return a.first > b.first;
        return a.second < b.second;
    });
}
```

1.11.2 Operaciones de bits

Consulta	oper	NEUT
XOR	^	0
OR	|	0
AND	&	(~0)

```
int count_bits(unsigned long long value) {
    return __builtin_popcountll(value);
} //cuenta cuantos bits estan en 1

int least_significant_bit_index(unsigned long long value) {
    assert(value != 0);
    return __builtin_ctzll(value);
} // devuelve cuantos ceros hay al final, o sea el indice del bit 1
     mas bajo

int most_significant_bit_index(unsigned long long value) {
    assert(value != 0);
    return 63 - __builtin_clzll(value);
} // obtiene el indice del bit 1 mas alto

unsigned long long lowest_set_bit(unsigned long long value) {
    return value & (~value + 1); // tambien value & -value para
     unsigned
} // devuelve solo el bit 1 mas bajo

void bitset_example() {
    bitset<64> bits; // requiere <bitset>; inicialmente todos en 0
    bits.set(5);    // pone el bit 5 en 1
    bits.flip(7);   // invierte el bit 7
    bits.reset(5);  // pone el bit 5 en 0
    bool seventh = bits.test(7); // consulta el bit de indice 7
    size_t active = bits.count(); // cuenta bits en 1
    cout << seventh << '\n'; // 1
    cout << active << '\n';  // 1
}
```

Cuidado: `__builtin_clz(0)` y `__builtin_ctz(0)` son indefinidos. Para `long long`, usar las versiones terminadas en `ll`.

1.11.3 Random y shuffle

```
mt19937_64& random_engine() {
    static mt19937_64 engine(chrono::steady_clock::now()
                             .time_since_epoch().count());
    return engine;
}

ll random_integer(ll left, ll right) {
    return uniform_int_distribution<ll>(left, right)(random_engine())
     ;
}

void shuffle_values(vector<int> &values) {
    shuffle(values.begin(), values.end(), random_engine());
}
```

1.11.4 Errores frecuentes

- `map[key]` y `unordered_map[key]` insertan la clave si no existe; usar `find` para consultar sin modificar.
- No desreferenciar `end()`, ni llamar `front/back/top` sobre un contenedor vacío.
- `erase(value)` en `multiset` borra todas las copias; `erase(find(value))` borra una.
- Un comparador de `sort/set/map` debe cumplir orden estricto.
- Evitar `endl` salvo que se necesite forzar el flush; usar `'\n'`.
- En expresiones modulares, normalizar negativos con $((x \bmod m) + m) \bmod m$.

1.12 Entrada y salida

```
void read_test_cases() {
    int t; cin >> t;
    while (t--) {
        int n; cin >> n;
        vector<ll> a(n);
        for (ll &x : a) cin >> x;
    }
}

void read_until_eof() {
    int a, b;
    while (cin >> a >> b) cout << a + b << '\n';
}

vector<string> read_grid() {
    int n, m; cin >> n >> m;
    vector<string> grid(n);
    for (string &row : grid) cin >> row;
    return grid;
```

```
}

string read_line_after_token() {
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    string line;
    getline(cin, line);
    return line;
}
```

Para salida decimal:

```
void print_decimal(long double answer) {
    cout << fixed << setprecision(10) << answer << '\n';
}
```

1.13 Parsing seguro

```
bool parse_int(const string &s, int &value) {
    if (s.empty()) return false;
    const char *first = s.data();
    const char *last = first + s.size();
    auto [ptr, ec] = from_chars(first, last, value);
    return ec == errc{} && ptr == last;
}

bool is_digit(char c) { return '0' <= c && c <= '9'; }
```

`from_chars` detecta valores fuera del rango y no acepta espacios sobrantes. Para dividir una línea en tokens puede usarse `istringstream`.

1.14 Letras a posiciones

```
char c = 'C';

int zero_indexed = c - 'A';     // A -> 0, B -> 1, ..., Z -> 25
int one_indexed = c - 'A' + 1; // A -> 1, B -> 2, ..., Z -> 26

char lower = 'c';
int lower_indexed = lower - 'a'; // a -> 0, ..., z -> 25
```

```
c = 'C'
zero_indexed = ord(c) - ord('A')      # A -> 0
one_indexed = ord(c) - ord('A') + 1   # A -> 1

lower = 'c'
lower_indexed = ord(lower) - ord('a') # a -> 0
```

2. Técnicas

2.1 Búsqueda binaria

Cuándo usar: Existe una condición monótona falso–verdadero o verdadero–falso y se busca el primer/último valor que la satisface, incluso sobre la respuesta.

Busca la primera posición donde una condición monótona pasa de falsa a verdadera.

Complejidad: Dominio discreto de n candidatos: $O(\log(n+1))$ evaluaciones; si cada chequeo cuesta C, tiempo $O(C\log(n+1))$. Para reales con ancho inicial W y tolerancia ε, $O(\log(1+W/\varepsilon))$ evaluaciones hasta alcanzar la tolerancia o el límite de precisión. Memoria adicional $O(1)$ aparte del chequeo.

```
ll l = start; // start y end son int; los limites pueden salir de ese
      dominio.
ll r = end;
while (l <= r) {
    ll m = l + (r - l) / 2;
    if (can((int)m)) {
        // La condicion se cumple en m, ajustar el limite superior
        r = m - 1;
    } else {
        // La condicion no se cumple en m, ajustar el limite inferior
       l = m + 1;
    }
}
// Esto es en caso de que la condicion inicialmente sea falsa y luego
      verdadera. Si la condicion es verdadero--falso, se debe
     invertir los contenidos del if-else
```

El algoritmo anterior permite usar la busqueda binaria unicamente en un dominio discreto. En este caso r nos da el ultimo valor que no cumple la condicion y l el menor valor que si la cumple. El dominio inicial es de `int`, pero los límites se guardan en `long long`: esto permite calcular el ancho completo y representar `end+1` o `start-1` sin desbordar. Si no hay ningún verdadero, $l = \texttt{end}+1$; si todos son verdaderos, $r = \texttt{start}-1$. Para reales, mantener un intervalo con `can(l)==false` y `can(r)==true`. Usar la tolerancia pedida por el enunciado y cortar si el punto medio coincide con un extremo para evitar ciclos por redondeo.

```
double l = start;
double r = end;
double eps = 1e-6; // Ajustar al error permitido por el enunciado.
while (r - l > eps) {
    double m = l + (r - l) / 2.0;
    if (m == l || m == r) break; // Limite de precision de double.
    if (can(m)) {
        // La condicion se cumple para m, ajustamos el limite
     superior
        r = m;
    } else {
       // La condicion no se cumple para m, ajustamos el limite
     inferior
        l = m;
    }
}
// r cumple la condicion; l no la cumple.
```

El ancho se divide por dos en cada iteración. La precisión final también está limitada por la representación de `double`: el corte por redondeo puede detener la búsqueda antes de alcanzar `eps`. Si se permite error relativo, adaptar la tolerancia a la escala del resultado.

2.2 Sumas parciales

Cuándo usar: Arreglo estático con muchas sumas de rango; también para transformar restricciones de subarreglos y construir diferencias.

Usar un vector de tamaño $n+1$ elimina el caso especial cuando $l = 0$. Los rangos son semiabiertos $[l, r)$.

Complejidad: construcción $O(n)$; consulta $O(1)$; memoria $O(n)$.

```
struct PrefixSum {
    vector<ll> pref;

    explicit PrefixSum(const vector<ll> &a) : pref(a.size() + 1, 0) {
        fore (i,0,SZ(a))
            pref[i + 1] = pref[i] + a[i];
    }

    ll query(int l, int r) const { // suma de [l, r)
        return pref[r] - pref[l];
    }
};
```

En 2D: $P[i+1][j+1] = a[i][j] + P[i][j+1] + P[i+1][j] - P[i][j]$.

2.3 Two pointers y sliding window

Cuándo usar: Subarreglos/pares ordenados con una propiedad monótona al mover los extremos, pudiendo agregar y quitar elementos en $O(1)$.

Si al avanzar el extremo derecho la validez cambia monótonamente, cada puntero avanza a lo sumo n veces.

Con valores no negativos se usa sliding window. Si hay negativos, la segunda variante usa máximos prefijos y búsqueda binaria: una ventana deslizante común no garantiza encontrar la respuesta.

Complejidad: Ventana con valores no negativos y pares sobre un arreglo ordenado: $O(n)$ tiempo y $O(1)$ memoria adicional. Variante con negativos: $O(n \log n)$ tiempo y $O(n)$ memoria adicional.

```
// Longitud maxima con suma <= limit; requiere a[i] >= 0.
// Tiempo O(n), memoria adicional O(1).
int longest_window_nonnegative(const vector<ll> &a, ll limit) {
    ll sum = 0;
    int ans = 0;
    for (int l = 0, r = 0; r < (int)a.size(); ++r) {
        sum += a[r];
        while (l <= r && sum > limit) sum -= a[l++];
        ans = max(ans, r - l + 1);
    }
    return ans;
}

// Pares con suma target en un arreglo ordenado.
bool has_pair_sum(const vector<ll> &a, ll target) {
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        i128 sum = (i128)a[l] + a[r];
        if (sum == target) return true;
        if (sum < target) ++l;
        else --r;
    }
    return false;
}

// Longitud maxima con suma <= limit; admite valores negativos.
// Prefijos y busqueda binaria: tiempo O(n log n), memoria O(n).
int longest_window_signed(const vector<ll> &a, ll limit) {
    vector<ll> maximum_prefix{0};
    vector<int> first_position{0};
    ll prefix = 0;
    int answer = 0;
    for (int r = 0; r < (int)a.size(); ++r) {
        prefix += a[r];
        // Buscamos el menor i con prefix[i] >= prefix[r+1] - limit.
        ll required = prefix - limit;
        auto it = lower_bound(
            maximum_prefix.begin(), maximum_prefix.end(), required
        );
        if (it != maximum_prefix.end()) {
            int k = it - maximum_prefix.begin();
            answer = max(answer, r + 1 - first_position[k]);
        }
        // Guardamos solamente nuevos maximos prefijos.
        if (prefix > maximum_prefix.back()) {
            maximum_prefix.push_back(prefix);
            first_position.push_back(r + 1);
        }
    }
    return answer;
```

```
}
```

2.4 Máscaras de bits

Cuándo usar: Representar subconjuntos pequeños, enumerar submáscaras o guardar flags compactos; suele combinarse con DP exponencial.

Complejidad: operaciones de bits $O(1)$; enumerar todos los subconjuntos $O(2^n)$ y todas las submáscaras de todas las máscaras $O(3^n)$; memoria $O(1)$.

Para evitar desplazamientos indefinidos, usar tipos sin signo y verificar que $0 \le k < 64$.

```
using ull = unsigned long long;

// Se debe asegurar que k este en el rango [0,63]
bool has_bit(ull mask, int k) { return (mask >> k) & 1ULL; }
void add_bit(ull &mask, int k) { mask |= 1ULL << k; }
void remove_bit(ull &mask, int k) { mask &= ~(1ULL << k); }
void toggle_bit(ull &mask, int k) { mask ^= 1ULL << k; }
// Las funciones anteriores funcionan con un unico bit
// Sin embargo tambien se puede trabajar con varios bits al mismo
     tiempo usando lo siguiente
bool has_bits(ull mask, ull bits) { return (ull)(bits & mask) == bits
     ; }
void add_bits(ull &mask, ull bits) { mask |= bits; }
void remove_bits(ull &mask, ull bits) { mask &= ~bits; }
void toggle_bits(ull &mask, ull bits) { mask ^= bits; }

vector<ull> nonempty_submasks(ull mask) {
    vector<ull> answer;
    for (ull sub = mask; sub; sub = (sub - 1) & mask)
        answer.push_back(sub);
    return answer;
}
```

Intersección: `a & b`; unión: `a | b`; diferencia: `a & ~b`; diferencia simétrica: `a ^ b`.

2.5 Conversión a base arbitraria

Cuándo usar: Extraer dígitos, trabajar con representaciones posicionales o convertir un entero no negativo a una base entre 2 y 36.

Complejidad: $O(\log_b n)$ tiempo y memoria para producir los dígitos.

Los dígitos se devuelven como enteros, por lo que admite bases mayores que 36. El signo se maneja por separado.

```
pair<bool, vector<int>> to_base(ll value, int base) {
    if (base < 2) throw invalid_argument("base < 2");
    bool negative = value < 0;
    unsigned long long x = negative
        ? (unsigned long long)(-(value + 1)) + 1
        : (unsigned long long)value;

    vector<int> digits;
    do {
        digits.push_back((int)(x % base));
        x /= base;
    } while (x);
    reverse(digits.begin(), digits.end());
    return {negative, digits};
}
```

```
char digit_to_char(int digit) {
    if (digit < 10) return char('0' + digit);
    return char('A' + digit - 10);
}

string to_base_string(ll value, int base) {
    if (base < 2 || base > 36)
        throw invalid_argument("base fuera de [2,36]");

    auto [negative, digits] = to_base(value, base);

    string result;
    if (negative) result += '-';
    for (int digit : digits)
        result += digit_to_char(digit);

    return result;
}
```

```
int char_to_digit(char c) {
    if ('0' <= c && c <= '9') return c - '0';
    if ('A' <= c && c <= 'Z') return c - 'A' + 10;
    if ('a' <= c && c <= 'z') return c - 'a' + 10;
    return -1;
}

ll from_base(const string& text, int base) {
    ll value = 0;
    for (char c : text) {
        int digit = char_to_digit(c);
        if (digit < 0 || digit >= base)
            throw invalid_argument("digito invalido");
        value = value * base + digit;
    }
    return value;
}
```

2.6 Compresión de coordenadas

Cuándo usar: Los valores son enormes pero sólo importa su orden/igualdad; mapearlos a $[0, k)$ para usar Fenwick, Segment Tree o arreglos.

Reemplaza valores grandes por sus posiciones ordenadas conservando igualdad y orden relativo.

Complejidad: $O(n \log n)$ de construcción; $O(\log n)$ por consulta; memoria $O(n)$.

```
template<class T>
struct Compressor {
    vector<T> values;

    explicit Compressor(vector<T> a) : values(move(a)) {
        sort(values.begin(), values.end());
        values.erase(unique(values.begin(), values.end()), values.end
      ());
    }

    int id(const T &x) const {
        return lower_bound(values.begin(), values.end(), x) - values.
      begin();
    }

    const T& value(int id) const { return values[id]; }
    int size() const { return (int)values.size(); }
};
```

2.7 Descomposición por raíz: frecuentes y poco frecuentes

Cuándo usar: Objetos con frecuencias muy distintas donde procesar uno cuesta $O(f)$ o $O(n/f)$; separar pesados y livianos balancea ambas cotas.

Usar cuando objetos con frecuencia f cuestan $O(f)$ o $O(n/f)$ por consulta. Elegir un umbral $B \approx \sqrt{n}$: procesar directamente los objetos con $f < B$ y precomputar respuestas para los que tienen $f \ge B$. Hay como máximo n/B objetos frecuentes.

Complejidad: típicamente preproceso $O(n\sqrt{n})$, consulta $O(\sqrt{n})$ y memoria entre $O(n)$ y $O(n\sqrt{n})$, según lo precomputado.

```
int sqrt_threshold(int total_items) {
    return max(1, (int)sqrt((long double)total_items) + 1);
}

vector<int> frequent_values(const vector<int> &frequency) {
    int total = accumulate(frequency.begin(), frequency.end(), 0);
    int block = sqrt_threshold(total);
    vector<int> result;
    for (int value = 0; value < (int)frequency.size(); ++value)
        if (frequency[value] >= block) result.push_back(value);
    return result;
}
```

Cuidado: No existe una plantilla universal: escribir primero las dos cotas y verificar que ambas quedan balanceadas en el umbral elegido.

2.8 Meet-in-the-middle: suma de subconjuntos

Cuándo usar: Unos 40 elementos hacen inviable 2^n, pero se pueden enumerar dos mitades y combinar sus sumas. Admite números negativos.

Complejidad: Tiempo $O(n2^{n/2})$ por enumeración, ordenamiento y búsquedas; memoria $O(2^{n/2})$. Con $n = 40$, las dos listas suelen ocupar unos 32 MiB.

```
optional<vector<int>> subset_sum_mitm(const vector<ll> &a, ll target)
     {
    int n = a.size(), middle = n / 2;
    assert(n <= 40);
    auto enumerate = [&](int begin, int length) {
        vector<pair<ll,int>> sums{{0, 0}};
        sums.reserve(1 << length);
        for (int i = 0; i < length; ++i) {
            int size = sums.size();
            for (int mask = 0; mask < size; ++mask)
                sums.push_back({sums[mask].first + a[begin + i],
                                sums[mask].second | (1 << i)});
        }
        return sums;
    };
    auto left = enumerate(0, middle), right = enumerate(middle, n -
     middle);
    sort(right.begin(), right.end());
    for (auto [sum, mask] : left) {
        i128 raw = (i128)target - sum;
        if (raw < numeric_limits<ll>::min() || raw > numeric_limits<
     ll>::max()) continue;
        ll wanted = (ll)raw;
        auto it = lower_bound(right.begin(), right.end(), pair<ll,int
     >{wanted, 0});
        if (it == right.end() || it->first != wanted) continue;
        vector<int> chosen;
        for (int i = 0; i < middle; ++i) if (mask >> i & 1) chosen.
```

```
      push_back(i);
        for (int i = 0; i < n - middle; ++i)
            if (it->second >> i & 1) chosen.push_back(middle + i);
        return chosen;
    }
    return nullopt;
}
void mitm_example() {
    auto chosen = subset_sum_mitm({7,-3,10,5}, 12);
    if (chosen) for (int i : *chosen) cout << i << ' '; // Una
     solucion: 0 3.
}
```

Cuidado: Todas las sumas de subconjuntos deben entrar en `long long`. Con $n = 50$ estas dos listas ocuparían aproximadamente 1 GiB: esta plantilla no escala a miles de elementos sin una reducción adicional.

3. Estructuras de datos

3.1 Disjoint Set Union (DSU)

Cuándo usar: Mantener componentes conexas mientras sólo se agregan conexiones; Kruskal, consultas de conectividad y agrupamiento por equivalencia.

Mantiene componentes bajo uniones. Usa compresión de caminos y unión por tamaño.

Complejidad: $O(\alpha(n))$ amortizado por operación; memoria $O(n)$.

```
struct DSU {
    vector<int> parent; // raiz: -tamano

    explicit DSU(int n) : parent(n, -1) {}

    int find(int x) {
        return parent[x] < 0 ? x : parent[x] = find(parent[x]);
    }

    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (parent[a] > parent[b]) swap(a, b);
        parent[a] += parent[b];
        parent[b] = a;
        return true;
    }

    bool same(int a, int b) { return find(a) == find(b); }
    int size(int x) { return -parent[find(x)]; }
};

DSU dsu(5);
dsu.unite(0, 1); // {0,1} {2} {3} {4}
dsu.unite(1, 2); // {0,1,2} {3} {4}
cout << dsu.same(0, 2) << '\n'; // 1: estan conectados
cout << dsu.same(0, 4) << '\n'; // 0: no estan conectados
cout << dsu.size(1) << '\n';    // 3
```

3.2 Fenwick Tree (BIT)

Cuándo usar: Sumas prefijas o de rango con actualizaciones puntuales; preferirlo a Segment Tree cuando la operación tiene inversa y alcanza esta interfaz.

Convención: la estructura es 1-indexada. Actualización puntual y suma de prefijo/rango.

Cuidado: `lower_bound` requiere que los valores actuales sean no negativos, para que las sumas prefijas sean monótonas.

Complejidad: $O(\log n)$ por operación; memoria $O(n)$.

```
struct Fenwick {
    int n;
    vector<ll> bit;

    explicit Fenwick(int size) : n(size), bit(size + 1, 0) {}

    void add(int i, ll delta) {
        assert(1 <= i && i <= n);
        for (; i <= n; i += i & -i) bit[i] += delta;
    }

    ll prefix_sum(int i) const { // [1, i]
        ll ans = 0;
        i = min(i, n);
        for (; i > 0; i -= i & -i) ans += bit[i];
        return ans;
    }

    ll query(int l, int r) const { // [l, r]
        if (l > r) return 0;
        return prefix_sum(r) - prefix_sum(l - 1);
    }

    // Menor i en [1,n] con prefix_sum(i) >= target.
    // Requiere frecuencias no negativas. Devuelve n+1 si no existe.
    int lower_bound(ll target) const {
        if (target <= 0) return 1;
        int pos = 0;
        ll accumulated = 0;
        int step = 1;
        while ((step << 1) <= n) step <<= 1;
        for (; step; step >>= 1) {
            int next = pos + step;
            if (next <= n && accumulated + bit[next] < target) {
                pos = next;
                accumulated += bit[next];
            }
        }
        return pos + 1;
    }
};
```

Para range-add/point-query, aplicar diferencias: sumar x en l y $-x$ en $r + 1$ (solamente si $r + 1 \leq n$).

Para XOR

```
struct FenwickXor {
    int n;
    vector<ll> bit;

    explicit FenwickXor(int size) : n(size), bit(size + 1, 0) {}

    void add(int i, ll value) {
        assert(1 <= i && i <= n);
        for (; i <= n; i += i & -i)
            bit[i] ^= value;
    }

    ll prefix_xor(int i) const {
        ll answer = 0;
        i = min(i, n);
        for (; i > 0; i -= i & -i)
            answer ^= bit[i];
        return answer;
    }

    ll query(int l, int r) const {
        if (l > r) return 0;
        return prefix_xor(r) ^ prefix_xor(l - 1);
    }
};
```

3.3 Segment Tree iterativo

Cuándo usar: Arreglo dinámico con updates puntuales y consultas de una operación asociativa sobre rangos; también para buscar la primera posición que cumple.

Versión para suma, actualización puntual y consultas semiabiertas $[l, r)$. Para cambiar la consulta, reemplazar las dos líneas de `SUMA` por las dos del caso comentado elegido: `combine` y `NEUTRAL`. No se cambia build, update ni query. El neutro también llena las hojas de relleno y es la respuesta de un rango vacío.

Operaciones comunes

Consulta	`combine(a,b)`	`NEUTRAL`
Minimo	`min`	mayor `ll`
Maximo	`max`	menor `ll`
Suma	`+`	`0`
Producto	`*`	`1`
XOR	`^`	`0`
OR bit a bit	`|`	`0`
AND bit a bit	`&`	`(~0)`
GCD	`gcd`	`0`
LCM	`lcm`	`1`

Complejidad: build $O(n)$; update/query $O(\log n)$; memoria $O(n)$.

```
struct SegmentTree {
    // SUMA (activo): reemplazar ambas lineas por un mismo caso.
    static constexpr ll NEUTRAL = 0;
    static ll combine(ll a, ll b) { return a + b; }
    // MIN: static constexpr ll NEUTRAL = numeric_limits<ll>::max();
    // MIN: static ll combine(ll a, ll b) { return min(a, b); }
    // MAX: static constexpr ll NEUTRAL = numeric_limits<ll>::min();
    // MAX: static ll combine(ll a, ll b) { return max(a, b); }
    // PRODUCTO: static constexpr ll NEUTRAL = 1;
    // PRODUCTO: static ll combine(ll a, ll b) { return a * b; }
    // XOR: static constexpr ll NEUTRAL = 0;
    // XOR: static ll combine(ll a, ll b) { return a ^ b; }
    // OR: static constexpr ll NEUTRAL = 0;
    // OR: static ll combine(ll a, ll b) { return a | b; }
    // AND: static constexpr ll NEUTRAL = ~0LL;
    // AND: static ll combine(ll a, ll b) { return a & b; }
    // GCD: static constexpr ll NEUTRAL = 0;
    // GCD: static ll combine(ll a, ll b) { return gcd(a, b); }
    // LCM: static constexpr ll NEUTRAL = 1;
    // LCM: static ll combine(ll a, ll b) { return lcm(a, b); }

    int n = 1;
    vector<ll> tree;

    explicit SegmentTree(int size) {
        while (n < size) n <<= 1;
        tree.assign(2 * n, NEUTRAL);
    }

    explicit SegmentTree(const vector<ll> &a) : SegmentTree(a.size())
      {
```

```
        copy(a.begin(), a.end(), tree.begin() + n);
        for (int p = n - 1; p; --p)
            tree[p] = combine(tree[p << 1], tree[p << 1 | 1]);
    }

    void set_value(int pos, ll value) {
        int p = pos + n;
        tree[p] = value;
        while (p >>= 1)
            tree[p] = combine(tree[p << 1], tree[p << 1 | 1]);
    }

    ll query(int l, int r) const { // [l, r)
        ll left = NEUTRAL, right = NEUTRAL;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) left = combine(left, tree[l++]);
            if (r & 1) right = combine(tree[--r], right);
        }
        return combine(left, right);
    }
};
```

Cuidado: GCD/LCM requieren C++17 y `<numeric>`; usar valores no negativos representables. Sumas, productos y LCM deben entrar en `ll`; para producto modular adaptar `combine`. El constructor por tamaño llena con el neutro, no necesariamente con cero: para representar un arreglo de ceros usar el constructor con `vector<ll>(size, 0)`.

3.3.1 Elegir entre normal, lazy y persistente

- **Normal:** cambia una posición por update y sólo importa el estado actual. Ejemplo: modificar un precio y consultar el mínimo o la suma de un intervalo. Update/query $O(\log n)$; memoria $O(n)$.
- **Lazy:** un update modifica un intervalo entero y su efecto se puede aplicar al resumen del nodo sin recorrerlo. Ejemplo: sumar v a todos los precios de $[l, r)$ y consultar el mínimo. Con las operaciones de la siguiente sección, update/query $O(\log n)$; memoria $O(n)$.
- **Persistente:** hace falta consultar estados anteriores o crear una versión desde otra. Ejemplo: cambiar un precio en la versión 3 y conservarla para futuras consultas. Con updates puntuales, update/query $O(\log n)$; memoria total $O(n + q \log n)$.

Lazy describe **cómo actualizar rangos**; persistencia describe **cómo conservar versiones**. Se pueden combinar, pero el struct persistente de este handbook sólo tiene asignaciones puntuales. Pedir una respuesta después de cada cambio no exige persistencia si nunca se vuelve a consultar el pasado: Emergency Rations usa lazy.

Antes de elegir: si no hay updates, para sumas suelen alcanzar prefijos y para mínimos/máximos puede convenir Sparse Table. Para sumas con incrementos puntuales, Fenwick suele ser más simple. Segment Tree sirve cuando el resumen de un intervalo se obtiene combinando sus dos partes de forma asociativa; definir también el neutro.

3.3.2 Reconocer el patrón y usar el normal

1. Identificar qué representa cada hoja: posición, valor comprimido o estado de una DP.
2. Definir el resumen que responde la consulta: suma, mínimo, GCD, etc. Debe poder combinarse desde los hijos en tiempo constante para mantener las complejidades indicadas.
3. Verificar qué cambia por operación. Si cambia una hoja, actualizar su camino hasta la raíz. Si cambia un intervalo, evaluar lazy.
4. Preguntar si se necesitan versiones viejas. Si sólo se consulta el presente, sobrescribir los nodos del `SegmentTree` normal.

Ejemplo con la configuración **SUMA** activa. Los rangos son $[l, r)$ y `set_value` asigna el nuevo valor, no suma un incremento. Copiar el cuerpo de esta función dentro del `main` del problema.

```
void example_segment_tree() {
    vector<ll> a = {2, 1, 3, 4};
    SegmentTree st(a);
    cout << st.query(1, 3) << '\n'; // 1 + 3 = 4.

    st.set_value(1, 7); // Actual: [2, 7, 3, 4].
    cout << st.query(1, 3) << '\n'; // 7 + 3 = 10.
    cout << st.query(0, 4) << '\n'; // Total = 16.
}
```

Para consultar mínimos, cambiar `combine` y `NEUTRAL` al caso MIN: las mismas llamadas imprimen 1, 3 y 2. No cambia la forma de actualizar ni de consultar.

3.4 Lazy Segment Tree

Cuándo usar: Actualizaciones y consultas sobre intervalos, por ejemplo sumar en un rango y consultar suma/mínimo/máximo sin tocar cada elemento.

Suma en rango y suma sobre rango. Los intervalos son semiabiertos $[l, r)$. Para consultar mínimos o máximos manteniendo las sumas en rango, reemplazar `NEUTRAL`, `combine` y la línea marcada en `apply` por el mismo caso comentado. No cambiar `push`, build, add ni query. Cada nodo combina sus hijos; `lazy[p]` guarda el incremento pendiente por elemento. El mínimo/máximo aumenta en v, pero la suma en $v(r-l)$.

Complejidad: build $O(n)$; update/query $O(\log n)$; memoria $O(n)$.

```
struct LazySegmentTree {
    // Para obtener el indice de la posicion buscada se debe carmbiar
       y devolver pares. Esto funciona solo para cuando se busca
     posiciones tipo MAX o MIN, no para sumas u operaciones
    // SUMA (activo). MIN/MAX: cambiar estas dos lineas y apply.
    static constexpr ll NEUTRAL = 0;
    // static constexpr pll NEUTRAL = {INF,INF}
    static ll combine(ll a, ll b) { return a + b; }
    // static pll combine(pll a, pll b) { return (a.fst <= b.fst) ? a
       : b; }
    // MIN: static constexpr ll NEUTRAL = numeric_limits<ll>::max();
    // MIN: static ll combine(ll a, ll b) { return min(a, b); }
    // MAX: static constexpr ll NEUTRAL = numeric_limits<ll>::min();
    // MAX: static ll combine(ll a, ll b) { return max(a, b); }

    int n;
    vector<ll> tree, lazy;

    explicit LazySegmentTree(const vector<ll> &a)
        : n(a.size()), tree(4 * max(1, n)), lazy(4 * max(1, n)) {
        if (n) build(1, 0, n, a);
    }

    void build(int p, int l, int r, const vector<ll> &a) {
        if (r - l == 1) { tree[p] = a[l]; return; }
        // if (r - l == 1) { tree[p] = {a[l], l}; return; } // Esto
      es para guardar el indice
        int m = (l + r) / 2;
        build(p << 1, l, m, a);
        build(p << 1 | 1, m, r, a);
        tree[p] = combine(tree[p << 1], tree[p << 1 | 1]);
    }

    void apply(int p, int l, int r, ll value) {
        tree[p] += value * (r - l); // SUMA (activo).
        // tree[p] += value;       // MIN/MAX: reemplaza la anterior.
        // tree[p].fst += value    // MIN/MAS: obteniendo indice
        lazy[p] += value; // Igual para SUMA, MIN y MAX.
    }

    void push(int p, int l, int r) {
        if (lazy[p] == 0 || r - l == 1) return;
        int m = (l + r) / 2;
        apply(p << 1, l, m, lazy[p]);
        apply(p << 1 | 1, m, r, lazy[p]);
        lazy[p] = 0;
    }

    void add(int ql, int qr, ll value) {
        if (n) add(1, 0, n, ql, qr, value);
    }
    void add(int p, int l, int r, int ql, int qr, ll value) {
        if (qr <= l || r <= ql) return;
        if (ql <= l && r <= qr) { apply(p, l, r, value); return; }
        push(p, l, r);
        int m = (l + r) / 2;
        add(p << 1, l, m, ql, qr, value);
        add(p << 1 | 1, m, r, ql, qr, value);
        tree[p] = combine(tree[p << 1], tree[p << 1 | 1]);
    }

    // Si quieres obtener el indice cambia ll por pll en ambas querys
    ll query(int ql, int qr) {
        return n ? query(1, 0, n, ql, qr) : NEUTRAL;
    }
    ll query(int p, int l, int r, int ql, int qr) {
        if (qr <= l || r <= ql) return NEUTRAL;
        if (ql <= l && r <= qr) return tree[p];
        push(p, l, r);
        int m = (l + r) / 2;
        return combine(query(p << 1, l, m, ql, qr),
                       query(p << 1 | 1, m, r, ql, qr));
    }
};
```

Con $n > 0$, `tree[1]` contiene la respuesta global en $O(1)$. `query(l,r)` cuesta $O(\log n)$ y devuelve el neutro si $l = r$. Las sumas de valores y los acumulados lazy deben entrar en `ll`.

Cuidado: Estas variantes conservan la actualización de sumar en rango. No basta cambiar `combine` para consultar GCD, producto u operaciones bit a bit: su resumen no permite aplicar una suma usando sólo este lazy.

Para asignación de rango se necesita un lazy adicional que distinga "sin asignación" de asignar cero.

3.4.1 Reconocer el patrón: costos por umbral

1. **Reordenar sin empeorar.** Si hay varias operaciones y se pide minimizar su cantidad, preguntar si se pueden agrupar por tipo. Demostrar que intercambiarlas conserva la validez y no aumenta el costo; no asumir que siempre conmutan. Si funciona, contar cuántas veces usar cada tipo puede reemplazar la simulación paso a paso.
2. **Fijar un umbral.** Si una operación afecta a todos los objetos y otra a uno solo, fijar x usos de la global y calcular el costo restante. Buscar una función $f(x) = g(x) + \sum_i h_i(x)$, donde cada objeto aporta según su relación con x. Probar que el costo calculado es necesario y que existe una estrategia que lo alcanza.
3. **Acotar los candidatos.** Obtener una solución factible de costo

B. Si $f(x) \geq x$, ningún $x > B$ mejora esa solución. Esta desigualdad, no el tamaño de los valores de entrada, justifica recorrer sólo $[0, B]$. Si no hay una cota pequeña, buscar puntos donde cambia la función y evaluar si se pueden comprimir coordenadas.

4. **Traducir cada cambio a un rango.** Si insertar un objeto suma un mismo valor a $f(x)$ para $x < c$, actualiza el prefijo $[0, c)$; si afecta a $x \geq c$, actualiza un sufijo. Borrar revierte esa contribución. Para sumar sobre intervalos y pedir mínimos/máximos, usar este lazy con el resumen adecuado. Si sólo cambian posiciones individuales, alcanza el Segment Tree puntual. Verificar siempre que el resumen permita aplicar la actualización sin recorrer las hojas.

3.4.2 Ejemplo: Emergency Rations

Una caja tiene c raciones. Cada día se vacía una caja o se quita una ración de todas las no vacías. Tras cada alta/baja, pedir el mínimo de días, sin consumir realmente las raciones.

Con x operaciones globales, cada caja con $c > x$ necesita una operación individual. Hacer primero las globales y luego vaciar esas cajas alcanza el costo (si todas se vacían antes, se omiten los días sobrantes). Así, el **mínimo** buscado es

$$\min_{0 \leq x \leq Q} f(x), \qquad f(x) = x + C(x), \quad C(x) = \#\{\text{cajas con } c > x\}.$$

Hay a lo sumo Q cajas: vaciarlas individualmente cuesta a lo sumo Q, y $f(x) \geq x$ permite descartar $x > Q$. Inicialmente $f(x) = x$. Agregar una caja suma 1 en $[0, \min(c, Q+1))$; quitarla resta 1 allí.

Antes de usar el ejemplo, configurar el struct anterior para **MIN**: `NEUTRAL = numeric_limits<ll>::max()`, `combine(a,b) = min(a,b)` y, en `apply`, `tree[p] += value` sin multiplicar por la longitud. Build y add deben combinar siempre con mínimo. El namespace permite conservar el ejemplo junto al template; al enviar, copiar solamente su `main` y el struct configurado para MIN.

```
namespace emergency_rations_example {
int main() {FIN;
    int Q;
    cin >> Q;

    vector<ll> a(Q + 1);
    iota(a.begin(), a.end(), 0LL);
    LazySegmentTree st(a); // Configurado para MIN.

    for (int i = 0; i < Q; ++i) {
        ll x;
        cin >> x;

        ll c = abs(x);
        int fin = min(c, (ll)Q + 1);
        // [0, fin): candidatos menores que c.
        st.add(0, fin, x > 0 ? 1 : -1);

        cout << st.tree[1]
             << (i + 1 == Q ? '\n' : ' ');
    }
    return 0;
}
}
```

Para `+1 +2 +2 +7 -2 +10 -1 -2`, las respuestas son `1 2 2 3 3 4 3 2`. Cada cambio cuesta $O(\log Q)$ y leer la raíz cuesta $O(1)$; tiempo total $O(Q \log Q)$ y memoria $O(Q)$.

3.5 Segment Tree persistente

Cuándo usar: Conservar versiones históricas tras updates puntuales, responder consultas por versión o calcular el k-ésimo en subarreglos por diferencia de raíces.

Cada actualización crea una versión nueva copiando solamente los nodos del camino modificado. Esta versión hace asignación puntual y suma en $[l, r)$.

Complejidad: build $O(n)$; update/query $O(\log n)$; cada versión usa $O(\log n)$ nodos nuevos; memoria total $O(n + q \log n)$ para q updates.

```
struct PersistentSegmentTree {
    struct Node { int left = -1, right = -1; ll sum = 0; };
    int n;
    vector<Node> nodes;

    explicit PersistentSegmentTree(const vector<ll> &a) : n(a.size())
      {
        assert(n > 0);
        nodes.reserve(2 * n + 20);
    }

    int make_node(Node node) {
        nodes.push_back(node);
        return (int)nodes.size() - 1;
    }

    int build(const vector<ll> &a) { return build(0, n, a); }
    int build(int l, int r, const vector<ll> &a) {
        if (r - l == 1) return make_node({-1, -1, a[l]});
        int m = (l + r) / 2;
        int left = build(l, m, a), right = build(m, r, a);
        return make_node({left, right, nodes[left].sum + nodes[right
     ].sum});
    }

    int set_value(int root, int position, ll value) {
        return set_value(root, 0, n, position, value);
    }

    int set_value(int root, int l, int r, int position, ll value) {
        if (r - l == 1) return make_node({-1, -1, value});
        int m = (l + r) / 2;
        int left = nodes[root].left, right = nodes[root].right;
        if (position < m) left = set_value(left, l, m, position,
     value);
        else right = set_value(right, m, r, position, value);
        return make_node({left, right, nodes[left].sum + nodes[right
     ].sum});
    }

    ll query(int root, int ql, int qr) const {
        return query(root, 0, n, ql, qr);
    }

    ll query(int root, int l, int r, int ql, int qr) const {
        if (qr <= l || r <= ql) return 0;
        if (ql <= l && r <= qr) return nodes[root].sum;
        int m = (l + r) / 2;
        return query(nodes[root].left, l, m, ql, qr)
             + query(nodes[root].right, m, r, ql, qr);
    }
};
```

Guardar cada raíz en un vector: `roots[v]` representa la versión `v`. No modificar nodos existentes.

3.5.1 Cuándo usar persistencia

La señal es que una consulta necesita un **estado anterior**, o que un update puede partir de una versión elegida en lugar de la *última*. Por ejemplo: "suma de $[l, r)$ en la versión v" o "crear una copia de la versión v y cambiar su posición p".

1. Representar cada estado con una raíz. Copiar una versión sin cambios es copiar ese entero, en $O(1)$; no copiar todo el arreglo.
2. Al actualizar, crear nodos sólo en el camino a la hoja modificada. Los subárboles que no cambian se comparten con las versiones anteriores. Por eso cada update cuesta $O(\log n)$ en tiempo y nodos nuevos.
3. Guardar la raíz devuelta: `set_value` no cambia la raíz recibida. Para consultar, pasar explícitamente la raíz de la versión.
4. Revisar la memoria: q updates acumulan $O(n + q \log n)$ nodos. Si no se consulta el pasado ni se crean ramas, usar el normal (o lazy si las actualizaciones son por rango).

3.5.2 Ejemplo: consultar y ramificar versiones

El constructor reserva memoria; **hay que llamar a build** para obtener la primera raíz. Requiere un arreglo no vacío y posiciones válidas. Copiar el cuerpo del ejemplo dentro del `main`.

```
void example_persistent_segment_tree() {
    vector<ll> a = {2, 1, 3, 4};
    PersistentSegmentTree st(a);
    vector<int> roots = {st.build(a)};

    // Version 1 desde version 0: [2, 7, 3, 4].
    roots.push_back(st.set_value(roots[0], 1, 7));
    // Version 2 tambien desde 0: [2, 1, 3, 0].
    roots.push_back(st.set_value(roots[0], 3, 0));

    cout << st.query(roots[0], 0, 4) << '\n'; // 10.
    cout << st.query(roots[1], 0, 4) << '\n'; // 16.
    cout << st.query(roots[2], 0, 4) << '\n'; // 6.
    cout << st.query(roots[1], 1, 3) << '\n'; // 10.
}
```

Las consultas sobre la versión 0 conservan sus resultados aunque se creen versiones nuevas. Para adaptar este struct a mínimos, cambiar la suma por mínimo en build, update y query, y el neutro por infinito. La copia de caminos y el manejo de raíces se conservan.

Otro patrón: versiones por prefijo. En un arreglo estático, crear una versión de frecuencias por cada prefijo, indexada por valores comprimidos. La diferencia entre las frecuencias de las versiones r y l describe el subarreglo $[l, r)$. Permite contar valores en un rango consultando ambas raíces; descendiendo por sus diferencias se puede buscar el k-ésimo menor en $O(\log \sigma)$, con σ valores distintos. Esto requiere construir frecuencias y agregar el descenso para k-ésimo; el struct anterior no incluye ese método. La resta de raíces significa restar sus conteos: no funciona restando mínimos o máximos.

3.6 Sparse Table

Cuándo usar: Arreglo estático y consultas idempotentes como mínimo, máximo o GCD. Elegir Disjoint Sparse Table si la operación no es idempotente.

Para operaciones idempotentes como min, max o gcd. No requiere inversa y no admite actualizaciones.

Esta adaptación del RMQ de KACTL (Johan Sannemo y pajenegod) conserva `SparseTableMin(a)` y las consultas **inclusivas** `query(l,r)`

del handbook. `table[k][i]` guarda el mínimo de 2^k elementos desde i; cada fila reserva sólo las posiciones donde ese bloque entra completo. Para consultar, tomar $k = \lfloor \log_2(r-l+1) \rfloor$ y combinar el bloque que empieza en l con el que termina en r. Cubren todo el rango y pueden superponerse: para mínimo, repetir elementos no cambia el resultado. Por eso no sirve reemplazar mínimo por suma; en ese caso usar Disjoint Sparse Table. El logaritmo se calcula con `__builtin_clz`, sin un arreglo auxiliar.

Complejidad: build $O(n \log n)$; query $O(1)$; memoria $O(n \log n)$.

```
struct SparseTableMin {
    vector<vector<ll>> table;

    explicit SparseTableMin(const vector<ll> &a) : table(1,a) {
        int n = a.size();
        for (int half=1,k=1; half<=n/2; half*=2,++k) {
            int count=n-2*half+1;
            table.emplace_back(count);
            for (int i=0;i<count;++i)
                table[k][i] = min(table[k - 1][i],
                                  table[k - 1][i + half]);
        }
    }

    ll query(int l, int r) const { // Requiere 0 <= l <= r < n.
        assert(l<=r); // Longitud positiva: clz(0) no está definido.
        int k = 31-__builtin_clz((unsigned)(r-l+1));
        return min(table[k][l], table[k][r - (1 << k) + 1]);
    }
};
```

```
SparseTableMin rmq(vector<ll>{5,1,4,2,3});
cout << rmq.query(1,3) << '\n'; // 1: incluye los indices 1, 2 y 3.
cout << rmq.query(2,2) << '\n'; // 4: un solo elemento.
```

Cuidado: El arreglo puede ser vacío al construir, pero toda consulta debe tener índices válidos y rango no vacío. El RMQ original de KACTL usa $[l, r)$; esta interfaz conserva $[l, r]$. Los valores son `long long` y no se modifican al construir. Requiere GCC/Clang para `__builtin_clz`.

3.7 Disjoint Sparse Table

Cuándo usar: Arreglo estático, operación asociativa y muchas consultas de rango en $O(1)$, incluso cuando la operación no es idempotente (suma, producto).

Consultas estáticas $O(1)$ para cualquier operación asociativa, incluso no idempotente y no conmutativa. Los rangos son inclusivos.

Complejidad: build $O(n \log n)$; query $O(1)$; memoria $O(n \log n)$.

```
template<class T, class Op = plus<T>>
struct DisjointSparseTable {
    int n = 0, levels = 0;
    Op op;
    vector<T> values;
    vector<vector<T>> table;

    explicit DisjointSparseTable(const vector<T> &a, Op operation =
     Op())
        : n(a.size()), op(operation), values(a) {
        while ((1 << levels) < max(1, n)) ++levels;
        table.assign(levels, vector<T>(n));
        for (int k = 0; k < levels; ++k) {
            int block = 1 << (k + 1);
            for (int begin = 0; begin < n; begin += block) {
                int mid = min(begin + (block >> 1), n);
                int end = min(begin + block, n);
                if (begin < mid) {
                    table[k][mid - 1] = values[mid - 1];
                    for (int i = mid - 2; i >= begin; --i)
                        table[k][i] = op(values[i], table[k][i + 1]);
                }
                if (mid < end) {
                    table[k][mid] = values[mid];
                    for (int i = mid + 1; i < end; ++i)
                        table[k][i] = op(table[k][i - 1], values[i]);
                }
            }
        }
    }

    T query(int l, int r) const { // [l, r], requiere indices validos
        if (l == r) return values[l];
        if (l > r) swap(l, r);
        int k = 31 - __builtin_clz((unsigned)(l ^ r));
        return op(table[k][l], table[k][r]);
    }
};
```

3.8 Stack y queue monótonas

Cuándo usar: Próximo mayor/menor, spans, histogramas o mínimo/máximo de cada ventana deslizante en tiempo lineal.

```
// Indice del siguiente elemento estrictamente menor; n si no existe.
vector<int> next_smaller(const vector<ll> &a) {
    int n = a.size();
    vector<int> answer(n, n), stack;
    for (int i = 0; i < n; ++i) {
        // Para maximo se cambia por a[i] > a[stack.back()]
        while (!stack.empty() && a[i] < a[stack.back()]) {
            answer[stack.back()] = i;
            stack.pop_back();
        }
        stack.push_back(i);
    }
    return answer;
}

// Minimo de cada ventana de longitud k.
vector<ll> sliding_minimum(const vector<ll> &a, int k) {
    assert(k > 0);
    deque<int> dq;
    vector<ll> answer;
    for (int i = 0; i < (int)a.size(); ++i) {
        while (!dq.empty() && dq.front() <= i - k) dq.pop_front();
        // Para maximo se cambia por a[dq.back()] <= a[i]
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();
        dq.push_back(i);
        if (i + 1 >= k) answer.push_back(a[dq.front()]);
    }
    return answer;
}
```

Complejidad: $O(n)$ tiempo: cada índice entra y sale una sola vez; memoria $O(n)$.

3.9 Ordered Set de PBDS

Cuándo usar: Conjunto ordenado dinámico con consultas por orden: obtener el k-ésimo elemento o contar elementos estrictamente menores que una clave.

Complejidad: insertar, borrar, `find_by_order` y `order_of_key`: $O(\log n)$; memoria $O(n)$.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template<class Key>
using OrderedSet = tree<
    Key,
    null_type,
    less<Key>,
    rb_tree_tag,
    tree_order_statistics_node_update
>;

void ordered_set_example() {
    OrderedSet<int> values;

    values.insert(20);
    values.insert(5);
    values.insert(10);

    cout << *values.find_by_order(0) << '\n'; // 5
    cout << *values.find_by_order(1) << '\n'; // 10
    cout << values.order_of_key(15) << '\n';  // 2
    cout << values.order_of_key(10) << '\n';  // 1

    values.erase(10);
}

// Para repetidos, distinguir cada copia con un ID.
using OrderedMultiset = OrderedSet<pair<int,int>>;
```

- `find_by_order(k)` devuelve un iterador al elemento de índice k, comenzando en cero.
- `order_of_key(x)` cuenta los elementos estrictamente menores que x.
- Si $k \geq size()$, `find_by_order(k)` devuelve `end()`; no debe desreferenciarse.

Cuidado: PBDS no pertenece al estándar de C++; usar solamente si el juez compila con GNU G++ y verificar su disponibilidad durante el contest de práctica.

3.10 Fenwick 2D offline

Cuándo usar: Altas/bajas de puntos y conteos o sumas dentro de rectángulos cuando todas las coordenadas actualizables se conocen antes de procesar.

Cada punto (x, y) tiene un peso: el número que aporta a las sumas. Para contar puntos, agregarlo usa peso $+1$ y eliminarlo peso -1; también puede representar dinero, población u otra cantidad asociada a la coordenada.

Se llama *offline* porque primero se leen todas las operaciones para conocer qué coordenadas podrán actualizarse. Luego se reservan esos puntos, se construye la estructura y se procesan las operaciones en su orden original. No usa archivos. Cada nodo del Fenwick exterior sobre x guarda otro Fenwick con solamente las coordenadas y que puede recibir.

Complejidad: reserva y construcción $O(n \log^2 n)$ en el peor caso; update y consulta $O(\log^2 n)$; memoria $O(n \log n)$.

```
struct OfflineFenwick2D {
    int size_x;
    vector<vector<int>> coordinates_y;
```

```
    vector<vector<ll>> bit;

    explicit OfflineFenwick2D(int x_count)
        : size_x(x_count), coordinates_y(x_count + 1), bit(x_count +
     1) {}

    void reserve_point(int x, int y) {
        for (int index = x + 1; index <= size_x; index += index & -
     index)
            coordinates_y[index].push_back(y);
    }

    void build() {
        for (int index = 1; index <= size_x; ++index) {
            auto &values = coordinates_y[index];
            sort(values.begin(), values.end());
            values.erase(unique(values.begin(), values.end()), values
     .end());
            bit[index].assign(values.size() + 1, 0);
        }
    }

    void add(int x, int y, ll delta) {
        for (int index = x + 1; index <= size_x; index += index & -
     index) {
            const auto &values = coordinates_y[index];
            auto iterator = lower_bound(values.begin(), values.end(),
      y);
            assert(iterator != values.end() && *iterator == y);
            int position = iterator - values.begin() + 1;
            for (int inner = position; inner < (int)bit[index].size()
     ;
                 inner += inner & -inner)
                bit[index][inner] += delta;
        }
    }

    ll prefix_sum(int x_exclusive, int y_exclusive) const {
        ll answer = 0;
        for (int index = x_exclusive; index > 0; index -= index & -
     index) {
            int position = lower_bound(coordinates_y[index].begin(),
                                       coordinates_y[index].end(),
     y_exclusive)
                           - coordinates_y[index].begin();
            for (int inner = position; inner > 0; inner -= inner & -
     inner)
                answer += bit[index][inner];
        }
        return answer;
    }

    ll rectangle_sum(int x1, int y1, int x2, int y2) const {
        return prefix_sum(x2, y2) - prefix_sum(x1, y2)
             - prefix_sum(x2, y1) + prefix_sum(x1, y1);
    }
};
```

3.10.1 Ejemplo de uso

En un mapa aparecen y desaparecen objetos. Se pide contar cuántos hay dentro de un rectángulo. Todas las coordenadas de futuras altas y bajas aparecen en la entrada, por lo que pueden reservarse antes de procesarla.

```
OfflineFenwick2D points(5); // x validas: [0,5)

// Primera pasada: reservar todo punto que pueda actualizarse.
points.reserve_point(2, 5);
points.reserve_point(4, 3);
points.reserve_point(3, 6);
points.build();

// Segunda pasada: procesar las operaciones en orden.
points.add(2, 5, 1); // aparece un objeto
points.add(4, 3, 1);
points.add(3, 6, 1);

// Rectangulo [2,5) x [3,7): contiene los tres objetos.
cout << points.rectangle_sum(2, 3, 5, 7) << '\n'; // 3

points.add(4, 3, -1); // desaparece un objeto
cout << points.rectangle_sum(2, 3, 5, 7) << '\n'; // 2
```

`prefix_sum(x,y)` suma los pesos de los puntos con coordenadas $X < x$ e $Y < y$. Por inclusión–exclusión, `rectangle_sum(x1,y1,x2,y2)` consulta el rectángulo semiabierto $[x_1, x_2) \times [y_1, y_2)$.

Cuidado: Las coordenadas x deben estar comprimidas en $[0, size_x)$. Llamar `reserve_point` para todo punto actualizable y después `build` exactamente una vez. No intentar actualizar una coordenada que no fue reservada.

3.11 Wavelet Tree

Cuándo usar: Arreglo sin updates y muchas consultas sobre subarreglos: k-ésimo menor, frecuencia de un valor o cantidad de elementos $\leq x$.

Cada nodo divide el rango de valores por la mitad. `go_left[i]` guarda cuántos de los primeros i elementos fueron al hijo izquierdo; así un subarreglo $[l, r)$ se traduce a intervalos de los hijos sin perder su orden. Es una alternativa al Segment Tree persistente cuando no se necesitan versiones ni actualizaciones.

Complejidad: build $O(n \log \sigma)$; consulta $O(\log \sigma)$; memoria $O(n \log \sigma)$, donde $\sigma = max - min + 1$.

```
struct WaveletTree {
    int low, high;
    vector<int> go_left;
    unique_ptr<WaveletTree> left_child, right_child;

    template<class Iterator>
    WaveletTree(Iterator from, Iterator to,
                int minimum_value, int maximum_value)
        : low(minimum_value), high(maximum_value) {
        if (from == to || low == high) return;
        int middle = low + (int)(((ll)high - low) / 2);
        auto goes_left = [middle](int value) { return value <= middle
     ; };
        go_left.reserve(to - from + 1);
        go_left.push_back(0);
        for (Iterator iterator = from; iterator != to; ++iterator)
            go_left.push_back(go_left.back() + goes_left(*iterator));
        Iterator pivot = stable_partition(from, to, goes_left);
        left_child = make_unique<WaveletTree>(from, pivot, low,
     middle);
        right_child = make_unique<WaveletTree>(pivot, to, middle + 1,
      high);
    }

    // k empieza en 1; intervalo [left,right).
    int kth(int left, int right, int k) const {
        assert(1 <= k && k <= right - left);
        if (low == high) return low;
        int before_left = go_left[left];
        int inside_left = go_left[right] - before_left;
        if (k <= inside_left)
            return left_child->kth(before_left, go_left[right], k);
        return right_child->kth(left - before_left,
                                right - go_left[right],
                                k - inside_left);
    }

    int count_leq(int left, int right, int value) const {
        if (left >= right || value < low) return 0;
        if (high <= value) return right - left;
        int before_left = go_left[left];
        int before_right = go_left[right];
        return left_child->count_leq(before_left, before_right, value
     )
             + right_child->count_leq(left - before_left,
                                      right - before_right, value);
    }
};
```

3.11.1 Ejemplo de uso

Para $a = [7, 1, 4, 2, 6, 4]$, el subarreglo $[1, 6)$ es $[1, 4, 2, 6, 4]$.

```
void wavelet_example() {
    vector<int> data = {7, 1, 4, 2, 6, 4};
    int minimum = *min_element(data.begin(), data.end());
    int maximum = *max_element(data.begin(), data.end());
    WaveletTree wavelet(data.begin(), data.end(), minimum, maximum);

    cout << wavelet.kth(1, 6, 3) << '\n';       // 4 el k-esimo menor
       de [left,right). k comienza en 1.
    cout << wavelet.count_leq(1, 6, 3) << '\n'; // 2: {1,2} elementos
       de [left,right) son menores o iguales a value

    int frequency_4 = wavelet.count_leq(1, 6, 4)
                    - wavelet.count_leq(1, 6, 3);
    cout << frequency_4 << '\n';                // 2
}
```

Cuidado: El constructor reordena el vector recibido; pasar una copia si hace falta conservar el orden original. Si el rango de valores es enorme, comprimir coordenadas reduce σ.

3.12 Treap implícito

Cuándo usar: Secuencia cuyo largo u orden cambia: insertar/borrar en el medio, invertir, cortar o consultar segmentos. Para un arreglo de largo fijo suele ser más simple un Segment Tree.

El recorrido inorder del árbol es la secuencia. No guarda claves: el tamaño de cada subárbol determina la posición. `split(t,k)` separa los primeros k elementos del resto y `merge(a,b)` concatena dos secuencias. Para operar sobre $[l, r)$ se lo aísla con dos splits y luego se vuelven a unir las tres partes.

Complejidad: split, merge, inserción, borrado, inversión y consulta $O(\log n)$ esperado; memoria $O(n)$.

```
struct ImplicitTreap {
    struct Node {
        ll value, sum;
        unsigned priority;
        int size = 1;
        bool reversed = false;
        Node *left_child = nullptr, *right_child = nullptr;
        Node(ll initial_value, unsigned initial_priority)
            : value(initial_value), sum(initial_value),
```

```
        priority(initial_priority) {}
};

Node *root = nullptr;

static unsigned next_priority() {
    static mt19937 generator(chrono::steady_clock::now()
                             .time_since_epoch().count());
    return generator();
}

static int node_size(Node *node) { return node ? node->size : 0;
 }
static ll node_sum(Node *node) { return node ? node->sum : 0; }
static void push(Node *node) {
    if (!node || !node->reversed) return;
    swap(node->left_child, node->right_child);
    if (node->left_child) node->left_child->reversed ^= true;
    if (node->right_child) node->right_child->reversed ^= true;
    node->reversed = false;
}
static void pull(Node *node) {
    node->size = 1 + node_size(node->left_child)
                   + node_size(node->right_child);
    node->sum = node->value + node_sum(node->left_child)
                            + node_sum(node->right_child);
}

static void split(Node *node, int left_size,
                  Node *&left_tree, Node *&right_tree) {
    if (!node) { left_tree = right_tree = nullptr; return; }
    push(node);
    if (node_size(node->left_child) >= left_size) {
        split(node->left_child, left_size,
              left_tree, node->left_child);
        right_tree = node; pull(right_tree);
    } else {
        int remaining = left_size - node_size(node->left_child) -
  1;
        split(node->right_child, remaining,
              node->right_child, right_tree);
        left_tree = node; pull(left_tree);
    }
}

static Node *merge(Node *left_tree, Node *right_tree) {
    if (!left_tree || !right_tree)
        return left_tree ? left_tree : right_tree;
    push(left_tree); push(right_tree);
    if (left_tree->priority > right_tree->priority) {
        left_tree->right_child = merge(left_tree->right_child,
 right_tree);
        pull(left_tree); return left_tree;
    }
    right_tree->left_child = merge(left_tree, right_tree->
 left_child);
    pull(right_tree); return right_tree;
}

void insert(int position, ll value) {
    assert(0 <= position && position <= node_size(root));
    Node *left, *right;
    split(root, position, left, right);
    Node *node = new Node(value, next_priority());
    root = merge(merge(left, node), right);
}

void erase(int position) {
    assert(0 <= position && position < node_size(root));
    Node *left, *middle, *right;
    split(root, position, left, middle);
    split(middle, 1, middle, right);
    delete middle;
    root = merge(left, right);
}

void reverse_range(int left, int right) { // [left,right)
    assert(0 <= left && left <= right && right <= node_size(root)
 );
    Node *prefix, *middle, *suffix;
    split(root, right, middle, suffix);
    split(middle, left, prefix, middle);
    if (middle) middle->reversed ^= true;
    root = merge(merge(prefix, middle), suffix);
}

ll range_sum(int left, int right) { // [left,right)
    assert(0 <= left && left <= right && right <= node_size(root)
 );
    Node *prefix, *middle, *suffix;
    split(root, right, middle, suffix);
    split(middle, left, prefix, middle);
    ll answer = node_sum(middle);
    root = merge(merge(prefix, middle), suffix);
    return answer;
}
};
```

3.12.1 Ejemplo de uso

```
void implicit_treap_example() {
    ImplicitTreap sequence;
    sequence.insert(0, 10); // [10]
    sequence.insert(1, 20); // [10,20]
    sequence.insert(2, 30); // [10,20,30]
    sequence.insert(1, 5);  // [10,5,20,30]

    cout << sequence.range_sum(1, 4) << '\n'; // 55
    sequence.reverse_range(0, 3);             // [20,5,10,30]
    cout << sequence.range_sum(0, 2) << '\n'; // 25
    sequence.erase(1);                        // [20,10,30]
}
```

La prioridad aleatoria mantiene el árbol balanceado en esperanza. La bandera `reversed` es lazy: invertir un segmento sólo marca su raíz y el cambio se propaga cuando se baja por ese subárbol.

Cuidado: Esta versión reserva nodos con `new`. Si se crean y destruyen muchas secuencias, agregar un destructor o usar un pool de memoria. No copiar la estructura directamente: dos copias compartirían los mismos punteros.

3.13 Convex Hull Trick y Li Chao Tree

Cuándo usar: Una DP o consulta debe calcular repetidamente $\min_j(m_j x + b_j)$. Usar CHT con deque si pendientes y consultas son monótonas; Li Chao si llegan en cualquier orden.

Cada estado anterior j aporta una recta $y = m_j x + b_j$ y cada nuevo estado consulta el menor valor en x_i. Esto reemplaza probar todos los j, que suele ser $O(n^2)$. La plantilla incluida es Li Chao para mínimos: acepta rectas y consultas desordenadas sobre un dominio entero conocido $[L, R]$.

Complejidad: CHT monótono: inserción amortizada $O(1)$ y consulta $O(1)$; Li Chao: $O(\log X)$ por operación y $O(q \log X)$ memoria en versión dinámica.

```
struct LiChaoTree {
    struct Line {
        ll slope = 0, intercept = (1LL << 62);
        ll evaluate(ll x) const { return slope * x + intercept; }
    };
    struct Node { Line line; int left_child = -1, right_child = -1;
     };
    ll domain_left, domain_right;
    vector<Node> nodes = vector<Node>(1);

    LiChaoTree(ll left_bound, ll right_bound)
        : domain_left(left_bound), domain_right(right_bound) {
        assert(domain_left <= domain_right);
    }

    int make_node() {
        nodes.push_back(Node{});
        return (int)nodes.size() - 1;
    }
    void add_line(Line added) {
        add_line(0, domain_left, domain_right, added);
    }
    void add_line(int node, ll left, ll right, Line added) {
        ll middle = left + (ll)(((i128)right - left) / 2);
        bool better_left =
            added.evaluate(left) < nodes[node].line.evaluate(left);
        bool better_middle =
            added.evaluate(middle) < nodes[node].line.evaluate(middle
     );
        if (better_middle) swap(added, nodes[node].line);
        if (left == right) return;
        if (better_left != better_middle) {
            if (nodes[node].left_child == -1)
                nodes[node].left_child = make_node();
            add_line(nodes[node].left_child, left, middle, added);
        } else {
            if (nodes[node].right_child == -1)
                nodes[node].right_child = make_node();
            add_line(nodes[node].right_child, middle + 1, right,
     added);
        }
    }
    ll query(ll x) const {
        assert(domain_left <= x && x <= domain_right);
        return query(0, domain_left, domain_right, x);
    }
    ll query(int node, ll left, ll right, ll x) const {
        ll answer = nodes[node].line.evaluate(x);
        if (left == right) return answer;
        ll middle = left + (ll)(((i128)right - left) / 2);
        int next = x <= middle ? nodes[node].left_child
                               : nodes[node].right_child;
        if (next == -1) return answer;
        if (x <= middle) return min(answer, query(next, left, middle,
      x));
        return min(answer, query(next, middle + 1, right, x));
    }
};
```

3.13.1 Ejemplo de uso

Se agregan $y = 2x + 3$, $y = -x + 10$ e $y = 5$ y se consulta el mínimo.

```
void li_chao_example() {
    LiChaoTree lines(0, 10);
    lines.add_line({2, 3});   // 2x+3
    lines.add_line({-1, 10}); // -x+10
```

```
    lines.add_line({0, 5});   // 5

    cout << lines.query(2) << '\n'; // min(7,8,5)  = 5
    cout << lines.query(5) << '\n'; // min(13,5,5) = 5
    cout << lines.query(8) << '\n'; // min(19,2,5) = 2
}
```

Cuidado: Puede desbordar $mx + b$; usar `__int128` si los límites lo requieren y elegir un infinito mayor que toda respuesta válida. Para máximos, negar rectas/respuestas o invertir las comparaciones.

3.14 Intervalos disjuntos (ODT)

Cuándo usar: El arreglo está formado por pocos intervalos constantes y predominan asignaciones de rango; apropiado con fragmentación controlada o datos aleatorios.

Usar para mantener una partición ordenada en intervalos constantes y aplicar asignaciones sobre rangos. Funciona muy bien con datos aleatorios o cuando cada operación crea pocos cortes; no garantiza buen peor caso.

Complejidad: `split` $O(\log I)$; una operación cuesta $O((I_{borrados} + 1)\log I)$; memoria $O(I)$. Peor caso total cuadrático.

```
struct DisjointIntervals {
    struct Interval { int right; mutable ll value; };
    map<int, Interval> intervals; // left -> {right,value}, rangos [
     left,right)

    auto split(int position) {
        auto iterator = prev(intervals.upper_bound(position));
        if (iterator->first == position) return iterator;
        int right = iterator->second.right;
        ll value = iterator->second.value;
        if (position >= right) return intervals.end();
        iterator->second.right = position;
        return intervals.emplace(position, Interval{right, value}).
     first;
    }

    void assign(int left, int right, ll value) {
        if (left >= right) return; // Rango vacio: no modificar la
     particion.
        auto after = split(right);
        auto first = split(left);
        intervals.erase(first, after);
        intervals[left] = {right, value};
    }
};
odt.assign(100, 500, 1);
odt.assign(300, 800, 2);
odt.assign(700, 900, 3);
```

Cuidado: Inicializar la partición cubriendo todo el dominio y operar dentro de sus límites. Asignar un rango vacío no hace nada. No usar ODT si un adversario puede fragmentar los intervalos en cada operación.

4. Grafos

4.1 BFS, DFS y bipartición

Cuándo usar: Explorar componentes, construir padres/profundidades, detectar ciclos o colorear un grafo; BFS da distancias mínimas cuando todas las aristas pesan uno.

Para recuperar caminos, activar los comentarios de `Reconstruccion`: agrega $O(V)$ memoria; reconstruir cuesta $O(V)$. Los inalcanzables valen -1.

```
vector<int> bfs_distances(const vector<vector<int>> &graph, int
     source
    // , vector<int>& parent // Reconstruccion: parametro de salida.
) {
    int n = graph.size();
    vector<int> distance(n, -1);
    // parent.assign(n, -1); // Reconstruccion: sin padre
     inicialmente.
    queue<int> pending;
    distance[source] = 0;
    pending.push(source);
    while (!pending.empty()) {
        int v = pending.front(); pending.pop();
        for (int to : graph[v]) if (distance[to] == -1) {
            distance[to] = distance[v] + 1;
            // parent[to] = v; // Reconstruccion: guardar predecesor.
            pending.push(to);
        }
    }
    return distance;
}

vector<int> undirected_components(const vector<vector<int>> &graph) {
    int n = graph.size(), component_count = 0;
    vector<int> component(n, -1), stack;
    for (int start = 0; start < n; ++start) if (component[start] ==
     -1) {
        component[start] = component_count++;
        stack.push_back(start);
        while (!stack.empty()) {
            int v = stack.back(); stack.pop_back();
            for (int to : graph[v]) if (component[to] == -1) {
                component[to] = component[v];
                stack.push_back(to);
            }
        }
    }
    return component;
}

bool is_bipartite(const vector<vector<int>> &graph) {
    int n = graph.size();
    vector<int> color(n, -1);
    for (int start = 0; start < n; ++start) if (color[start] == -1) {
        queue<int> q;
        color[start] = 0; q.push(start);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int to : graph[v]) {
                if (color[to] == -1) color[to] = color[v] ^ 1, q.push
     (to);
                else if (color[to] == color[v]) return false;
            }
        }
    }
    return true;
}
```

Complejidad: $O(V + E)$ tiempo y $O(V)$ memoria.

Ejemplo opcional; requiere activar las tres líneas de reconstrucción:

```
// void bfs_path_example() {
//     vector<vector<int>> graph = {{1,2}, {3}, {3}, {}, {}};
//     vector<int> parent;
//     int source = 0, target = 3;
//     auto distance = bfs_distances(graph, source, parent);
//     vector<int> path;
//     if (distance[target] != -1) {
//         for (int v = target; v != -1; v = parent[v]) path.
     push_back(v);
//         reverse(path.begin(), path.end());
//     }
//     for (int v : path) cout << v << ' '; // 0 1 3
// }
```

4.2 BFS multiorigen

Cuándo usar: Distancia al origen más cercano o expansión simultánea desde muchas fuentes en un grafo sin pesos.

Usar cuando se pide la distancia al origen más cercano, expansión simultánea, Voronoi sobre un grafo no pesado o el instante en que llega primero alguna fuente. Es una BFS normal inicializando la cola con todas las fuentes a distancia cero. Los comentarios opcionales agregan padres y `owner[v]`, la fuente más cercana. En empates gana la primera de `sources`; los inalcanzables tienen distancia y fuente -1. Sin fuentes, todos quedan inalcanzables.

Complejidad: tiempo $O(n + m)$; memoria $O(n)$.

```
vector<int> multi_source_bfs(const vector<vector<int>> &graph,
                             const vector<int> &sources
    // , vector<int>& parent, vector<int>& owner // Reconstruccion y
     fuente.
) {
    int node_count = (int)graph.size();
    vector<int> distance(node_count, -1);
    // parent.assign(node_count, -1); owner.assign(node_count, -1);
    queue<int> pending;
    for (int source : sources) if (distance[source] == -1) {
        distance[source] = 0;
        // owner[source] = source; // Cada fuente se pertenece a si
     misma.
        pending.push(source);
    }
    while (!pending.empty()) {
        int vertex = pending.front(); pending.pop();
        for (int to : graph[vertex]) if (distance[to] == -1) {
            distance[to] = distance[vertex] + 1;
            // parent[to] = vertex; owner[to] = owner[vertex];
            pending.push(to);
        }
    }
    return distance;
}
```

Activar las cuatro líneas comentadas agrega $O(V)$ memoria. Recuperar un camino hasta la fuente ganadora cuesta $O(V)$:

```
// void multi_source_path_example() {
//     vector<vector<int>> graph = {{1}, {0,2}, {1,3}, {2,4}, {3}};
//     vector<int> parent, owner;
//     auto distance = multi_source_bfs(graph, {0,4}, parent, owner);
//     int target = 2;
//     cout << owner[target] << '\n'; // 0: empate ganado por la
     primera fuente.
//     vector<int> path;
//     if (distance[target] != -1) {
//         for (int v = target; v != -1; v = parent[v]) path.
     push_back(v);
//         reverse(path.begin(), path.end());
```

```
//     }
//     for (int v : path) cout << v << ' '; // 0 1 2
// }
```

4.3 Orden topológico

Cuándo usar: Procesar dependencias de un DAG, hacer DP en grafo dirigido o detectar ciclos comprobando si se ordenaron todos los vértices.

Kahn devuelve un orden solamente si el grafo dirigido es un DAG.

```
vector<int> topological_sort(const vector<vector<int>> &graph) {
    int n = graph.size();
    vector<int> indegree(n, 0), order;
    for (int v = 0; v < n; ++v)
        for (int to : graph[v]) ++indegree[to];

    queue<int> q;
    for (int v = 0; v < n; ++v) if (indegree[v] == 0) q.push(v);
    while (!q.empty()) {
        int v = q.front(); q.pop();
        order.push_back(v);
        for (int to : graph[v]) if (--indegree[to] == 0) q.push(to);
    }
    if ((int)order.size() != n) return {}; // existe un ciclo
    return order;
}
```

Complejidad: $O(V + E)$ tiempo y memoria.

4.4 Componentes fuertemente conexas (Tarjan)

Cuándo usar: Contraer ciclos de un grafo dirigido, obtener su DAG de componentes o resolver problemas de alcanzabilidad y 2-SAT.

Tarjan encuentra las componentes en una sola DFS, sin construir el grafo transpuesto. Esta adaptación de KACTL (Lukas Polacek) conserva el estado dentro de la función y devuelve `component[v]`, sin globales ni callback obligatorio.

`discovery[v]` guarda inicialmente el orden de visita; al regresar del DFS se reutiliza para el mínimo alcanzable. La pila contiene los vértices todavía sin componente. El DFS devuelve el menor orden alcanzable dentro de esa zona activa; si coincide con el de `v`, se desapila una componente completa. Las aristas hacia componentes ya cerradas se ignoran.

Tarjan las cierra en orden topológico inverso. Al terminar se invierten los IDs para conservar el contrato del handbook: si existe una arista $u \to v$ entre componentes distintas, `component[u] < component[v]`. Los IDs son consecutivos desde cero; el grafo vacío devuelve un vector vacío. Esto mantiene compatible la comparación usada por 2-SAT.

```
vector<int> strongly_connected_components(
        const vector<vector<int>> &graph) {
    int n = graph.size();
    vector<int> discovery(n),stack;
    vector<int> component(n, -1);
    int timer=0,count=0;
    auto dfs = [&](auto &&self,int v) -> int {
        int low=discovery[v]=++timer;
        stack.push_back(v);
        for (int to : graph[v]) if (component[to]==-1)
            low=min(low,discovery[to]?discovery[to]:self(self,to));
        if (low==discovery[v]) {
            int u;
            do {
                u=stack.back(); stack.pop_back();
                component[u]=count;
            } while (u!=v);
            ++count;
        }
        return discovery[v]=low;
    };
    for (int v=0;v<n;++v) if (component[v]==-1) dfs(dfs,v);
    for (int &id : component) id=count-1-id; // Orden topológico del
      handbook.
    return component;
}
```

Complejidad: $O(V + E)$ tiempo y $O(V)$ memoria adicional, incluida la pila recursiva; la lista de adyacencia de entrada ocupa $O(V + E)$.

Cuidado: La DFS recursiva puede agotar la pila en grafos muy profundos, como una cadena larga; en ese caso usar una implementación iterativa.

4.5 2-SAT

Cuándo usar: Elegir verdadero/falso para cada variable bajo restricciones de dos literales: "al menos uno", incompatibilidades e implicaciones.

Cada variable tiene dos nodos: falso $2v$ y verdadero $2v+1$. La negación se obtiene con XOR 1. Una cláusula $(a \vee b)$ equivale a las implicaciones $(\neg a \Rightarrow b)$ y $(\neg b \Rightarrow a)$.

Complejidad: $O(V + E)$ tiempo y memoria usando SCC sobre $2V$ literales.

```
struct TwoSAT {
    int variables;
    vector<vector<int>> implication;

    explicit TwoSAT(int variable_count)
        : variables(variable_count),
          implication(2 * variable_count) {}

    // Nodo que representa (variable == value).
    int literal(int variable, bool value) const {
        return 2 * variable + (value ? 1 : 0);
    }

    void add_implication(int from, int to) {
        implication[from].push_back(to);
    }

    // (x == value_x) OR (y == value_y).
    void add_or(int x, bool value_x, int y, bool value_y) {
        int a = literal(x, value_x);
        int b = literal(y, value_y);
        add_implication(a ^ 1, b);
        add_implication(b ^ 1, a);
    }

    // Obliga a que variable == value.
    void force(int variable, bool value) {
        add_or(variable, value, variable, value);
    }

    // (x == value_x) implica (y == value_y).
    void imply(int x, bool value_x, int y, bool value_y) {
        add_or(x, !value_x, y, value_y);
    }

    // Exactamente una de las dos condiciones debe cumplirse.
    void add_xor(int x, bool value_x, int y, bool value_y) {
        add_or(x, value_x, y, value_y);
        add_or(x, !value_x, y, !value_y);
    }

    // x e y deben tener el mismo valor.
    void add_equal(int x, int y) {
        imply(x, true, y, true);
        imply(y, true, x, true);
    }

    // x e y deben tener valores diferentes.
    void add_different(int x, int y) {
        add_xor(x, true, y, true);
    }

    optional<vector<bool>> solve() const {
        vector<int> component =
            strongly_connected_components(implication);

        vector<bool> assignment(variables);
        for (int variable = 0; variable < variables; ++variable) {
            int is_false = literal(variable, false);
            int is_true = literal(variable, true);

            if (component[is_false] == component[is_true])
                return nullopt;

            // SCC del handbook: IDs en orden topologico.
            assignment[variable] =
                component[is_true] > component[is_false];
        }
        return assignment;
    }
};
```

Operaciones frecuentes:

```
TwoSAT sat(3);
int A = 0, B = 1, C = 2;
sat.add_or(A, true, B, true);  // A o B
sat.force(C, true);            // C debe ser verdadero
sat.imply(A, true, C, true);   // A => C
sat.add_xor(A, true, B, true); // Exactamente uno: A o B
sat.add_equal(A, C);           // A == C
sat.add_different(B, C);       // B != C
auto answer = sat.solve();
if (!answer) { cout << "Imposible\n";
} else {
    for (bool value : *answer)
        cout << value << ' ';
    cout << '\n';
}
```

Cuidado: La SCC del handbook invierte los IDs internos de Tarjan para devolverlos en orden topológico; por eso `solve` conserva la comparación `>`. Si se copia otra SCC que devuelve orden topológico inverso, usar `<`. También se requiere `#include <optional>`.

4.6 Dijkstra

Cuándo usar: Caminos mínimos desde una fuente con pesos no negativos. Para pesos 0/1 usar 0–1 BFS; con negativos, Bellman–Ford.

Distancias desde una fuente con pesos no negativos. Para reconstruir caminos, descomentar las líneas marcadas con `Reconstruccion` y el ejemplo opcional. Guardar padres no cambia el orden del tiempo de ejecución y agrega $O(V)$ de memoria; reconstruir un camino cuesta $O(V)$ tiempo y memoria en el peor caso.

```
vector<ll> dijkstra(const vector<vector<pair<int,ll>>> &graph, int
     source
    // , vector<int>& parent // Reconstruccion: parametro de salida.
) {
    int n = graph.size();
    vector<ll> distance(n, LINF);
    // parent.assign(n, -1); // Reconstruccion: sin padre
      inicialmente.
    priority_queue<pair<ll,int>, vector<pair<ll,int>>,
                   greater<pair<ll,int>>> pending;
    distance[source] = 0;
    pending.push({0, source});

    while (!pending.empty()) {
        auto [dist_v, v] = pending.top(); pending.pop();
        if (dist_v != distance[v]) continue;
        for (auto [to, weight] : graph[v]) {
            if (weight < 0) throw invalid_argument("negative edge");
            if (distance[to] > dist_v + weight) {
                distance[to] = dist_v + weight;
                // parent[to] = v; // Reconstruccion: guardar
     predecesor.
                pending.push({distance[to], to});
            }
        }
    }
    return distance;
}
// Ejemplo de uso: grafo dirigido con pesos no negativos.
vector<vector<pair<int,ll>>> graph(4);
graph[0] = {{1, 4}, {2, 1}};
graph[2] = {{1, 2}};
graph[1] = {{3, 1}};
auto distance = dijkstra(graph, 0);
cout << distance[3] << '\n'; // 4

// Ejemplo opcional de reconstruccion (reemplaza la llamada anterior)
    :
// vector<int> parent;
// auto distance = dijkstra(graph, 0, parent);
// int source = 0, target = 3;
// vector<int> path;
// if (distance[target] != LINF) { // Si es inalcanzable, queda vacio
    .
//     for (int v = target; v != -1; v = parent[v])
//         path.push_back(v);
//     reverse(path.begin(), path.end());
// }
// for (int v : path) cout << v << ' '; // 0 2 1 3
// Si target == source, path contiene solo source.
```

Complejidad: $O((V+E)\log V)$; memoria $O(V+E)$.

4.7 0–1 BFS

Cuándo usar: Caminos mínimos cuando cada arista pesa exactamente 0 o 1; evita el factor logarítmico de Dijkstra usando una deque.

Para pesos exclusivamente cero o uno. Para reconstruir caminos, activar las tres líneas de `Reconstruccion`: agrega $O(V)$ memoria y recuperar un camino cuesta $O(V)$.

```
vector<int> zero_one_bfs(const vector<vector<pair<int,int>>> &graph,
                         int source
    // , vector<int>& parent // Reconstruccion: parametro de salida.
) {
    int n = graph.size();
    vector<int> distance(n, INF);
    // parent.assign(n, -1); // Reconstruccion: sin padre
      inicialmente.
    deque<int> q;
    distance[source] = 0; q.push_front(source);
    while (!q.empty()) {
        int v = q.front(); q.pop_front();
        for (auto [to, weight] : graph[v]) {
            assert(weight == 0 || weight == 1);
            if (distance[to] > distance[v] + weight) {
                distance[to] = distance[v] + weight;
                // parent[to] = v; // Reconstruccion: guardar
     predecesor.
                if (weight == 0) q.push_front(to);
                else q.push_back(to);
            }
        }
    }
    return distance;
}
vector<vector<pair<int,int>>> graph(4);
graph[0].push_back({1, 0}); // 0 -> 1, costo 0
graph[0].push_back({2, 1}); // 0 -> 2, costo 1
graph[1].push_back({2, 0}); // 1 -> 2, costo 0
graph[2].push_back({3, 1}); // 2 -> 3, costo 1

vector<int> distance = zero_one_bfs(graph, 0);
cout << distance[3] << '\n'; // 1
```

Complejidad: $O(V+E)$ tiempo y memoria.

Ejemplo opcional; `INF` indica un destino inalcanzable:

```
// void zero_one_bfs_path_example() {
//     vector<vector<pair<int,int>>> graph(4);
//     graph[0] = {{1,0}, {2,1}}; graph[1] = {{2,0}}; graph[2] =
     {{3,1}};
//     vector<int> parent;
//     int source = 0, target = 3;
//     auto distance = zero_one_bfs(graph, source, parent);
//     vector<int> path;
//     if (distance[target] != INF) {
//         for (int v = target; v != -1; v = parent[v]) path.
     push_back(v);
//         reverse(path.begin(), path.end());
//     }
//     for (int v : path) cout << v << ' '; // 0 1 2 3; costo 1.
// }
```

4.8 Bellman–Ford

Cuándo usar: Caminos mínimos con pesos negativos y detección de ciclos negativos alcanzables; sólo para grafos moderados por su costo cuadrático.

Admite pesos negativos. Tras $V-1$ rondas, una relajación adicional indica un ciclo negativo alcanzable desde la fuente. Usa `i128` internamente, sin recortar distancias durante la detección. Devuelve `{distancias,false}` o `{{},true}` si hay ciclo negativo alcanzable. Los inalcanzables valen `LINF`; lanza `overflow_error` si una distancia finita no entra en `long long` o coincide con esa marca. Para reconstruir, descomentar las líneas marcadas `Reconstruccion` y el ejemplo opcional: agrega $O(V)$ memoria; recuperar un camino cuesta $O(V)$.

```
struct WeightedEdge { int from, to; ll weight; };

pair<vector<ll>, bool> bellman_ford(
        int n, const vector<WeightedEdge> &edges, int source
        // , vector<int>& parent // Reconstruccion: parametro de
     salida.
) {
    assert(n > 0 && 0 <= source && source < n);
    // parent.assign(n, -1); // Reconstruccion: sin padre
      inicialmente.
    const i128 INF128 = (i128)1 << 126;
    vector<i128> distance(n, INF128);
    distance[source] = 0;
    for (int iteration = 0; iteration < n - 1; ++iteration) {
        bool changed = false;
        for (auto [from, to, weight] : edges) {
            if (distance[from] == INF128) continue;
            i128 candidate = distance[from] + weight;
            if (candidate < distance[to]) {
                distance[to] = candidate;
                // parent[to] = from; // Reconstruccion: guardar
     predecesor.
                changed = true;
            }
        }
        if (!changed) break;
    }
    for (auto [from, to, weight] : edges)
        if (distance[from] != INF128 && distance[from] + weight <
     distance[to])
            return {{}, true};
    vector<ll> result(n, LINF);
    for (int v = 0; v < n; ++v) if (distance[v] != INF128) {
        if (distance[v] < numeric_limits<ll>::min() ||
            distance[v] > numeric_limits<ll>::max() || distance[v] ==
      LINF)
            throw overflow_error("distance outside output range");
        result[v] = (ll)distance[v];
    }
    return {std::move(result), false};
}

// Ejemplo opcional: activar antes las tres lineas de Reconstruccion.
// vector<WeightedEdge> edges = {{0,1,4}, {0,2,5}, {1,2,-2},
     {2,3,3}};
// vector<int> parent;
// int source = 0, target = 3;
// auto [distance, negative_cycle] = bellman_ford(4, edges, source,
     parent);
// vector<int> path;
// if (!negative_cycle && distance[target] != LINF) {
//     for (int v = target; v != -1; v = parent[v]) path.push_back(v)
     ;
//     reverse(path.begin(), path.end());
// }
// for (int v : path) cout << v << ' '; // 0 1 2 3, costo 5
// Si no hay ciclo y target == source, path contiene solo source.
```

Complejidad: $O(VE)$ tiempo y $O(V)$ memoria.

Cuidado: Requiere vértices 0-indexados y pesos `long long`. No usar las distancias si el indicador de ciclo es verdadero.

Ejemplo de manejo de salida: cada arista entra en `long long`, pero la suma de dos costos máximos no. El `catch` imprime `overflow`.

```
void bellman_ford_output_example() {
    ll weight = numeric_limits<ll>::max();
    vector<WeightedEdge> edges = {{0,1,weight}, {1,2,weight}};
    try {
        auto [distance, negative_cycle] = bellman_ford(3, edges, 0);
        if (negative_cycle) cout << "Ciclo negativo\n";
        else if (distance[2] == LINF) cout << "Inalcanzable\n";
        else cout << distance[2] << '\n';
    } catch (const overflow_error&) {
        cout << "overflow\n";
    }
}
```

4.9 Floyd–Warshall

Cuándo usar: Floyd–Warshall se usa cuando necesitás caminos mínimos entre muchos o todos los pares de vértices y el grafo es pequeño. Todos contra todos. Inicializar diagonal en cero, aristas en su peso mínimo y el resto en `LINF`. Usar pesos y distancias finitas estrictamente entre $-$`LINF` y `LINF`; alcanza con $V \max|w| <$ `LINF`. Al detectar una diagonal negativa se activa la saturación por debajo en $-$`LINF`, antes de procesar ese vértice como intermediario. Evita que los ciclos negativos hagan decrecer los valores hasta desbordar y conserva el bucle sin saturación mientras no se detecta un ciclo negativo. Un par afectado por un ciclo negativo no tiene distancia mínima finita: consultar `negative_cycle_between`, no interpretar su valor numérico. Para reconstruir, activar las líneas comentadas: `next[a][b]` guarda el siguiente vértice hacia b. Agrega $O(V^2)$ memoria; recuperar un camino cuesta $O(V)$, incluido el chequeo de ciclos negativos.

```
void floyd_warshall(vector<vector<ll>> &distance
    // , vector<vector<int>>& next // Reconstruccion: parametro de
     salida.
) {
    int n = distance.size();
    bool negative_cycle = false;
    // next.assign(n, vector<int>(n, -1)); // Reconstruccion:
     inicializar saltos.
    // for (int i = 0; i < n; ++i) for (int j = 0; j < n; ++j)
    //     if (distance[i][j] != LINF) next[i][j] = j;
    for (int k = 0; k < n; ++k) {
        negative_cycle = negative_cycle || distance[k][k] < 0;
        auto relax = [&](bool saturate) {
            for (int i = 0; i < n; ++i) if (distance[i][k] != LINF)
                for (int j = 0; j < n; ++j) if (distance[k][j] !=
     LINF) {
                    ll candidate = distance[i][k] + distance[k][j];
                    if (saturate) candidate = max(-LINF, candidate);
                    // if (candidate < distance[i][j])
                    //     next[i][j] = next[i][k]; // Reconstruccion
     : primer salto.
                    distance[i][j] = min(distance[i][j], candidate);
                }
        };
        if (negative_cycle) relax(true);
        else relax(false);
    }
}

bool negative_cycle_between(const vector<vector<ll>> &d, int a, int b
     ) {
    for (int k = 0; k < (int)d.size(); ++k)
        if (d[a][k] != LINF && d[k][k] < 0 && d[k][b] != LINF)
            return true;
    return false;
}

// vector<int> floyd_path(const vector<vector<ll>>& distance,
//                        const vector<vector<int>>& next, int a, int
      b) {
//     if (distance[a][b] == LINF || negative_cycle_between(distance,
     a, b))
//         return {}; // Inalcanzable o sin minimo finito.
//     vector<int> path{a};
//     while (a != b) { a = next[a][b]; path.push_back(a); }
//     return path; // Si a == b y no hay ciclo negativo, contiene
     solo a.
// }
```

Complejidad: $O(V^3)$ tiempo y $O(V^2)$ memoria.

Ejemplo opcional de reconstrucción; activar también los comentarios anteriores:

```
// void floyd_path_example() {
//     vector<vector<ll>> distance(4, vector<ll>(4, LINF));
//     for (int v = 0; v < 4; ++v) distance[v][v] = 0;
//     distance[0][1] = 4; distance[0][2] = 5;
//     distance[1][2] = -2; distance[2][3] = 3;
//     vector<vector<int>> next;
//     floyd_warshall(distance, next);
//     if (negative_cycle_between(distance, 0, 3)) cout << "Ciclo
     negativo\n";
//     else if (distance[0][3] == LINF) cout << "Inalcanzable\n";
//     else for (int v : floyd_path(distance, next, 0, 3)) cout << v
     << ' ';
//     // Imprime 0 1 2 3; costo 5.
// }
```

4.10 Árbol de expansión mínima (Kruskal)

Cuándo usar: Conectar todos los vértices con costo total mínimo. Kruskal es cómodo con lista de aristas; Prim con adyacencias y grafos densos/implícitos.

Kruskal acepta pesos negativos. Si usa menos de $n-1$ aristas, el grafo es disconexo y el resultado es un bosque de expansión mínima.

```
struct MstEdge { int u, v; ll weight; };

pair<ll, vector<MstEdge>> kruskal(int n, vector<MstEdge> edges) {
    sort(edges.begin(), edges.end(), [](const MstEdge &a, const
     MstEdge &b) {
        return a.weight < b.weight;
    });
    DSU dsu(n);
    ll cost = 0;
    vector<MstEdge> chosen;
    for (const auto &edge : edges) if (dsu.unite(edge.u, edge.v)) {
        cost += edge.weight;
        chosen.push_back(edge);
    }
    return {cost, chosen};
}
```

Complejidad: Kruskal $O(E \log E)$ tiempo y $O(V+E)$ memoria.

Prim es conveniente cuando el grafo ya está en listas de adyacencia.

```
optional<ll> prim(const vector<vector<pair<int,ll>>> &graph) {
    int n = graph.size();
    if (n == 0) return 0LL;
    vector<char> used(n, false);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>,
                   greater<pair<ll,int>>> q;
    q.push({0, 0});
    ll cost = 0;
    int visited = 0;
    while (!q.empty()) {
        auto [weight, v] = q.top(); q.pop();
        if (used[v]) continue;
        used[v] = true; ++visited; cost += weight;
        for (auto [to, edge_weight] : graph[v])
            if (!used[to]) q.push({edge_weight, to});
    }
    if (visited != n) return nullopt;
    return cost;
}
```

Complejidad: Prim $O(E \log V)$ tiempo y $O(V+E)$ memoria.

4.11 Puentes y puntos de articulación

Cuándo usar: Detectar una arista o vértice cuya eliminación aumenta el número de componentes de un grafo no dirigido.

Versión para multigrafos no dirigidos: identifica la arista padre por ID, no solamente por vértice. La DFS usa una pila explícita en memoria dinámica y admite cadenas largas.

```
struct LowLink {
    int timer = 0;
    vector<vector<pair<int,int>>> graph; // (to, edge_id)
    vector<int> tin, low;
    vector<char> articulation;
    vector<pair<int,int>> bridges;

    explicit LowLink(int n) : graph(n), tin(n, -1), low(n),
                              articulation(n, false) {}

    void add_edge(int u, int v, int id) {
        graph[u].push_back({v, id});
        graph[v].push_back({u, id});
    }

    void run() {
        struct Frame { int v, parent_edge, next = 0, children = 0; };
        vector<Frame> pending;
        for (int root = 0; root < (int)graph.size(); ++root) {
            if (tin[root] != -1) continue;
            tin[root] = low[root] = timer++;
            pending.push_back({root, -1});
            while (!pending.empty()) {
                auto &frame = pending.back();
                int v = frame.v;
                if (frame.next < (int)graph[v].size()) {
                    auto [to, id] = graph[v][frame.next++];
                    if (id == frame.parent_edge) continue;
                    if (tin[to] != -1) low[v] = min(low[v], tin[to]);
                    else {
                        ++frame.children;
                        tin[to] = low[to] = timer++;
                        pending.push_back({to, id});
                    }
                } else {
                    if (frame.parent_edge == -1 && frame.children >
     1)
                        articulation[v] = true;
                    pending.pop_back();
```

```
                if (pending.empty()) continue;
                int p = pending.back().v;
                low[p] = min(low[p], low[v]);
                if (low[v] > tin[p]) bridges.push_back({p, v});
                if (pending.back().parent_edge != -1 && low[v] >=
      tin[p])
                    articulation[p] = true;
            }
        }
    }
};

LowLink lowlink(5);

int id = 0;
lowlink.add_edge(0, 1, id++); // 5 veces con los q correspondan
     conexiones
lowlink.run();
for (auto [u, v] : lowlink.bridges)
    cout << "Puente: " << u << ' ' << v << '\n';
for (int v = 0; v < 5; ++v)
    if (lowlink.articulation[v])
        cout << "Articulacion: " << v << '\n';
```

Complejidad: $O(V + E)$ tiempo y $O(V + E)$ memoria.

4.12 Componentes biconexas por vértices

Cuándo usar: Separar un grafo no dirigido en bloques que no se desconectan al borrar un solo vértice; construir el block-cut tree o localizar articulaciones.

Una componente biconexa por vértices es un bloque maximal que no puede separarse eliminando uno de sus vértices. Los puntos de articulación pueden pertenecer a varios bloques; una arista puente forma por sí sola un bloque de dos vértices.

Por ejemplo:

```
0---1
 \ /
  2---3
      / \
     4---5
```

produce los bloques $\{0, 1, 2\}$, $\{2, 3\}$ y $\{3, 4, 5\}$. Los vértices 2 y 3 son articulaciones porque aparecen en más de un bloque.

Durante el DFS se apilan las aristas recorridas. Si para un hijo `to` se cumple

`low[to] >= discovery[vertex]`,

su subárbol no puede alcanzar un antecesor de `vertex` sin pasar por él; las aristas de la pila hasta `vertex-to` forman un bloque completo.

Complejidad: Esta implementación: tiempo $O(n + m \log(m + 1))$ y memoria $O(n + m)$. El DFS es lineal, pero ordenar los extremos de las aristas de cada bloque para eliminar repetidos agrega el factor logarítmico.

```
struct VertexBiconnectedComponents {
    struct StackedEdge { int from, to, id; };

    int timer = 0;
    vector<int> discovery, low;
    vector<StackedEdge> edge_stack;
    vector<vector<int>> components;
    const vector<vector<pair<int,int>>> &graph; // (to, edge_id)

    explicit VertexBiconnectedComponents(
            const vector<vector<pair<int,int>>> &input_graph)
        : discovery(input_graph.size(), -1),
          low(input_graph.size()), graph(input_graph) {
        for (int vertex = 0; vertex < (int)graph.size(); ++vertex) {
            if (discovery[vertex] != -1) continue;
            if (graph[vertex].empty()) {
                discovery[vertex] = low[vertex] = timer++;
                components.push_back({vertex});
            } else {
                dfs(vertex, -1);
            }
        }
    }

    void extract_component(int stop_edge) {
        vector<int> vertices;
        while (true) {
            auto edge = edge_stack.back();
            edge_stack.pop_back();
            vertices.push_back(edge.from);
            vertices.push_back(edge.to);
            if (edge.id == stop_edge) break;
        }
        sort(vertices.begin(), vertices.end());
        vertices.erase(unique(vertices.begin(), vertices.end()),
                       vertices.end());
        components.push_back(move(vertices));
    }

    void dfs(int vertex, int parent_edge) {
        discovery[vertex] = low[vertex] = timer++;

        for (auto [to, edge_id] : graph[vertex]) {
            if (edge_id == parent_edge) continue;

            if (discovery[to] == -1) {
                edge_stack.push_back({vertex, to, edge_id});
                dfs(to, edge_id);
                low[vertex] = min(low[vertex], low[to]);

                if (low[to] >= discovery[vertex])
                    extract_component(edge_id);
            } else if (discovery[to] < discovery[vertex]) {
                edge_stack.push_back({vertex, to, edge_id});
                low[vertex] = min(low[vertex], discovery[to]);
            }
        }
    }
};
```

Ejemplo de uso:

```
vector<vector<pair<int,int>>> graph(n);
for (int id = 0; id < m; ++id) {
    int u, v;
    cin >> u >> v;
    graph[u].push_back({v, id});
    graph[v].push_back({u, id});
}
VertexBiconnectedComponents bcc(graph);
for (const auto &component : bcc.components) {
    for (int vertex : component) cout << vertex << ' ';
    cout << '\n';
}
```

Un vértice es de articulación si aparece en más de un bloque:

```
vector<int> appearances(n);
for (const auto &component : bcc.components)
    for (int vertex : component)
        ++appearances[vertex];

for (int vertex = 0; vertex < n; ++vertex)
    if (appearances[vertex] > 1)
        cout << vertex << " es articulacion\n";
```

Cuidado: Cada arista no dirigida debe usar el mismo `edge_id` en ambas direcciones. Los vértices de articulación aparecen en varias componentes; esto es intencional.

4.13 Ancestro común más bajo (LCA)

Cuándo usar: Consultar ancestros comunes y distancias en un árbol enraizado; versión con arreglos de tamaño fijo.

Complejidad: Preproceso $O(NK)$ y consulta $O(K)$, con $K = O(\log N)$; memoria $O(K2^K)$ por los arreglos y $O(N)$ por la recursión. `F[k][v]` guarda el ancestro 2^k de v y `D[v]` su profundidad. Un DFS calcula padres y profundidades; luego se construyen los saltos. Para consultar, igualar profundidades y subir ambos nodos, desde el salto mayor, mientras sus ancestros sean distintos. La distancia es $D[x] + D[y] - 2D[\text{lca}(x, y)]$. Elegir K con $2^K \geq N$; un árbol muy profundo puede agotar la pila del DFS.

```
const int K = 10;
vector<int> g[1<<K]; int N; // K de tal manera que 2^K>=N
int F[K][1<<K], D[1<<K]; // F[i][j] es el padre 2^i del nodo j
// F[i][j] donde 'i' representa la potencia de dos en la que
//se encuentra el padre de 'j'

void lca_dfs(int x){
    for(int hijo : g[x]){
        // Si es el padre, se continua
        if(hijo==F[0][x]) continue;
         // Se marca x como el padre del hijo
        F[0][hijo]=x;
        // La distancia del hijo es la del padre mas 1
        D[hijo]=D[x]+1;
        // Se recorren los hijos del nodo hijo
        lca_dfs(hijo);
    }
}

void lca_init(){
    D[0]=0;F[0][0]=-1;
    lca_dfs(0); // Se recorre el arbol
    // Para cada potencia de 2 y para cada nodo
    fore(k,1,K) fore(x,0,N){
        // Si el padre que esta en la potencia 'k-1' no existe,
        // entonces el padre en la potencia k, tampoco existe
        if(F[k-1][x]<0) F[k][x]=-1;
        // Si no, el padre es el padre 'k-1' del padre 'k-1' de x
        else F[k][x] = F[k-1][F[k-1][x]];
    }
}

int lca(int x, int y){
    // Nos aseguramos de que el x sea el de mayor profundidad
    if (D[x]<D[y]) swap(x,y);
```

```
    // Buscamos el padre del nodo x que esta en la misma
    // profundidad que y.
    for(int k=K-1;k>=0;--k) if(D[x]-(1<<k)>=D[y]) x=F[k][x];
    if (x==y) return x; // Si son el mismo nodo, devolvemos x
    // Buscamos los primeros padres distintos de ambos nodos
    for(int k=K-1;k>=0;--k) if(F[k][x]!=F[k][y]) x=F[k][x], y=F[k][y
     ];
    return F[0][x]; // Devolvemos su padre.
}
```

4.14 LCA con binary lifting

Cuándo usar: Encontrar el ancestro común más cercano, calcular distancias o subir k niveles en un árbol enraizado.

El LCA de dos vértices es el punto donde se unen sus caminos hacia la raíz. Se precomputa `up[k][v]`, el ancestro ubicado 2^k niveles arriba de v, para realizar saltos de $1, 2, 4, 8, \ldots$ niveles.

Para un árbol sin pesos:

$$dist(a, b) = depth[a] + depth[b] - 2\,depth[LCA(a, b)].$$

Complejidad: Preproceso $O(n \log n)$; consulta y salto $O(\log n)$; memoria $O(n \log n)$.

```
struct LCA {
    int n, levels;
    vector<int> depth;
    vector<vector<int>> up; // up[k][v]: ancestro 2^k de v
    LCA(const vector<vector<int>> &tree, int root = 0)
        : n(tree.size()) {
        assert(n > 0 && 0 <= root && root < n);
        levels = 1;
        while ((1 << levels) <= n) ++levels;
        depth.assign(n, 0);
        vector<int> parent(n, -1);
        // Enraizar el arbol y calcular padres y profundidades.
        queue<int> q;
        parent[root] = root;
        q.push(root);
        while (!q.empty()) {
            int v = q.front();
            q.pop();

            for (int to : tree[v]) {
                if (parent[to] != -1) continue;
                parent[to] = v;
                depth[to] = depth[v] + 1;
                q.push(to);
            }
        }
        // Construir la tabla de saltos.
        up.assign(levels, vector<int>(n));
        up[0] = parent;
        for (int k = 1; k < levels; ++k)
            for (int v = 0; v < n; ++v)
                up[k][v] = up[k - 1][up[k - 1][v]];
    }
    // Sube distance niveles desde v.
    int jump(int v, int distance) const {
        for (int k = 0; k < levels; ++k)
            if (distance & (1 << k))
                v = up[k][v];
        return v;
    }
    // Ancestro comun mas cercano de a y b.
    int query(int a, int b) const {
        // Igualar profundidades.
        if (depth[a] < depth[b]) swap(a, b);
        a = jump(a, depth[a] - depth[b]);
        if (a == b) return a;
        // Subir ambos sin alcanzar todavia el LCA.
        for (int k = levels - 1; k >= 0; --k) {
            if (up[k][a] != up[k][b]) {
                a = up[k][a];
                b = up[k][b];
            }
        }
        return up[0][a];
    }
    // Cantidad de aristas en el camino entre a y b.
    int distance(int a, int b) const {
        int ancestor = query(a, b);
        return depth[a] + depth[b] - 2 * depth[ancestor];
    }
};
vector<vector<int>> tree(n);
auto add_edge = [&](int u, int v) {
    tree[u].push_back(v);
    tree[v].push_back(u);
};
add_edge(0, 1);
add_edge(0, 2);
add_edge(1, 3);
add_edge(1, 4);
LCA lca(tree, 0);
cout << lca.query(3, 4) << '\n';    // 1
cout << lca.jump(3, 2) << '\n';     // 0
cout << lca.distance(3, 2) << '\n'; // 3
```

Cuidado: El grafo debe ser un árbol conectado. `jump(v,k)` supone $0 \le k \le depth[v]$; la distancia devuelta cuenta aristas.

4.15 LCA $O(1)$ con Euler tour y RMQ

Cuándo usar: Árbol estático con muchísimas consultas de LCA. Usar binary lifting si también se necesita subir k niveles o ahorrar memoria.

El Euler tour guarda un vértice al visitarlo y cada vez que el DFS regresa a él. El LCA de a y b es el vértice de menor profundidad entre sus primeras apariciones en el recorrido. Una Sparse Table responde este mínimo en $O(1)$. La DFS usa marcos explícitos para registrar las entradas y los regresos, sin consumir pila de llamadas proporcional a la altura.

Complejidad: Preproceso $O(n \log n)$; consulta $O(1)$; memoria $O(n \log n)$.

```
struct EulerRMQLCA {
    vector<int> first, depth, euler, logarithm;
    vector<vector<int>> table;
    EulerRMQLCA(const vector<vector<int>> &tree, int root = 0)
        : first(tree.size(), -1), depth(tree.size()) {
        assert(!tree.empty());
        dfs(root, -1, tree);
        int length = euler.size();
        logarithm.assign(length + 1, 0);
        for (int i = 2; i <= length; ++i)
            logarithm[i] = logarithm[i / 2] + 1;
        // table[k][i]: vertice de menor profundidad
        // en euler[i .. i + 2^k).
        table.push_back(euler);

        for (int k = 1; (1 << k) <= length; ++k) {
            int half = 1 << (k - 1);
            int count = length - (1 << k) + 1;
            table.push_back(vector<int>(count));
            for (int i = 0; i < count; ++i) {
                int a = table[k - 1][i];
                int b = table[k - 1][i + half];
                table[k][i] = depth[a] < depth[b] ? a : b;
            }
        }
    }
    void dfs(int v, int parent,
             const vector<vector<int>> &tree) {
        struct Frame { int v, parent, next = 0; };
        vector<Frame> pending{{v, parent}};
        first[v] = euler.size();
        euler.push_back(v);
        while (!pending.empty()) {
            auto &frame = pending.back();
            int from = frame.v;
            if (frame.next == (int)tree[from].size()) {
                pending.pop_back();
                if (!pending.empty()) euler.push_back(pending.back().
     v);
            } else {
                int to = tree[from][frame.next++];
                if (to == frame.parent) continue;
                depth[to] = depth[from] + 1;
                first[to] = euler.size(); euler.push_back(to);
                pending.push_back({to, from});
            }
        }
    }
    int lca(int a, int b) const {
        int left = first[a], right = first[b];
        if (left > right) swap(left, right);
        int k = logarithm[right - left + 1];
        int x = table[k][left];
        int y = table[k][right - (1 << k) + 1];

        return depth[x] < depth[y] ? x : y;
    }
    int distance(int a, int b) const {
        int ancestor = lca(a, b);
        return depth[a] + depth[b] - 2 * depth[ancestor];
    }
};

EulerRMQLCA lca(tree, 0);
cout << lca.lca(3, 4) << '\n';      // Ancestro comun
cout << lca.distance(3, 2) << '\n'; // Distancia en aristas
```

Cuidado: El grafo debe ser un árbol conectado no vacío. Los marcos de DFS se guardan en memoria dinámica; no se necesita aumentar el límite de pila.

4.16 Virtual Tree

Cuándo usar: Cada consulta marca pocos vértices de un árbol grande y sólo importa la unión de los caminos que los conectan.

El árbol virtual comprime el árbol original conservando únicamente los vértices marcados y los LCA necesarios para conectarlos. Una arista virtual (u, v) representa el camino completo desde el ancestro u hasta v en el árbol original.

Por ejemplo, si los marcados son a, b, c, puede ser necesario agregar

$LCA(a,b)$ para conectarlos. No se agregan los vértices intermedios que no cambian la estructura de los caminos.

Requiere precomputar `tin`, `tout` y LCA. La función devuelve los vértices virtuales y sus aristas `(padre,hijo)`.

Complejidad: Por consulta $O(k \log k + kT_{LCA})$ tiempo y $O(k)$ memoria adicional, donde T_{LCA} es el costo de consultar un LCA. Con Euler tour y RMQ, $T_{LCA} = O(1)$; con binary lifting, $T_{LCA} = O(\log n)$ y el tiempo es $O(k \log k + k \log n)$. Estas cotas excluyen el preproceso y almacenamiento del LCA, `tin` y `tout`. Para $k \geq 1$ marcados se crean como máximo $2k-1$ vértices; sin marcados se devuelve un árbol vacío.

```
template<class LCAFunction>
pair<vector<int>,vector<pair<int,int>>> build_virtual_tree(
        vector<int> marked,const vector<int>& tin,const vector<int>&
     tout,
        LCAFunction get_lca) {
    auto ancestor=[&](int a,int b) {
        return tin[a]<=tin[b]&&tout[b]<=tout[a];
    };
    auto by_tin=[&](int a,int b) { return tin[a]<tin[b]; };
    sort(marked.begin(),marked.end(),by_tin);
    marked.erase(unique(marked.begin(),marked.end()),marked.end());
    int original_size=marked.size();
    for(int i=1;i<original_size;++i)
        marked.push_back(get_lca(marked[i-1],marked[i]));
    sort(marked.begin(),marked.end(),by_tin);
    marked.erase(unique(marked.begin(),marked.end()),marked.end());
    vector<pair<int,int>> edges;
    vector<int> st;
    for(int vertex:marked) {
        while(!st.empty()&&!ancestor(st.back(),vertex)) st.pop_back()
     ;
        if(!st.empty()) edges.push_back({st.back(),vertex});
        st.push_back(vertex);
    }
    return {marked,edges};
}

// Uso
auto [vertices,edges]=build_virtual_tree(
    marked,tin,tout,[&](int a,int b){ return lca.query(a,b); });
for(auto [parent,child]:edges)
    cout<<parent<<" "<<child<<" "
        <<depth[child]-depth[parent]<<'\n';
```

Si el árbol original tiene pesos, reemplazar la diferencia de profundidades por:

$$distRoot[child] - distRoot[parent].$$

Cuidado: Los LCA agregados no necesariamente estaban marcados originalmente. Guardar aparte los marcados si la DP necesita distinguirlos. Incluir la raíz original explícitamente sólo si la consulta necesita que forme parte del árbol virtual.

4.17 Heavy-Light Decomposition

Cuándo usar: Updates o consultas sobre caminos y subárboles, combinando el árbol con Fenwick o Segment Tree.

Para cada vértice se elige como hijo pesado al hijo con mayor subárbol. Las aristas pesadas forman cadenas cuyos vértices ocupan posiciones consecutivas. Así, un camino se divide en $O(\log n)$ intervalos del arreglo base. La construcción usa recorridos iterativos, sin depender de la altura para consumir pila de llamadas.

Complejidad: Build $O(n)$; camino con Segment Tree $O(\log^2 n)$; subárbol $O(\log n)$; memoria $O(n)$.

```
struct HLD {
    int n,timer=0;
    vector<vector<int>> tree;
    vector<int> parent,depth,size,heavy,head,pos;
    HLD(vector<vector<int>> tree,int root=0):n(tree.size()),
    tree(move(tree)),parent(n,-1),depth(n),size(n),heavy(n,-1),
        head(n),pos(n) {
        assert(n>0);
        vector<int> order{root};
        for(int at=0;at<(int)order.size();++at) {
            int v=order[at];
            for(int to:this->tree[v]) if(to!=parent[v]) {
                parent[to]=v; depth[to]=depth[v]+1; order.push_back(
     to);
            }
        }
        for(int at=n-1;at>=0;--at) {
            int v=order[at]; size[v]=1;
            for(int to:this->tree[v]) if(to!=parent[v]) {
                size[v]+=size[to];
                if(heavy[v]==-1||size[to]>size[heavy[v]]) heavy[v]=to
     ;
            }
        }
        head[root]=root; order.clear(); order.push_back(root);
        while(!order.empty()) {
            int v=order.back(); order.pop_back(); pos[v]=timer++;
            for(auto it=this->tree[v].rbegin();it!=this->tree[v].rend
     ();++it)
                if(*it!=parent[v]&&*it!=heavy[v]) {
                    head[*it]=*it; order.push_back(*it);
                }
            if(heavy[v]!=-1) {
                head[heavy[v]]=head[v]; order.push_back(heavy[v]);
            }
        }
    }
    // Intervalos [l,r) que cubren el camino a--b.
    template<class Callback>
    void for_each_path(int a,int b,Callback callback) const {
        while(head[a]!=head[b]) {
            if(depth[head[a]]<depth[head[b]]) swap(a,b);
            callback(pos[head[a]],pos[a]+1);
            a=parent[head[a]];
        }
        if(depth[a]>depth[b]) swap(a,b);
        callback(pos[a],pos[b]+1);
    }
    // template<class Callback>
    // void for_each_edge_path(int a, int b, Callback callback) const
       {
    //     while (head[a] != head[b]) {
    //         if (depth[head[a]] < depth[head[b]]) swap(a,b);
    //         callback(pos[head[a]], pos[a]+1);
    //         a = parent[head[a]];
    //     }
    //     if (depth[a] > depth[b]) swap(a,b);
    //     if (a != b) callback(pos[a]+1, pos[b]+1); // Excluye solo
      el LCA.
    // }
    // Los vertices del subarbol de v son consecutivos.
    pair<int,int> subtree_interval(int v) const {
        return {pos[v],pos[v]+size[v]};
    }
};
```

Ejemplo con un Segment Tree de suma:

```
HLD hld(tree,0);
vector<ll> base(n);
for(int v=0;v<n;++v) base[hld.pos[v]]=value[v];
SegmentTree segment(base);

// Cambiar el valor del vertice v.
segment.set_value(hld.pos[v],new_value);

// Suma de los vertices del camino a--b.
ll path_sum=0;
hld.for_each_path(a,b,[&](int l,int r) {
    path_sum+=segment.query(l,r);
});

// Suma del subarbol de v.
auto [l,r]=hld.subtree_interval(v);
ll subtree_sum=segment.query(l,r);
```

Para guardar valores en aristas, almacenar cada arista en la posición de su vértice hijo y dejar cero en la raíz. Activar `for_each_edge_path`: excluye el LCA sólo en el último segmento; los anteriores se incluyen completos. Si los extremos coinciden, el camino no tiene aristas. Mantiene $O(\log^2 n)$ por consulta con Segment Tree y $O(1)$ memoria adicional.

```
// void hld_edge_example() {
//     vector<vector<int>> tree = {{1,2}, {0,3,4}, {0}, {1}, {1}};
//     HLD hld(tree,0);
//     vector<ll> edge_to_parent = {0,4,7,2,3}, base(5);
//     for (int v = 0; v < 5; ++v) base[hld.pos[v]] = edge_to_parent[
    v];
//     SegmentTree segment(base);
//     auto path_sum = [&](int a, int b) {
//         ll sum = 0;
//         hld.for_each_edge_path(a,b,[&](int l, int r) { sum +=
     segment.query(l,r); });
//         return sum;
//     };
//     cout << path_sum(3,2) << '\n'; // 13: 2+4+7
//     segment.set_value(hld.pos[3],5); // Cambia la arista 1--3.
//     cout << path_sum(3,2) << '\n'; // 16
//     cout << path_sum(1,1) << '\n'; // 0
// }
```

Cuidado: El callback no entrega necesariamente los segmentos en el orden del camino. Esto no importa para suma, mínimo, máximo o XOR; para operaciones no conmutativas se deben separar y orientar los segmentos de ambos lados.

4.18 Descomposición por centroides

Cuándo usar: Dividir un árbol en componentes balanceadas para contar caminos o mantener consultas de distancia, por ejemplo al vértice marcado más cercano.

Un centroide de una componente de tamaño s es un vértice cuya eliminación deja componentes de tamaño como máximo $s/2$. Se elige un centroide y se descompone recursivamente cada componente restante. El árbol de centroides tiene altura $O(\log n)$; sus aristas no son necesariamente aristas originales.

Complejidad: Construcción $O(n \log n)$ tiempo y $O(n)$ memoria, incluyendo la pila de los DFS. La profundidad de la descomposición es $O(\log n)$, pero un DFS auxiliar puede alcanzar profundidad $O(n)$.

```
struct CentroidDecomposition {
    const vector<vector<int>> &tree;
    vector<int> size, parent;
    vector<char> removed;
    int root = -1;

    explicit CentroidDecomposition(const vector<vector<int>> &
     input_tree)
        : tree(input_tree), size(tree.size()), parent(tree.size(),
     -1),
          removed(tree.size(), false) {
        if (!tree.empty()) root = decompose(0, -1);
    }

    int calculate_size(int v, int previous) {
        size[v] = 1;
        for (int to : tree[v])
            if (to != previous && !removed[to])
                size[v] += calculate_size(to, v);
        return size[v];
    }

    int find_centroid(int v, int previous, int total) {
        for (int to : tree[v])
            if (to != previous && !removed[to] && size[to] > total /
     2)
                return find_centroid(to, v, total);
        return v;
    }

    int decompose(int entry, int previous_centroid) {
        int total = calculate_size(entry, -1);
        int centroid = find_centroid(entry, -1, total);
        parent[centroid] = previous_centroid;
        removed[centroid] = true;
        for (int to : tree[centroid])
            if (!removed[to]) decompose(to, centroid);
        return centroid;
    }
};
```

`parent[v]` es el padre de v en el árbol de centroides; la raíz tiene padre -1. La lista de adyacencia original no se modifica. Al terminar, todos los vértices quedan marcados en `removed`.

```
void centroid_example() {
    vector<vector<int>> tree = {{1}, {0, 2, 3}, {1}, {1, 4}, {3}};
    CentroidDecomposition decomposition(tree);
    cout << decomposition.root << '\n'; // 1

    // Ancestros centroides del vertice 4, incluyendo al propio 4.
    for (int v = 4; v != -1; v = decomposition.parent[v])
        cout << v << ' '; // 4 3 1
}
```

Para consultas de distancias, agregar la información del problema en cada centroide y recorrer estos ancestros. Las distancias deben medirse en el árbol original, mediante LCA o precomputándolas durante la construcción.

Cuidado: Requiere un árbol no dirigido conectado; los vértices son 0-indexados. El constructor conserva una referencia: el árbol debe vivir más que la estructura. Los DFS recursivos pueden desbordar la pila en cadenas largas; usar recorridos iterativos si los límites lo requieren.

4.19 Rerooting DP

Cuándo usar: Se necesita una respuesta para cada vértice considerando que ese vértice es la raíz o punto de inicio: suma de distancias a todos, cantidad de nodos alcanzables, mejor costo al elegir una capital o valor de una DP para cada raíz. La respuesta de raíces vecinas debe poder actualizarse reutilizando la información de sus subárboles. Evita ejecutar una DFS $O(n)$ desde cada vértice, lo que costaría $O(n^2)$.
Complejidad: tiempo $O(n)$; memoria $O(n)$.
Se calcula un orden padre–hijos y se recorre en ambos sentidos, sin DFS recursiva. El grafo debe ser un árbol conectado; se admite el árbol vacío.

```
struct DistanceRerooting {
    int node_count;
    const vector<vector<int>> &tree;
    vector<int> subtree_size;
    vector<ll> distance_sum;

    explicit DistanceRerooting(const vector<vector<int>> &input_tree)
        : node_count(input_tree.size()), tree(input_tree),
          subtree_size(node_count, 1), distance_sum(node_count) {
        if (!node_count) return;
        vector<int> parent(node_count, -1), order{0};
        for (int at = 0; at < node_count; ++at) {
            int v = order[at];
            for (int to : tree[v]) if (to != parent[v]) {
                parent[to] = v; order.push_back(to);
            }
        }
        for (int at = node_count - 1; at > 0; --at) {
            int v = order[at], p = parent[v];
            subtree_size[p] += subtree_size[v];
            distance_sum[p] += distance_sum[v] + subtree_size[v];
        }
        for (int at = 1; at < node_count; ++at) {
            int v = order[at];
            distance_sum[v] = distance_sum[parent[v]]
                            + node_count - 2 * subtree_size[v];
        }
    }
};
```

```
tree[u].push_back(v);
tree[v].push_back(u);

DistanceRerooting rerooting(tree);
for(int v=0;v<n;++v)
    cout<<rerooting.distance_sum[v]<<'\n';
```

Para una combinación genérica no invertible, usar prefijos y sufijos de las contribuciones de los hijos para excluir uno en $O(1)$.

4.20 Diámetro de un àrbol y camino

Cuándo usar: Hallar el camino más largo de un àrbol con pesos no negativos. Con pesos uno, su centro y radio también ayudan a conectar varios àrboles.
Complejidad: Dos recorridos: tiempo y memoria $O(n)$. Son iterativos para admitir cadenas largas sin agotar la pila.

```
pair<ll, vector<int>> tree_diameter(
        const vector<vector<pair<int,ll>>> &tree) {
    int n = tree.size();
    if (n == 0) return {0, {}};
    vector<ll> distance(n);
    vector<int> parent(n);
    auto farthest = [&](int source) {
        fill(distance.begin(), distance.end(), -1);
        fill(parent.begin(), parent.end(), -1);
        vector<int> order{source};
        distance[source] = 0;
        for (int i = 0; i < (int)order.size(); ++i) {
            int v = order[i];
            for (auto [to, weight] : tree[v]) if (distance[to] == -1)
      {
                assert(weight >= 0);
                distance[to] = distance[v] + weight;
                parent[to] = v; order.push_back(to);
            }
        }
        assert((int)order.size() == n);
        return (int)(max_element(distance.begin(), distance.end()) -
      distance.begin());
    };
    int start = farthest(0), finish = farthest(start);
    vector<int> path;
    for (int v = finish; v != -1; v = parent[v]) path.push_back(v);
    reverse(path.begin(), path.end());
    return {distance[finish], path};
}
void tree_diameter_example() {
    vector<vector<pair<int,ll>>> tree(4);
    auto add = [&](int a, int b, ll w) {
        tree[a].push_back({b,w}); tree[b].push_back({a,w});
    };
    add(0,1,2); add(1,2,5); add(1,3,4);
    auto [length, path] = tree_diameter(tree);
    cout << length << '\n'; // 9; camino 2--1--3 (o su inverso).
}
```

Cuidado: Requiere un àrbol conectado, cada arista en ambos sentidos y distancias que entren en `long long`. Con pesos negativos, usar DP de àrbol en lugar de estos dos recorridos. Para un bosque, procesar componentes.
Si todas las aristas pesan uno, el radio es $\lceil diametro/2 \rceil$ y un centro es `path[path.size()/2]`. Es la herramienta base de Adjoin the Networks (NCPC 2015); no resolver cada componente recorriendo de nuevo todos los vértices del bosque.

4.21 Camino euleriano

Cuándo usar: Recorrer cada arista exactamente una vez; también cuando objetos son aristas entre dos tipos de entidad, como filas y columnas.
Hierholzer usa cada arista exactamente una vez. El chequeo final de longitud tambíen valida que todas las aristas pertenezcan al componente recorrido.
Complejidad: $O(V + E)$ tiempo y memoria.

```
vector<int> directed_eulerian_path(
        const vector<vector<pair<int,int>>> &graph, int edge_count) {
    int n = graph.size();
    if (n == 0) return edge_count == 0 ? vector<int>{} : vector<int
     >{};
    vector<int> indegree(n, 0), outdegree(n, 0);
    for (int v = 0; v < n; ++v)
        for (auto [to, id] : graph[v]) {
```

```
            (void)id;
            ++outdegree[v]; ++indegree[to];
        }

    int start = -1, finish = -1;
    for (int v = 0; v < n; ++v) {
        int difference = outdegree[v] - indegree[v];
        if (difference == 1) {
            if (start != -1) return {};
            start = v;
        } else if (difference == -1) {
            if (finish != -1) return {};
            finish = v;
        } else if (difference != 0) return {};
    }
    if ((start == -1) != (finish == -1)) return {};
    if (start == -1) {
        start = 0;
        while (start < n && outdegree[start] == 0) ++start;
        if (start == n) return {0};
    }

    vector<int> next_edge(n, 0), stack{start}, path;
    vector<char> used(edge_count, false);
    while (!stack.empty()) {
        int v = stack.back();
        while (next_edge[v] < (int)graph[v].size() &&
               used[graph[v][next_edge[v]].second]) ++next_edge[v];
        if (next_edge[v] == (int)graph[v].size()) {
            path.push_back(v); stack.pop_back();
        } else {
            auto [to, id] = graph[v][next_edge[v]++];
            if (id < 0 || id >= edge_count) return {};
            used[id] = true;
            stack.push_back(to);
        }
    }
    if ((int)path.size() != edge_count + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}

vector<int> undirected_eulerian_path(
        const vector<vector<pair<int,int>>> &graph, int edge_count) {
    int n = graph.size();
    if (n == 0) return {};
    vector<int> odd;
    for (int v = 0; v < n; ++v)
        if (graph[v].size() % 2 == 1) odd.push_back(v);
    if (!odd.empty() && odd.size() != 2) return {};

    int start = odd.empty() ? 0 : odd[0];
    while (start < n && graph[start].empty()) ++start;
    if (start == n) return {0};

    vector<int> next_edge(n, 0), stack{start}, path;
    vector<char> used(edge_count, false);
    while (!stack.empty()) {
        int v = stack.back();
        while (next_edge[v] < (int)graph[v].size() &&
               used[graph[v][next_edge[v]].second]) ++next_edge[v];
        if (next_edge[v] == (int)graph[v].size()) {
            path.push_back(v); stack.pop_back();
        } else {
            auto [to, id] = graph[v][next_edge[v]++];
            if (id < 0 || id >= edge_count) return {};
            used[id] = true;
            stack.push_back(to);
        }
    }
    if ((int)path.size() != edge_count + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}
```

En el grafo no dirigido, cada arista debe aparecer dos veces en la lista de adyacencia con el mismo ID. En el dirigido aparece una sola vez.

4.22 Flujo máximo (Ford–Fulkerson)

Cuándo usar: Red pequeña con capacidades enteras y flujo total bajo, o como implementación conceptual rápida. Para casos grandes preferir Dinic.

Busca caminos aumentantes con DFS y actualiza el grafo residual. Esta versión usa capacidades enteras no negativas.

Cada arista representa cuánto flujo puede enviarse desde `source` hasta `sink`. Al enviar x, se resta x de la capacidad directa y se suma a la inversa; esta arista residual permite deshacer o redirigir flujo anterior. Ford–Fulkerson repite: buscar un camino con capacidad, enviar el mínimo de sus capacidades y terminar cuando ya no existe otro camino.

```
struct FordFulkerson {
    struct Edge { int to, reverse; ll capacity; };
    int n;
    vector<vector<Edge>> graph;
    vector<char> seen;

    explicit FordFulkerson(int size)
        : n(size), graph(size), seen(size) {}

    void add_edge(int from, int to, ll capacity) {
        assert(capacity >= 0);
        Edge forward{to, (int)graph[to].size(), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    ll dfs(int vertex, int sink, ll pushed) {
        if (vertex == sink) return pushed;
        seen[vertex] = true;
        for (Edge &edge : graph[vertex]) {
            if (seen[edge.to] || edge.capacity == 0) continue;
            ll flow = dfs(edge.to, sink, min(pushed, edge.capacity));
            if (flow == 0) continue;
            edge.capacity -= flow;
            graph[edge.to][edge.reverse].capacity += flow;
            return flow;
        }
        return 0;
    }

    ll max_flow(int source, int sink) {
        assert(source != sink);
        ll answer = 0;
        while (true) {
            fill(seen.begin(), seen.end(), false);
            ll pushed = dfs(source, sink, LINF);
            if (pushed == 0) break;
            answer += pushed;
        }
        return answer;
    }
};
```

Complejidad: $O(EF)$ para capacidades enteras, donde F es el flujo máximo; es pseudopolinomial; memoria $O(V + E)$. Con capacidades grandes suele convenir Dinic.

4.23 Flujo máximo (Dinic)

Cuándo usar: Capacidades, cortes mínimos, asignaciones con cupos y selección con restricciones que se modelan como una red fuente–sumidero.

Calcula el mismo flujo máximo que Ford–Fulkerson, pero agrupa los aumentos. Un BFS construye niveles usando aristas residuales y los DFS envían todo el flujo posible avanzando sólo al nivel siguiente. `next_edge` evita revisar aristas que ya se demostraron inútiles en la fase actual. Es la opción usual de flujo máximo en competencia.

```
struct Dinic {
    struct Edge { int to, reverse; ll capacity; };
    int n;
    vector<vector<Edge>> graph;
    vector<int> level, next_edge;

    explicit Dinic(int size)
        : n(size), graph(size), level(size), next_edge(size) {}

    void add_edge(int from, int to, ll capacity) {
        Edge forward{to, (int)graph[to].size(), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    bool bfs(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> q;
        level[source] = 0; q.push(source);
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (const Edge &edge : graph[v])
                if (edge.capacity > 0 && level[edge.to] == -1)
                    level[edge.to] = level[v] + 1, q.push(edge.to);
        }
        return level[sink] != -1;
    }

    ll dfs(int v, int sink, ll pushed) {
        if (v == sink || pushed == 0) return pushed;
        for (int &i = next_edge[v]; i < (int)graph[v].size(); ++i) {
            Edge &edge = graph[v][i];
            if (edge.capacity == 0 || level[edge.to] != level[v] + 1)
      continue;
            ll flow = dfs(edge.to, sink, min(pushed, edge.capacity));
            if (flow == 0) continue;
            edge.capacity -= flow;
            graph[edge.to][edge.reverse].capacity += flow;
            return flow;
        }
        return 0;
    }

    ll max_flow(int source, int sink) {
        assert(source != sink);
        ll answer = 0;
        while (bfs(source, sink)) {
            fill(next_edge.begin(), next_edge.end(), 0);
```

```
        while (ll pushed = dfs(source, sink, LINF)) answer +=
  pushed;
      }
      return answer;
  }

  // pair<int,int> add_tracked_edge(int from, int to, ll capacity)
   {
  //     assert(from != to && capacity >= 0);
  //     int index = graph[from].size();
  //     add_edge(from, to, capacity);
  //     return {from, index}; // ID estable de la arista original.
  // }
  // ll edge_flow(pair<int,int> id) const {
  //     const Edge& edge = graph[id.first][id.second];
  //     return graph[edge.to][edge.reverse].capacity;
  // }
  // vector<char> min_cut_side(int source) const {
  //     vector<char> side(n, false);
  //     queue<int> q; q.push(source); side[source] = true;
  //     while (!q.empty()) {
  //         int v = q.front(); q.pop();
  //         for (const Edge& edge : graph[v])
  //             if (edge.capacity > 0 && !side[edge.to]) {
  //                 side[edge.to] = true; q.push(edge.to);
  //             }
  //     }
  //     return side;
  // }
};
```

Complejidad: general $O(V^2E)$ tiempo; memoria $O(V+E)$; suele ser mucho más rápido en práctica.

Opcional: activar los helpers comentados y guardar el ID al crear cada arista con extremos distintos. `edge_flow` cuesta $O(1)$. Después de `max_flow`, `min_cut_side` devuelve el lado de la fuente en $O(V+E)$ tiempo y $O(V)$ memoria; el corte cruza de ese lado al otro. Capacidades no negativas y flujo total representable en `long long`.

```
// void dinic_recovery_example() {
//     Dinic flow(4);
//     vector<pair<int,int>> ids;
//     ids.push_back(flow.add_tracked_edge(0,1,3));
//     ids.push_back(flow.add_tracked_edge(0,2,2));
//     ids.push_back(flow.add_tracked_edge(1,2,1));
//     ids.push_back(flow.add_tracked_edge(1,3,2));
//     ids.push_back(flow.add_tracked_edge(2,3,3));
//     cout << flow.max_flow(0,3) << '\n'; // 5
//     auto side = flow.min_cut_side(0);
//     for (auto id : ids) {
//         const auto& edge = flow.graph[id.first][id.second];
//         cout << id.first << "->" << edge.to << ": " << flow.
    edge_flow(id) << '\n';
//         if (side[id.first] && !side[edge.to])
//             cout << "Corte: " << id.first << "->" << edge.to << '\
    n';
//     }
// }
```

4.24 Flujo de costo mínimo

Cuándo usar: Enviar cierta cantidad de flujo optimizando un costo: transporte, matching ponderado con capacidades o asignación de recursos.

SPFA por aumento; admite costos negativos si no hay ciclos negativos de costo alcanzables con capacidad residual positiva.

Cada arista tiene capacidad y costo por unidad. En cada iteración se busca el camino residual de menor costo, se envía todo lo permitido y se acumula `flujo` × `costo`. `run(s,t,k)` intenta enviar hasta k unidades y devuelve `{flujo_enviado,costo_total}`; si no existe capacidad suficiente, el flujo devuelto es menor que k.

```
struct MinCostMaxFlow {
    struct Edge { int to, reverse; ll capacity, cost; };
    int n;
    vector<vector<Edge>> graph;

    explicit MinCostMaxFlow(int size) : n(size), graph(size) {}

    void add_edge(int from, int to, ll capacity, ll cost) {
        Edge forward{to, (int)graph[to].size(), capacity, cost};
        Edge backward{from, (int)graph[from].size(), 0, -cost};
        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    pair<ll,ll> run(int source, int sink, ll flow_limit = LINF) {
        ll total_flow = 0, total_cost = 0;
        vector<ll> distance(n);
        vector<int> parent_vertex(n), parent_edge(n);
        while (total_flow < flow_limit) {
            fill(distance.begin(), distance.end(), LINF);
            vector<char> in_queue(n, false);
            queue<int> q;
            distance[source] = 0; q.push(source); in_queue[source] =
     true;
            while (!q.empty()) {
                int v = q.front(); q.pop(); in_queue[v] = false;
                for (int i = 0; i < (int)graph[v].size(); ++i) {
                    Edge const &edge = graph[v][i];
                    if (edge.capacity == 0 ||
                        distance[edge.to] <= distance[v] + edge.cost)
         continue;
                    distance[edge.to] = distance[v] + edge.cost;
                    parent_vertex[edge.to] = v; parent_edge[edge.to]
     = i;
                    if (!in_queue[edge.to]) q.push(edge.to), in_queue
     [edge.to] = true;
                }
            }
            if (distance[sink] == LINF) break;
            ll pushed = flow_limit - total_flow;
            for (int v = sink; v != source; v = parent_vertex[v])
                pushed = min(pushed,
                    graph[parent_vertex[v]][parent_edge[v]].capacity)
    ;
            for (int v = sink; v != source; v = parent_vertex[v]) {
                Edge &edge = graph[parent_vertex[v]][parent_edge[v]];
                edge.capacity -= pushed;
                graph[v][edge.reverse].capacity += pushed;
            }
            total_flow += pushed;
            total_cost += pushed * distance[sink];
        }
        return {total_flow, total_cost};
    }
};
```

Complejidad: $O(FVE)$ en el peor caso para flujo entero F; memoria $O(V+E)$.

4.24.1 Elección rápida

- Sólo maximizar cantidad: preferir **Dinic**.
- Maximizar cantidad y minimizar costo: **Min-Cost Max-Flow**.
- Ford–Fulkerson: implementación conceptual o flujo total pequeño.

En matching bipartito se modela `source` → `izquierda` → `derecha` → `sink`, normalmente con capacidades 1. En la versión ponderada, el costo de la arista central es el costo de realizar esa asignación.

4.25 Matching bipartito: Hopcroft–Karp

Cuándo usar: Emparejar el máximo número de elementos entre dos conjuntos con compatibilidades binarias y sin costos.

Los vértices izquierdos son $[0, n)$ y los derechos $[0, m)$.

Una arista (l, r) indica que el elemento izquierdo l puede emparejarse con el derecho r; cada vértice puede pertenecer a lo sumo a una pareja.

Hopcroft–Karp busca caminos aumentantes: alterna aristas fuera/dentro del matching para reemplazar parejas y aumentar su tamaño en uno. El BFS agrupa los caminos aumentantes más cortos por niveles y los DFS procesan varios en la misma fase. `maximum_matching()` devuelve la cantidad de parejas; `match_left[l]` y `match_right[r]` contienen su pareja o -1.

Complejidad: $O(E\sqrt{V})$ tiempo y $O(V+E)$ memoria.

```
struct HopcroftKarp {
    int left_size, right_size;
    vector<vector<int>> graph;
    vector<int> match_left, match_right, distance;

    HopcroftKarp(int left_count, int right_count)
        : left_size(left_count), right_size(right_count), graph(
     left_count),
          match_left(left_count, -1), match_right(right_count, -1),
          distance(left_count) {}

    void add_edge(int left, int right) { graph[left].push_back(right)
     ; }

    bool bfs() {
        queue<int> q;
        fill(distance.begin(), distance.end(), -1);
        for (int left = 0; left < left_size; ++left)
            if (match_left[left] == -1) distance[left] = 0, q.push(
     left);
        bool augmenting_path = false;
        while (!q.empty()) {
            int left = q.front(); q.pop();
            for (int right : graph[left]) {
                int next_left = match_right[right];
                if (next_left == -1) augmenting_path = true;
                else if (distance[next_left] == -1) {
                    distance[next_left] = distance[left] + 1;
                    q.push(next_left);
                }
            }
        }
        return augmenting_path;
    }

    bool dfs(int left) {
        for (int right : graph[left]) {
            int next_left = match_right[right];
            if (next_left == -1 ||
                (distance[next_left] == distance[left] + 1 && dfs(
     next_left))) {
                match_left[left] = right;
```

```
                match_right[right] = left;
                return true;
            }
        }
        distance[left] = -1;
        return false;
    }

    int maximum_matching() {
        int matching = 0;
        while (bfs())
            for (int left = 0; left < left_size; ++left)
                if (match_left[left] == -1 && dfs(left)) ++matching;
        return matching;
    }
};
```

Ejemplo: asignar el máximo número de personas a trabajos compatibles.

```
HopcroftKarp matching(3,3);
matching.add_edge(0,0); matching.add_edge(0,1);
matching.add_edge(1,1); matching.add_edge(2,1);
int pairs=matching.maximum_matching();
for(int person=0;person<3;++person)
    cout<<person<<" -> "<<matching.match_left[person]<<'\n';
```

4.26 Asignación de costo mínimo: Húngaro

Cuándo usar: Asignar cada elemento de un lado a uno distinto del otro minimizando un costo total en una matriz completa o casi completa.

Matriz rectangular con $n \leq m$. Devuelve costo y columna asignada a cada fila. Admite costos negativos.

`cost[i][j]` es el costo de asignar la fila i a la columna j; todas las filas reciben columnas distintas. El algoritmo incorpora una fila por vez y usa potenciales para conservar costos reducidos no negativos mientras encuentra la reasignación más barata. Devuelve `{costo,assignment}`, donde `assignment[i]` es la columna elegida para la fila i.

Complejidad: $O(n^2 m)$ tiempo y $O(n+m)$ memoria.

```
pair<ll, vector<int>> hungarian(const vector<vector<ll>> &cost) {
    int n = cost.size();
    if (n == 0) return {0, {}};
    int m = cost[0].size();
    assert(n <= m && m > 0);

    vector<ll> row_potential(n + 1), column_potential(m + 1);
    vector<int> row_for_column(m + 1), previous_column(m + 1);
    for (int row = 1; row <= n; ++row) {
        row_for_column[0] = row;
        int column0 = 0;
        vector<ll> minimum(m + 1, LINF);
        vector<char> used(m + 1, false);
        do {
            used[column0] = true;
            int row0 = row_for_column[column0], column1 = 0;
            ll delta = LINF;
            for (int column = 1; column <= m; ++column) if (!used[
    column]) {
                ll current = cost[row0 - 1][column - 1]
                           - row_potential[row0] - column_potential[
    column];
                if (current < minimum[column])
                    minimum[column] = current, previous_column[column
    ] = column0;
                if (minimum[column] < delta)
                    delta = minimum[column], column1 = column;
            }
            for (int column = 0; column <= m; ++column) {
                if (used[column]) {
                    row_potential[row_for_column[column]] += delta;
                    column_potential[column] -= delta;
                } else minimum[column] -= delta;
            }
            column0 = column1;
        } while (row_for_column[column0] != 0);

        do {
            int column1 = previous_column[column0];
            row_for_column[column0] = row_for_column[column1];
            column0 = column1;
        } while (column0 != 0);
    }

    vector<int> assignment(n, -1);
    for (int column = 1; column <= m; ++column)
        if (row_for_column[column])
            assignment[row_for_column[column] - 1] = column - 1;
    return {-column_potential[0], assignment};
}
```

Ejemplo: asignar tres personas a trabajos distintos minimizando el costo.

```
vector<vector<ll>> cost={{4,1,3},{2,0,5},{3,2,2}};
auto [minimum_cost,job]=hungarian(cost);
cout<<minimum_cost<<'\n'; // 5
for(int person=0;person<3;++person)
    cout<<person<<" -> "<<job[person]<<'\n';
```

4.26.1 Elección rápida

- Compatibilidad sí/no y maximizar parejas: **Hopcroft–Karp**.
- Matriz de costos y asignar todas las filas: **Húngaro**.
- Capacidades, cupos o cantidad de asignaciones variable con costos: **Min-Cost Max-Flow**.

4.27 Cobertura mínima bipartita: König

Cuándo usar: Elegir el menor conjunto de vértices que toca todas las aristas de un grafo bipartito. El complemento es un conjunto independiente máximo.

Complejidad: Con matching máximo ya calculado: tiempo $O(V+E)$ y memoria $O(V)$. Incluyendo Hopcroft–Karp, tiempo $O(E\sqrt{V}+V+E)$.

Usar el `HopcroftKarp` de la sección de matching. Desde los vértices izquierdos libres se recorren caminos alternantes; si Z son los alcanzados, la cobertura es $(L \setminus Z) \cup (R \cap Z)$.

```
pair<vector<int>, vector<int>> minimum_vertex_cover(const
     HopcroftKarp &matching) {
    vector<char> left(matching.left_size), right(matching.right_size)
     ;
    queue<int> q;
    for (int u = 0; u < matching.left_size; ++u)
        if (matching.match_left[u] == -1) { left[u] = true; q.push(u)
     ; }
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : matching.graph[u]) {
            if (matching.match_left[u] == v || right[v]) continue;
            right[v] = true;
            int next = matching.match_right[v];
            assert(next != -1); // Si llega a uno libre, el matching
     no era maximo.
            if (!left[next]) { left[next] = true; q.push(next); }
        }
    }
    vector<int> cover_left, cover_right;
    for (int u = 0; u < matching.left_size; ++u) if (!left[u])
     cover_left.push_back(u);
    for (int v = 0; v < matching.right_size; ++v) if (right[v])
     cover_right.push_back(v);
    return {cover_left, cover_right};
}
void konig_example() {
    HopcroftKarp matching(3, 2);
    matching.add_edge(0,0); matching.add_edge(1,0); matching.add_edge
     (2,1);
    int size = matching.maximum_matching();
    auto [left, right] = minimum_vertex_cover(matching);
    cout << size << ' ' << left.size() + right.size() << '\n'; // 2 2
}
```

Cuidado: Calcular el matching máximo antes y recalcularlo si se agregan aristas. Los IDs izquierdos y derechos pertenecen a conjuntos distintos. El teorema no permite sustituir cobertura por matching en grafos generales.

5. Strings

5.1 Rolling hash

Cuándo usar: Comparar substrings, detectar repeticiones o hacer binary search sobre longitudes aceptando una probabilidad pequeña de colisión.

Hash doble para comparar substrings en $O(1)$ tras preprocesar. Los rangos son semiabiertos $[l, r)$. Una igualdad de hashes sigue siendo probabilística.

```
struct RollingHash {
    static const int MOD1 = 1000000007;
    static const int MOD2 = 1000000009;
    static const int BASE = 911382323;
    vector<int> power1, power2, hash1, hash2;

    explicit RollingHash(const string &s) {
        int n = s.size();
        power1.assign(n + 1, 1); power2.assign(n + 1, 1);
        hash1.assign(n + 1, 0);  hash2.assign(n + 1, 0);
        for (int i = 0; i < n; ++i) {
            int value = (unsigned char)s[i] + 1;
            power1[i + 1] = (ll)power1[i] * BASE % MOD1;
            power2[i + 1] = (ll)power2[i] * BASE % MOD2;
            hash1[i + 1] = ((ll)hash1[i] * BASE + value) % MOD1;
            hash2[i + 1] = ((ll)hash2[i] * BASE + value) % MOD2;
        }
    }

    pair<int,int> get(int l, int r) const {
        ll first = hash1[r] - (ll)hash1[l] * power1[r - l] % MOD1;
        ll second = hash2[r] - (ll)hash2[l] * power2[r - l] % MOD2;
        if (first < 0) first += MOD1;
        if (second < 0) second += MOD2;
        return {(int)first, (int)second};
    }
};
```

Complejidad: construcción $O(n)$; consulta $O(1)$; memoria $O(n)$.

Comparar dos substrings de la misma longitud:

```
string s="banana";
RollingHash hash(s);
bool equal=hash.get(1,4)==hash.get(3,6);
cout<<equal<<'\n'; // 1: "ana" == "ana"
```

Buscar apariciones de un patrón:

```
string text="bananana",pattern="ana";
RollingHash text_hash(text),pattern_hash(pattern);
auto target=pattern_hash.get(0,pattern.size());
for(int i=0;i+(int)pattern.size()<=(int)text.size();++i)
    if(text_hash.get(i,i+pattern.size())==target)
        cout<<i<<' '; // 1 3 5
```

`get(l,r)` devuelve el hash del substring semiabierto $[l, r)$. Sólo deben compararse substrings de igual longitud. Aunque se use hash doble, existe una probabilidad muy pequeña de colisión.

5.2 Prefix function, KMP

Cuándo usar: Buscar exactamente un patrón en un texto o analizar bordes y períodos de una cadena sin hashing.

```
vector<int> prefix_function(const string &s) {
    int n = s.size();
    vector<int> pi(n, 0);
    for (int i = 1; i < n; ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_occurrences(const string &text, const string &pattern
    ) {
    if (pattern.empty()) {
        vector<int> all(text.size() + 1);
        iota(all.begin(), all.end(), 0);
        return all;
    }
    vector<int> pi = prefix_function(pattern), answer;
    int matched = 0;
    for (int i = 0; i < (int)text.size(); ++i) {
        while (matched > 0 && text[i] != pattern[matched])
            matched = pi[matched - 1];
        if (text[i] == pattern[matched]) ++matched;
        if (matched == (int)pattern.size()) {
            answer.push_back(i - matched + 1);
            matched = pi[matched - 1];
        }
    }
    return answer;
}
```

Complejidad: $O(n + m)$ tiempo; $O(m)$ memoria para buscar.

5.2.1 Ejemplo de KMP

`pi[i]` es la longitud del mayor prefijo propio de `s[0..i]` que también es sufijo. KMP usa esta información para no volver al inicio del patrón después de una diferencia.

```
string text="bananana",pattern="ana";
vector<int> positions=kmp_occurrences(text,pattern);
for(int position:positions) cout<<position<<' '; // 1 3 5
```

También permite encontrar el mayor borde de una cadena:

```
string s="abcab";
vector<int> pi=prefix_function(s);
int longest_border=pi.back(); // 2: "ab"
```

Un borde es un prefijo propio que también aparece como sufijo.

5.3 Z-function

Cuándo usar: Conocer para cada posición cuánto coincide el sufijo que empieza allí con el prefijo; ùtil para patrones, bordes y periodicidad.

`z[i]` es la longitud del prefijo de s que coincide desde i.

```
vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i < r) z[i] = min(r - i, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) ++z[i];
        if (i + z[i] > r) l = i, r = i + z[i];
    }
    return z;
}
```

Complejidad: $O(n)$ tiempo y memoria.

5.3.1 Ejemplo de Z-function

`z[i]` indica cuántos caracteres desde `s[i]` coinciden con el prefijo completo de `s`. Por ejemplo, para **''aaaaa''**:

$$z = [0, 4, 3, 2, 1].$$

Para buscar un patrón, concatenar patrón, un separador ausente y texto:

```
string text="bananana",pattern="ana";
string combined=pattern+"#"+text;
vector<int> z=z_function(combined);
int offset=pattern.size()+1;
for(int i=offset;i<(int)combined.size();++i)
    if(z[i]>=(int)pattern.size())
        cout<<i-offset<<' '; // 1 3 5
```

Cuidado: El separador, aquí `#`, no debe aparecer ni en el patrón ni en el texto.

- **KMP/prefix function:** buscar exactamente un patrón y estudiar bordes o períodos.
- **Z-function:** conocer cuánto coincide cada sufijo con el prefijo; suele resultar más directa para matching mediante concatenación.
- Ambos son deterministas y cuestan $O(n + m)$; no tienen colisiones como rolling hash.

5.4 Trie

Cuándo usar: Mantener un diccionario de cadenas y consultar palabras exactas o cuántas comienzan con cierto prefijo en $O(|s|)$. Útil para autocompletado, prefijos compartidos y XOR máximo mediante un Trie binario. Conviene con alfabetos pequeños; puede consumir mucha memoria si hay pocas coincidencias entre prefijos.

Versión para letras minúsculas inglesas.

```
struct Trie {
    struct Node {
        array<int, 26> next;
        int terminal = 0, passing = 0;
        Node() { next.fill(-1); }
    };
    vector<Node> nodes{Node()};

    void insert(const string &word) {
        int v = 0;
        ++nodes[v].passing;
        for (char c : word) {
            int letter = c - 'a';
            assert(0 <= letter && letter < 26);
            if (nodes[v].next[letter] == -1) {
                nodes[v].next[letter] = nodes.size();
                nodes.emplace_back();
            }
            v = nodes[v].next[letter];
            ++nodes[v].passing;
        }
        ++nodes[v].terminal;
    }

    int count_word(const string &word) const {
        int v = 0;
        for (char c : word) {
            int letter = c - 'a';
            if (letter < 0 || letter >= 26 || nodes[v].next[letter]
     == -1)
                return 0;
            v = nodes[v].next[letter];
        }
        return nodes[v].terminal;
    }

    int count_prefix(const string &prefix) const {
        int v = 0;
        for (char c : prefix) {
            int letter = c - 'a';
            if (letter < 0 || letter >= 26 || nodes[v].next[letter]
     == -1)
                return 0;
            v = nodes[v].next[letter];
        }
        return nodes[v].passing;
    }
};
```

Complejidad: $O(|s|)$ por inserción o búsqueda; memoria proporcional a la suma de longitudes, hasta $O(\Sigma|s|)$.

Cada nodo representa un prefijo. `terminal` cuenta cuántas veces se insertó exactamente la palabra que termina allí; `passing` cuenta cuántas palabras, incluyendo repetidas, pasan por ese prefijo.

Ejemplo de uso:

```
Trie trie;
trie.insert("car");
trie.insert("cat");
trie.insert("car");
trie.insert("dog");
cout<<trie.count_word("car")<<'\n';   // 2
cout<<trie.count_word("ca")<<'\n';    // 0
cout<<trie.count_prefix("ca")<<'\n';  // 3
cout<<trie.count_prefix("do")<<'\n';  // 1
cout<<trie.count_prefix("x")<<'\n';   // 0
```

`count_word(s)` exige que la palabra completa esté insertada; `count_prefix(s)` cuenta todas las palabras que comienzan con `s`.

5.5 Manacher

Cuándo usar: Calcular en tiempo lineal el radio del palíndromo máximo centrado en cada posición, para responder existencia/longitud de substrings palindrómicos.

`odd[i]` es el radio del palíndromo impar centrado en i, incluyendo el centro. `even[i]` es el radio del palíndromo par centrado entre $i-1$ e i.

Complejidad: $O(n)$ tiempo y memoria.

```
pair<vector<int>, vector<int>> manacher(const string &s) {
    int n = s.size();
    vector<int> odd(n), even(n);

    for (int i = 0, left = 0, right = -1; i < n; ++i) {
        int radius = i > right ? 1 : min(odd[left + right - i], right
      - i + 1);
        while (i - radius >= 0 && i + radius < n &&
               s[i - radius] == s[i + radius]) ++radius;
        odd[i] = radius--;
        if (i + radius > right) left = i - radius, right = i + radius
     ;
    }

    for (int i = 0, left = 0, right = -1; i < n; ++i) {
        int radius = i > right ? 0
                   : min(even[left + right - i + 1], right - i + 1);
        while (i - radius - 1 >= 0 && i + radius < n &&
               s[i - radius - 1] == s[i + radius]) ++radius;
        even[i] = radius--;
        if (i + radius > right) left = i - radius - 1, right = i +
     radius;
    }
    return {odd, even};
}
```

La longitud máxima es $\max(2 \cdot \texttt{odd[i]} - 1, 2 \cdot \texttt{even[i]})$.

5.6 Aho–Corasick

Cuándo usar: Buscar simultáneamente muchas palabras en uno o varios textos, reportando coincidencias mientras se recorre cada texto una sola vez.

Busca simultáneamente muchos patrones no vacíos. Esta versión usa letras minúsculas inglesas y devuelve el ID de cada patrón junto con su posición final. Cada nodo guarda sólo sus propios patrones; un enlace de salida apunta al sufijo terminal más cercano, evitando copiar las coincidencias heredadas.

Complejidad: Con $L = \sum |p|$, alfabeto de tamaño $\Sigma = 26$ y z coincidencias: construcción $O(L\Sigma)$ con transiciones completas; búsqueda $O(|texto| + z)$. Autómata: memoria $O(L\Sigma)$; devolver las coincidencias agrega $O(z)$ memoria de salida.

```
struct AhoCorasick {
    static const int ALPHABET = 26;
    struct Node {
        array<int, ALPHABET> next;
        int suffix_link = 0;
        int output_link = -1; // Sufijo terminal mas cercano.
        vector<int> output;
        Node() { next.fill(-1); }
    };
    vector<Node> nodes{Node()};

    void insert(const string &pattern, int pattern_id) {
        assert(!pattern.empty());
        int v = 0;
        for (char character : pattern) {
            int letter = character - 'a';
            assert(0 <= letter && letter < ALPHABET);
            if (nodes[v].next[letter] == -1) {
                nodes[v].next[letter] = nodes.size();
                nodes.emplace_back();
            }
            v = nodes[v].next[letter];
        }
        nodes[v].output.push_back(pattern_id);
    }

    void build() {
        queue<int> q;
        for (int letter = 0; letter < ALPHABET; ++letter) {
            int child = nodes[0].next[letter];
            if (child == -1) nodes[0].next[letter] = 0;
            else nodes[child].suffix_link = 0, q.push(child);
        }

        while (!q.empty()) {
            int v = q.front(); q.pop();
            int link = nodes[v].suffix_link;
            nodes[v].output_link = !nodes[link].output.empty()
                                   ? link : nodes[link].output_link;
            for (int letter = 0; letter < ALPHABET; ++letter) {
                int child = nodes[v].next[letter];
                if (child == -1) {
                    nodes[v].next[letter] = nodes[link].next[letter];
                } else {
                    nodes[child].suffix_link = nodes[link].next[
     letter];
                    q.push(child);
                }
            }
        }
    }

    vector<pair<int,int>> find_all(const string &text) const {
        vector<pair<int,int>> matches; // (end_position, pattern_id)
        int state = 0;
        for (int position = 0; position < (int)text.size(); ++
     position) {
            int letter = text[position] - 'a';
            assert(0 <= letter && letter < ALPHABET);
            state = nodes[state].next[letter];
            for (int v = state; v != -1; v = nodes[v].output_link)
                for (int pattern_id : nodes[v].output)
                    matches.push_back({position, pattern_id});
        }
        return matches;
    }
};
```

Llamar `build()` exactamente una vez después de insertar todos los patrones. Para contar sin enumerar coincidencias, acumular visitas por el árbol de suffix links.

```
vector<string> patterns={"he","she","his","hers"};
AhoCorasick aho;
for(int id=0;id<(int)patterns.size();++id)
    aho.insert(patterns[id],id);
aho.build(); // Una vez, despues de insertar todos.

string text="ahishers";
for(auto [end,id]:aho.find_all(text)) {
    int start=end-(int)patterns[id].size()+1;
    cout<<patterns[id]<<" ["<<start<<","<<end+1<<")\n";
}
// his [1,4), she [3,6), he [4,6), hers [4,8)
// find_all devuelve (posicion final,id); admite coincidencias
     superpuestas.
```

5.7 Suffix Array y LCP

Cuándo usar: Ordenar todos los sufijos para comparar substrings, buscar patrones, contar substrings distintos o conocer prefijos comunes sin usar hashing.

`sa[i]` es la posición donde comienza el i-ésimo sufijo en orden lexicográfico. `lcp[i]` es la longitud del prefijo común entre los sufijos `sa[i]` y `sa[i+1]`. Para una cadena de longitud n, SA tiene n posiciones y LCP tiene $\max(0, n-1)$ valores. Ambas funciones aceptan la cadena vacía.

Construcción basada en el doubling con counting sort de KACTL (Luo Suiqian y chilli): en cada ronda se ordenan pares de clases que representan prefijos de longitud k, obteniendo clases para longitud $2k$. Ordenar por clases enteras en tiempo lineal por ronda evita el `sort` comparativo.

Internamente se agrega un centinela menor que cualquier carácter y se elimina su posición antes de devolver SA. Se codifica cada byte como `(unsigned char)c+1`, reservando cero para el centinela: no se modifica la entrada y también se admiten bytes NUL y valores mayores que 127. `lcp_array` conserva la convención del handbook y usa Kasai; no devuelve el LCP adicional asociado al centinela de la interfaz original de KACTL.

Complejidad: Suffix Array $O(n \log n)$; LCP $O(n)$; memoria $O(n)$.

```
vector<int> suffix_array(const string &s) {
    int n=(int)s.size()+1, classes=257; // Incluye el centinela.
    vector<int> order(n),rank(n),work(n),count(max(n,classes));
    iota(order.begin(),order.end(),0);
    for(int i=0;i<n-1;++i) rank[i]=(unsigned char)s[i]+1;
    for(int length=0,groups=0;groups<n;
        length=length?2*length:1,classes=groups) {
        int at=length;
        iota(work.begin(),work.begin()+length,n-length);
        for(int position:order)
            if(position>=length) work[at++]=position-length;
        fill(count.begin(),count.begin()+classes,0);
        for(int value:rank) ++count[value];
        for(int i=1;i<classes;++i) count[i]+=count[i-1];
        for(int i=n;i--;)
            order[--count[rank[work[i]]]]=work[i];
        rank.swap(work); // work conserva las clases anteriores.
        groups=1; rank[order[0]]=0;
        for(int i=1;i<n;++i) {
            int a=order[i-1],b=order[i];
            bool same=work[a]==work[b] &&
                (a+length<n?work[a+length]:-1)==
                (b+length<n?work[b+length]:-1);
            rank[b]=same?groups-1:groups++;
        }
    }
    order.erase(order.begin()); // Quitar el sufijo vacío, posición s
      .size().
    return order;
}

vector<int> lcp_array(const string &s,const vector<int> &sa) {
    int n=s.size();
    assert((int)sa.size()==n);
```

```
    if(n<=1) return {};
    vector<int> rank(n),lcp(n-1);
    for(int i=0;i<n;++i) rank[sa[i]]=i;
    for(int position=0,length=0;position<n;++position) {
        int index=rank[position];
        if(index==n-1) {
            length=0;
            continue;
        }
        int next=sa[index+1];
        while(position+length<n&&next+length<n&&
              s[position+length]==s[next+length]) ++length;
        lcp[index]=length;
        if(length) --length;
    }
    return lcp;
}
```

Ejemplo de uso:

```
string s="banana";
vector<int> sa=suffix_array(s);  // 5 3 1 0 4 2
vector<int> lcp=lcp_array(s,sa); // 1 3 0 0 2
// Cantidad de substrings distintos:
ll n=s.size();
ll total=n*(n+1)/2;
ll repeated=accumulate(lcp.begin(),lcp.end(),0LL);
ll distinct=total-repeated;
cout<<distinct<<'\n'; // 15
// Cada lcp[i] cuenta prefijos compartidos por dos sufijos
     consecutivos.
```

La cantidad de substrings distintos es

$$\frac{n(n+1)}{2} - \sum_i lcp[i].$$

Para consultar repetidamente el LCP entre dos sufijos cualesquiera, guardar la posición de cada sufijo en `sa` y construir un RMQ sobre el arreglo `lcp`.

6. Matemática

6.1 Teoremas e identidades para reconocer soluciones

Cuándo usar: Antes de programar, cuando el problema pide contar, demostrar una cota, trabajar módulo primo o transformar una suma aparentemente cuadrática.

Complejidad: Las fórmulas son $O(1)$ una vez conocidos sus datos; inclusión–exclusión cuesta normalmente $O(2^k)$ y Burnside recorre las simetrías. Precomputaciones y factorizaciones conservan sus propios costos. La memoria depende del algoritmo elegido; las identidades solas usan $O(1)$.

6.1.1 Conteo y cotas

- **Palomar:** al repartir m objetos entre n cajas, alguna recibe al menos $\lceil m/n \rceil$. Se usa para demostrar que una repetición o colisión es inevitable sin identificar cuál. Por ejemplo, entre 13 personas al menos dos cumplen años en el mismo mes.
- **Handshake:**

$$\sum_v deg(v) = 2|E|.$$

  Cada arista aporta uno al grado de cada extremo, por eso se cuenta dos veces. Como la suma es par, la cantidad de vértices de grado impar también es par. Es útil para validar secuencias de grados y recorridos eulerianos.
- **Suma triangular:**

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}.$$

  Aparece cuando la iteración i cuesta i, se eligen pares o cada paso consume una unidad más. Si la suma no supera n, entonces $k \approx \sqrt{2n}$ y por tanto $k = O(\sqrt{n})$.
- **Serie armónica entera:**

$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = O(n \log n).$$

  Cuenta, por ejemplo, todos los múltiplos procesados por bucles de la forma `for(j=i;j<=n;j+=i)`. Además, $\lfloor n/i \rfloor$ toma sólo $O(\sqrt{n})$ valores distintos; para un cociente q, el último índice con el mismo valor es $\lfloor n/q \rfloor$, permitiendo agrupar rangos.
- **Inclusión–exclusión:** suma los casos que cumplen cada propiedad y corrige los contados varias veces:

$$|A \cup B| = |A| + |B| - |A \cap B|,$$

$$|A \cup B \cup C| = \sum |A_i| - \sum |A_i \cap A_j| + |A \cap B \cap C|.$$

  Para k propiedades se recorren normalmente las 2^k intersecciones. Se usa para contar objetos que cumplen al menos una restricción o, tomando el complemento, los que no cumplen ninguna.

6.1.2 Combinatoria

Antes de elegir una fórmula. Para tomar k elementos entre n opciones, preguntar si importa el orden y si una opción puede repetirse. Cambiar dos elementos importa si produce otro resultado: un equipo no tiene orden, pero los cargos de presidente y vicepresidente sí.

Orden	Repetición	Cantidad	Ejemplo
No	No	$\binom{n}{k}$	3 personas entre 5: 10
Sí	No	$n!/(n-k)!$	3 cargos entre 5: 60
Sí	Sí	n^k	clave de 4 dígitos: 10^4
No	Sí	$\binom{n+k-1}{k}$	3 bochas, 5 sabores: 35

Las opciones de cada caso no tienen restricciones adicionales. En la última fila $n \geq 1$: sólo importa cuántas veces se elige cada tipo, no el orden de las elecciones. En C++, calcular n^k módulo m con `mod_power(n,k,m)`, no con `pow` de punto flotante.

Los snippets aparecen junto a cada fórmula. Requieren los tipos del template base, las funciones de "Aritmética modular" y, donde se usa, la clase `Combinations` de "Combinatoria módulo primo". Al copiarlos a C++, colocar esas dependencias antes de las funciones.

```
void counting_examples() {
    const ll MOD = 1000000007;
    Combinations c(100, MOD); // Incluye el maximo n de cada formula.
    cout << c.choose(5, 3) << '\n'; // 10: equipo sin cargos.
    cout << mod_power(10, 4, MOD) << '\n'; // 10000: clave con
      repeticion.
    auto division = mod_divide(10, 3, 7);
    cout << *division << '\n'; // 1: 10 * inverso(3) modulo 7.
    assert(!mod_inverse(2, 4)); // No hay inverso; no significa "
      conteo cero".
}
```

- **Reglas de suma y producto:** si los casos son excluyentes, sus cantidades se suman. Si una elección de a formas va seguida de otra independiente de b formas, se multiplican: ab. Por ejemplo, 3 camisas y 2 pantalones producen $3 \cdot 2 = 6$ conjuntos.
- **Permutaciones:** ordenar n objetos distintos produce $n!$ órdenes. Elegir y ordenar k de ellos produce

$$P(n,k) = n(n-1)\cdots(n-k+1) = \frac{n!}{(n-k)!}.$$

  Se usa cuando el orden importa; elegir presidente y vicepresidente entre n personas da $P(n,2)$, no $\binom{n}{2}$.
- **Combinaciones:** elegir k objetos entre n sin importar el orden:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} = \binom{n}{n-k}.$$

  Si un multiconjunto tiene cantidades $c_1, \ldots, c_r$ con $\sum c_i = n$, sus órdenes distintos son

$$\frac{n!}{c_1!c_2!\cdots c_r!}.$$

  Por ejemplo, `AAB` tiene $3!/2! = 3$ permutaciones distintas. `BANANA` tiene $6!/(3!2!1!) = 60$: permutar las tres A entre sí no crea otra palabra, y lo mismo ocurre con las dos N. Para el snippet, las cantidades son no negativas y su suma n debe estar precomputada, con $n < p$. Usar factoriales inversos, no división entera de valores ya reducidos módulo p.

```
ll multinomial(const vector<int> &counts, const Combinations &c) {
    int n = accumulate(counts.begin(), counts.end(), 0);
    ll answer = c.factorial[n];
    for (int amount : counts)
        answer = answer * c.inverse_factorial[amount] % c.mod;
    return answer; // {3,2,1}: BANANA tiene 60 ordenes.
}
```

  Consulta $O(k)$ para k tipos; memoria adicional $O(1)$ tras el preproceso.
- **Subconjuntos:** cada uno de los n elementos puede estar o no, por lo que existen 2^n subconjuntos. Los de tamaño k son $\binom{n}{k}$ y

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n.$$

- **Identidad de Pascal:**

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

  Al fijar un elemento, una selección o lo contiene o no lo contiene. Permite calcular combinaciones mediante DP, incluso cuando no existe división modular. Es la alternativa para n chico y módulo compuesto; no exige primalidad ni inversos.

```
vector<vector<ll>> pascal_mod(int n, ll mod) {
    assert(n >= 0 && mod > 0);
    vector<vector<ll>> c(n + 1, vector<ll>(n + 1));
    for (int i = 0; i <= n; ++i) {
        c[i][0] = c[i][i] = 1 % mod;
        for (int j = 1; j < i; ++j)
            c[i][j] = mod_add(c[i-1][j-1], c[i-1][j], mod);
    }
    return c; // pascal_mod(5, 12)[5][2] == 10.
}
```

  Construcción $O(N^2)$ tiempo y memoria; consulta $O(1)$. Si sólo hace falta una fila, actualizar de derecha a izquierda usa $O(N)$ memoria.

- **Hockey-stick:**

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}.$$

Convierte una suma de combinaciones consecutivas en una sola. Combinatoriamente, clasifica cada selección de $r+1$ elementos según cuál es su elemento mayor. Reconocerla cuando el índice inferior queda fijo mientras el superior recorre un intervalo consecutivo.

- **Binomio de Newton:**

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}.$$

El coeficiente $\binom{n}{k}$ cuenta en cuáles k de los n factores se elige x. Sustituir valores produce identidades; con $x = y = 1$ se obtiene $\sum_k \binom{n}{k} = 2^n$.

- **Stars and bars:** la cantidad de soluciones enteras no negativas de

$$x_1 + \cdots + x_k = n$$

es $\binom{n+k-1}{k-1}$: se ordenan n estrellas y $k-1$ separadores. Si todas las variables deben ser positivas, entregar una unidad a cada una deja $n-k$ por repartir y resulta $\binom{n-1}{k-1}$. Sirve para distribuir objetos idénticos entre cajas distinguibles, con $n \geq 0$ y $k \geq 1$. Para positivos se necesita $n \geq k$; de otro modo hay cero soluciones.
Ejemplo: repartir 6 caramelos idénticos entre 3 personas da $\binom{8}{2} = 28$ formas permitiendo ceros, o $\binom{5}{2} = 10$ si todos reciben. Con mínimos $x_i \geq l_i$, sustituir $y_i = x_i - l_i$ y repartir $n - \sum l_i$; si es negativo, la respuesta es cero.
Con cotas superiores $x_i \leq u_i$, usar inclusión–exclusión:

$$\sum_{S \subseteq \{1,\ldots,k\}} (-1)^{|S|} \binom{n - \sum_{i \in S}(u_i+1) + k - 1}{k-1}.$$

Cada caja que viola su cota recibe al menos $u_i + 1$: retirarlos transforma esa intersección en otro reparto sin cotas. Si quedan objetos negativos, el término vale cero. Permitiendo ceros, 6 caramelos en 3 cajas de capacidad 3 dan $28 - 3\binom{4}{2} = 10$; dos cajas no pueden recibir 4 a la vez. El siguiente código implementa las cotas superiores, sin mínimos. Si también hay mínimos, retirarlos y reemplazar u_i por $u_i - l_i$; una capacidad negativa hace imposible el reparto. Requiere $n \geq 0$, capacidades no negativas, k pequeño (por ejemplo $k \leq 20$) y factoriales hasta $n + k - 1 < p$.

```
ll bounded_distributions(int n, const vector<int> &upper,
                         const Combinations &c) {
    int k = upper.size();
    assert(n >= 0 && 1 <= k && k < 63);
    ll answer = 0;
    for (unsigned long long mask = 0; mask < (1ULL << k); ++mask) {
        ll remaining = n;
        for (int i = 0; i < k; ++i) if (mask >> i & 1ULL)
            remaining -= (ll)upper[i] + 1;
        if (remaining < 0) continue; // Interseccion imposible.
        ll term = c.choose((int)remaining + k - 1, k - 1);
        answer = __builtin_popcountll(mask) & 1
               ? mod_subtract(answer, term, c.mod)
               : mod_add(answer, term, c.mod);
    }
    return answer; // n=6, upper={3,3,3}: devuelve 10.
}
```

Tiempo $O(k2^k)$ y memoria adicional $O(1)$ tras el preproceso.

- **Vandermonde:**

$$\sum_{i} \binom{r}{i}\binom{s}{n-i} = \binom{r+s}{n}.$$

Elegir n elementos de dos grupos de tamaños r y s puede hacerse directamente o fijando cuántos i se toman del primero. Por ejemplo, elegir 3 personas entre 4 mujeres y 3 hombres da $\sum_{i=0}^{3} \binom{4}{i}\binom{3}{3-i} = \binom{7}{3} = 35$.

- **Catalan:**

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}.$$

Cuenta estructuras que nunca cruzan una frontera: paréntesis balanceados con n pares, caminos que no bajan de cero, triangulaciones de un polígono de $n+2$ lados y varios árboles ordenados. Los primeros valores son $1, 1, 2, 5, 14, 42$. En paréntesis, todo prefijo debe tener al menos tantas aperturas como cierres: `)(` tiene cantidades iguales, pero es inválido. Para 3 pares hay $C_3 = 5$ secuencias válidas. La expresión con resta evita dividir por $n+1$; el método elegido para calcular los binomios mantiene sus propias precondiciones. Con `Combinations`, precomputar hasta $2n < p$:

```
ll catalan(int n, const Combinations &c) {
    return mod_subtract(c.choose(2*n, n), c.choose(2*n, n+1), c.mod
      );
} // catalan(0,c)=1; catalan(3,c)=5.
```

Consulta $O(1)$ tiempo y memoria adicional, tras el preproceso.

- **Desarreglos:** permutaciones donde ningún elemento queda en su posición original:

$$D_n = (n-1)(D_{n-1} + D_{n-2}), \qquad D_0 = 1,\ D_1 = 0.$$

Al decidir dónde va el primer elemento, el caso restante se reduce a un desarreglo de $n-1$ o de $n-2$. Aparece en asignaciones donde nadie puede recibir su objeto original. Por ejemplo, repartir regalos entre 4 personas sin que nadie reciba el suyo da $D_4 = 9$. No sirve directamente si se prohíben otras parejas arbitrarias además de la diagonal.

```
vector<ll> derangements(int n, ll mod) {
    assert(n >= 0 && mod > 0);
    vector<ll> d(n + 1);
    d[0] = 1 % mod;
    for (int i = 2; i <= n; ++i)
        d[i] = mod_multiply(i-1, mod_add(d[i-1], d[i-2], mod), mod)
      ;
    return d; // derangements(4,1000000007)[4] == 9.
}
```

Tiempo y memoria $O(n)$; admite módulo compuesto. Si sólo se necesita D_n, conservar los dos valores anteriores reduce memoria a $O(1)$.

- **Burnside:** para contar configuraciones iguales bajo un grupo de rotaciones o reflexiones,

$$\#\text{clases} = \frac{1}{|G|} \sum_{g \in G} |\operatorname{Fix}(g)|.$$

Para cada simetría se cuentan las configuraciones que no cambian y luego se promedia. Es útil para collares, pulseras, coloraciones y tableros considerados iguales bajo simetrías. No dividir simplemente el total por $|G|$: algunas configuraciones quedan fijas bajo varias simetrías.

Ejemplo: colorear los 4 vértices de un cuadrado con 2 colores, considerando iguales sólo las rotaciones. Los giros de $0, 90, 180, 270$ grados fijan respectivamente $16, 2, 4, 2$ coloraciones, así que hay $(16+2+4+2)/4 = 6$ clases. Si también se permiten reflexiones, hay que incorporarlas al grupo.
En general, un giro de r posiciones de un ciclo de n posiciones tiene $\gcd(n, r)$ ciclos; una coloración fija debe ser constante en cada ciclo. Con c colores hay $c^{\gcd(n,r)}$ coloraciones fijas, y el promedio es

$$\frac{1}{n} \sum_{r=0}^{n-1} c^{\gcd(n,r)}.$$

El snippet modular requiere que n tenga inverso módulo m.

```
ll rotation_colorings(int n, ll colors, ll mod) {
    assert(n > 0 && colors >= 0 && mod > 1);
    auto inverse = mod_inverse(n, mod);
    assert(inverse); // Si no existe, este promedio modular no se
      puede usar.
    ll sum = 0;
    for (int r = 0; r < n; ++r)
        sum = mod_add(sum, mod_power(colors, gcd(n, r), mod), mod);
    return mod_multiply(sum, *inverse, mod);
} // rotation_colorings(4,2,1000000007) == 6.
```

Tiempo $O(n \log n + \log m)$ y memoria adicional $O(\log m)$ por Euclides. Si n no es invertible módulo m, el conteo sigue existiendo, pero hace falta otro tratamiento para hacer la división exacta.

6.1.3 Teoría de números

Si $n = \prod p_i^{a_i}$, cada divisor elige para p_i un exponente entre 0 y a_i. De esta observación se obtienen:

$$\tau(n) = \prod_i (a_i+1), \qquad \sigma(n) = \prod_i \frac{p_i^{a_i+1} - 1}{p_i - 1}, \qquad \varphi(n) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right).$$

$\tau(n)$ cuenta divisores, $\sigma(n)$ suma sus valores y $\varphi(n)$ cuenta enteros de 1 a n coprimos con n. Por ejemplo, $12 = 2^2 \cdot 3$ da $\tau(12) = 6$, $\sigma(12) = 28$ y $\varphi(12) = 4$.

- **Bézout:** existen enteros x, y tales que

$$ax + by = \gcd(a, b).$$

Por tanto, $ax+by = c$ tiene soluciones enteras si y sólo si $\gcd(a,b) \mid c$. Euclides extendido encuentra x, y y permite construir inversos modulares cuando el gcd es 1. Para $a, b > 0$, si $ax_0 + by_0 = g = \gcd(a,b)$ y $g \mid c$, todas las soluciones son

$$x = x_0(c/g) + (b/g)t, \qquad y = y_0(c/g) - (a/g)t, \quad t \in \mathbb{Z}.$$

Si se exigen soluciones no negativas o acotadas, restringir t usando ambas desigualdades; una solución entera cualquiera no garantiza cumplirlas.

- **Pequeño teorema de Fermat:** si p es primo y $p \nmid a$,

$$a^{p-1} \equiv 1 \pmod{p}.$$

Entonces $a^{-1} \equiv a^{p-2} \pmod{p}$ y los exponentes pueden reducirse módulo $p-1$ cuando la base no es divisible por p.

- **Teorema de Euler:** si $\gcd(a, m) = 1$, entonces

$$a^{\varphi(m)} \equiv 1 \pmod{m}.$$

Generaliza Fermat a módulos no primos y permite reducir exponentes módulo $\varphi(m)$ bajo la condición de coprimalidad. No omitirla: $2^2 \bmod 4 = 0$, pero reducir el exponente módulo $\varphi(4) = 2$ produciría $2^0 \bmod 4 = 1$, que es incorrecto.

- **Wilson:** $p > 1$ es primo si y sólo si

$$(p-1)! \equiv -1 \pmod{p}.$$

Es útil en demostraciones y algunos cálculos cercanos a p, pero calcular el factorial cuesta $O(p)$ y no es una prueba de primalidad eficiente para valores grandes.

6.1.4 Grafos y geometría

- **Caracterización de árboles:** un árbol conectado con n vértices tiene exactamente $n-1$ aristas. En un árbol existe un único camino entre cada par; agregar una arista crea exactamente un ciclo y quitar cualquiera lo desconecta. Un grafo conectado con $n-1$ aristas es un árbol.
- **Caminos eulerianos:** ignorando vértices aislados, las aristas deben pertenecer a una sola componente conexa. Existe circuito euleriano si todos los grados son pares; existe camino abierto que usa cada arista una vez si exactamente dos grados son impares, que serán sus extremos. Con cero impares también existe camino, pero comienza y termina en el mismo vértice.
- **Fórmula de Euler planar:** para un grafo planar conexo,

$$V - E + F = 2,$$

donde F incluye la cara exterior. Para un grafo planar simple con $V \geq 3$ implica $E \leq 3V - 6$, una cota útil para demostrar que un grafo no es planar o que un algoritmo sobre todas sus aristas es lineal en V.
- **Teorema de Pick:** para un polígono simple cuyos vértices son puntos enteros,

$$A = I + \frac{B}{2} - 1,$$

donde I es la cantidad de puntos enteros interiores y B los de la frontera. Los puntos de frontera de un lado (x_1, y_1)–(x_2, y_2) se cuentan mediante $\gcd(|x_2 - x_1|, |y_2 - y_1|)$ al sumar todos los lados.

6.2 GCD, LCM y Euclides extendido

Cuándo usar: Divisibilidad, simplificar fracciones, resolver $ax+by = gcd(a,b)$ o construir inversos cuando los números son coprimos.
Complejidad: $O(\log \min(|a|, |b|))$ tiempo; memoria logarítmica por el stack en la versión recursiva.

```
optional<ll> checked_lcm(ll a, ll b) {
    if (a == 0 || b == 0) return 0;
    if (a == numeric_limits<ll>::min() || b == numeric_limits<ll>::
     min())
        return nullopt;
    ll divisor = gcd(a, b);
    i128 value = (i128)(a / divisor) * b;
    if (value < 0) value = -value;
    if (value > numeric_limits<ll>::max()) return nullopt;
    return (ll)value;
}

ll extended_gcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) { x = 1; y = 0; return a; }
    ll x1, y1;
    ll g = extended_gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```

6.3 Aritmética modular

Cuándo usar: Potencias, divisiones e inversos bajo módulo; el inverso por Fermat requiere módulo primo y denominador no divisible por él.

```
ll mod_normalize(ll value, ll mod) {
    assert(mod > 0);
    value %= mod;
    if (value < 0) value += mod;
    return value;
}

ll mod_add(ll a, ll b, ll mod) {
    return ((i128)mod_normalize(a, mod) + mod_normalize(b, mod)) %
     mod;
}

ll mod_subtract(ll a, ll b, ll mod) {
    i128 result = (i128)mod_normalize(a, mod) - mod_normalize(b, mod)
     ;
    if (result < 0) result += mod;
    return (ll)result;
}

ll mod_multiply(ll a, ll b, ll mod) {
    return (i128)mod_normalize(a, mod) * mod_normalize(b, mod) % mod;
}

ll mod_power(ll base, unsigned long long exponent, ll mod) {
    ll result = 1 % mod;
    base = mod_normalize(base, mod);
    while (exponent) {
        if (exponent & 1ULL) result = mod_multiply(result, base, mod)
     ;
        base = mod_multiply(base, base, mod);
        exponent >>= 1;
    }
    return result;
}

optional<ll> mod_inverse(ll a, ll mod) {
    assert(mod > 0);
    ll x, y;
    ll g = extended_gcd(mod_normalize(a, mod), mod, x, y);
    if (g != 1) return nullopt;
    return mod_normalize(x, mod);
}

optional<ll> mod_divide(ll a, ll b, ll mod) {
    auto inverse = mod_inverse(b, mod);
    if (!inverse) return nullopt;
    return mod_multiply(a, *inverse, mod);
}
```

Complejidad: $O(\log e)$ para potencia; $O(\log m)$ para inverso; memoria $O(1)$ salvo el stack de Euclides.

6.4 Teorema chino del resto para dos congruencias

Te permite saber **qué números cumplen las dos condiciones al mismo tiempo** y **cada cuánto se repiten**.
Cuándo usar: Combinar condiciones $x \equiv a \pmod{m}$; esta versión también detecta si dos congruencias con módulos no coprimos son incompatibles.
Complejidad: $O(\log \min(m_1, m_2))$ tiempo por combinación; memoria logarítmica por Euclides extendido.
Admite módulos no coprimos. Devuelve la solución normalizada y el módulo mínimo, o `nullopt` si no hay solución o el módulo no entra en `long long`.

```
optional<pair<ll,ll>> crt_pair(ll a1, ll m1, ll a2, ll m2) {
    assert(m1 > 0 && m2 > 0);
    a1 = mod_normalize(a1, m1);
    a2 = mod_normalize(a2, m2);
    ll x, y;
    ll g = extended_gcd(m1, m2, x, y);
    ll difference = a2 - a1;
    if (difference % g != 0) return nullopt;

    i128 modulus = (i128)(m1 / g) * m2;
    if (modulus > numeric_limits<ll>::max()) return nullopt;
    ll reduced = m2 / g;
    i128 t = (i128)(difference / g) * x % reduced;
    if (t < 0) t += reduced;
    i128 answer = ((i128)a1 + (i128)m1 * t) % modulus;
    if (answer < 0) answer += modulus;
    return pair<ll,ll>{(ll)answer, (ll)modulus};
}
```

6.5 Combinatoria módulo primo

Cuándo usar: Muchas consultas de combinaciones/permutaciones módulo primo con $n < mod$, precomputando factoriales e inversos. Para n enorme y primo pequeño, Lucas.
Precalcula factoriales para responder $\binom{n}{k}$ cuando el módulo es primo y $n < mod$.

```
struct Combinations {
    ll mod;
    vector<ll> factorial, inverse_factorial;

    Combinations(int maximum_n, ll prime_mod) : mod(prime_mod),
        factorial(max(0, maximum_n) + 1, 1),
        inverse_factorial(max(0, maximum_n) + 1, 1) {
        assert(maximum_n >= 0 && maximum_n < prime_mod);
        for (int i = 1; i <= maximum_n; ++i)
            factorial[i] = factorial[i - 1] * i % mod;
        inverse_factorial[maximum_n] = mod_power(
            factorial[maximum_n], mod - 2, mod);
        for (int i = maximum_n; i > 0; --i)
            inverse_factorial[i - 1] = inverse_factorial[i] * i % mod
     ;
    }

    ll choose(int n, int k) const {
        if (k < 0 || k > n) return 0;
        return factorial[n] * inverse_factorial[k] % mod
             * inverse_factorial[n - k] % mod;
    }
};
```

Complejidad: preproceso $O(N + \log mod)$; consulta $O(1)$; memoria $O(N)$.

6.5.1 Elegir el método antes de programar

- Muchas consultas, primo p y máximo $N < p$: usar `Combinations(N,p)`. Consultar sólo $0 \leq n \leq N$.
- n chico y módulo cualquiera: usar Pascal, sin inversos.
- n enorme y primo pequeño: usar Lucas; reserva $O(p)$ valores, por lo que no conviene con $p = 10^9 + 7$.

Los snippets junto a los teoremas usan las funciones de “Aritmética

modular" y `Combinations`. En esta plantilla los productos de factoriales usan `long long`: para 10^9+7 y 998244353 entran; con un primo grande de 64 bits se deben promover los productos antes de multiplicar. Las fórmulas exactas no autorizan usar división entera después de tomar módulo: dividir requiere un inverso coprimo con el módulo.

6.6 Teorema de Lucas

Cuándo usar: Calcular $\binom{n}{k} \bmod p$ con p primo pequeño y n, k demasiado grandes para precomputar factoriales hasta n.
Usar para calcular $\binom{n}{k} \bmod p$ cuando p es primo pequeño pero n y k pueden ser enormes. Se descomponen ambos números en base p.
Complejidad: preproceso $O(p)$; consulta $O(\log_p n)$; memoria $O(p)$.

```
struct LucasTheorem {
    ll prime;
    vector<ll> factorial, inverse_factorial;

    explicit LucasTheorem(ll modulus)
        : prime(modulus), factorial(prime), inverse_factorial(prime)
     {
        factorial[0] = 1;
        for (ll value = 1; value < prime; ++value)
            factorial[value] = factorial[value - 1] * value % prime;
        inverse_factorial[prime - 1] = mod_pow(factorial[prime - 1],
     prime - 2);
        for (ll value = prime - 1; value > 0; --value)
            inverse_factorial[value - 1] = inverse_factorial[value] *
      value % prime;
    }

    ll mod_pow(ll base, ll exponent) const {
        ll result = 1;
        while (exponent) {
            if (exponent & 1) result = result * base % prime;
            base = base * base % prime;
            exponent >>= 1;
        }
        return result;
    }

    ll small_combination(ll n, ll k) const {
        if (k < 0 || k > n) return 0;
        return factorial[n] * inverse_factorial[k] % prime
             * inverse_factorial[n - k] % prime;
    }

    ll combination(ll n, ll k) const {
        ll answer = 1;
        while (n || k) {
            answer = answer * small_combination(n % prime, k % prime)
      % prime;
            n /= prime; k /= prime;
        }
        return answer;
    }
};
```

Cuidado: Requiere p primo y guardar $O(p)$ valores; no usar si el módulo es grande.

6.7 Criba lineal y menor factor primo

Cuándo usar: Muchas consultas de primalidad/factorización para valores acotados por N; precomputar una vez y factorizar luego por el menor primo.

```
struct LinearSieve {
    vector<int> primes, smallest_prime;

    explicit LinearSieve(int n) : smallest_prime(max(0, n) + 1, 0) {
        for (int value = 2; value <= n; ++value) {
            if (smallest_prime[value] == 0) {
                smallest_prime[value] = value;
                primes.push_back(value);
            }
            for (int prime : primes) {
                if (prime > smallest_prime[value] || 1LL * value *
     prime > n)
                    break;
                smallest_prime[value * prime] = prime;
            }
        }
    }

    vector<int> factorize(int value) const {
        assert(1 <= value && value < (int)smallest_prime.size());
        vector<int> factors;
        while (value > 1) {
            factors.push_back(smallest_prime[value]);
            value /= smallest_prime[value];
        }
        return factors;
    }
};
```

Complejidad: preproceso $O(N)$; factorización $O(\log n)$; memoria $O(N)$.

6.8 Criba segmentada: primos en un intervalo

Cuándo usar: Listar los primos de $[L, R]$ cuando el intervalo es corto y R es grande. Debe ser viable cribar hasta $\lfloor\sqrt{R}\rfloor$ y almacenar $R-L+1$ marcas; por ejemplo, $R \le 10^{12}$ y ancho hasta 10^6.
Complejidad: Con $B = \lfloor\sqrt{R}\rfloor$ y $W = R - L + 1$, tiempo $O(B \log\log B + \pi(B) + W \log\log R)$ y memoria $O(B + W)$ bits para las marcas, más $O(k)$ palabras para devolver los k primos. Las cotas se entienden asintóticamente; intervalos con $R < 2$ se resuelven directamente.

```
// Requiere 0 <= L <= R y memoria/tiempo suficientes para sqrt(R) y
     el ancho.
vector<ll> segmented_primes(ll L, ll R) {
    assert(0 <= L && L <= R);
    if (R < 2) return {};

    ll root = (ll)sqrtl((long double)R);
    while (root > R / root) --root;
    while (root + 1 <= R / (root + 1)) ++root;

    vector<bool> base((size_t)root + 1, true);
    base[0] = base[1] = false;
    vector<bool> segment((size_t)(R - L) + 1, true);

    for (ll p = 2; p <= root; ++p) {
        if (!base[p]) continue;
        if (p <= root / p)
            for (ll x = p * p; x <= root; x += p)
                base[x] = false;

        // Primer multiplo >= L, sin usar (L+p-1) ni una suma fuera
     de R.
        ll delta = (p - L % p) % p;
        if (delta > R - L) continue;
        ll start = max(p * p, L + delta);
        if (start > R) continue;
        for (ll x = start; ; ) {
            segment[x - L] = false;
            if (R - x < p) break;
            x += p;
        }
    }

    if (L == 0) segment[0] = false;
    if (L <= 1) segment[1 - L] = false;
    vector<ll> answer;
    for (size_t i = 0; i < segment.size(); ++i)
        if (segment[i]) answer.push_back(L + (ll)i);
    return answer;
}
```

Ejemplo: `segmented_primes(100,120)` devuelve $101, 103, 107, 109, 113$. Se marca desde $\max(p^2, \lceil L/p\rceil p)$ para no borrar al propio primo p. No construye la criba completa hasta R. Tampoco evita el costo de cribar hasta $\sqrt{R}$: para un solo entero grande, usar Miller–Rabin. Si hay muchas consultas con el mismo límite, conviene reutilizar los primos base; esta versión los recalcula para mantener una interfaz autocontenida.

6.9 Factorización por división por tentativa

Cuándo usar: Una o pocas consultas con n moderado, por ejemplo hasta 10^9. No necesita criba ni generador aleatorio. Para muchas consultas, usar SPF; para factores grandes de 64 bits, Miller–Rabin y Brent.
Complejidad: $O(\sqrt{n})$ tiempo en el peor caso; $O(\log n)$ memoria de salida.

```
vector<ll> trial_factorize(ll n) {
    assert(n >= 1);
    vector<ll> out;
    while (n % 2 == 0) { out.push_back(2); n /= 2; }
    for (ll d = 3; d <= n / d; d += 2)
        while (n % d == 0) { out.push_back(d); n /= d; }
    if (n > 1) out.push_back(n);
    return out;
}
```

Devuelve factores ordenados con repeticiones; para 1, un vector vacío. La condición `d <= n/d` evita overflow de `d*d`.

6.10 Listar divisores

Cuándo usar: Listar todos los divisores de un número aislado moderado sin necesitar preprocesamiento.

```
vector<ll> divisors(ll n) {
    assert(n > 0);
    vector<ll> answer;
    for (ll d = 1; d <= n / d; ++d) if (n % d == 0) {
        answer.push_back(d);
        if (d != n / d) answer.push_back(n / d);
    }
    sort(answer.begin(), answer.end());
    return answer;
}
```

Complejidad: $O(\sqrt{n})$ tiempo y $O(\tau(n))$ memoria de salida.

6.11 Pollard–Rho con Brent

Cuándo usar: Factorizar enteros de 64 bits con poco código. Incluye Miller–Rabin determinista de siete bases y Brent con bloque 128 y reinicios.

Complejidad: Primalidad: $O(\log n)$ productos modulares. Para encontrar un factor p, tiempo heurístico $O(\sqrt{p})$, aproximadamente $O(n^{1/4})$ en el caso difícil; memoria $O(\log n)$ por recursión. Sin cota de tiempo total por los reinicios.

Esta versión es completa: copiar sus dos bloques. Usa `u128%n`; no requiere el código optimizado de abajo. Requiere $1 \le n < 2^{64}$. Miller–Rabin verifica primalidad; Brent encuentra un divisor compuesto o primo.

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 multiply_mod_u64(u64 a, u64 b, u64 mod) {
    return (u128)a * b % mod;
}

u64 power_mod_u64(u64 base, u64 exponent, u64 mod) {
    u64 result = 1;
    while (exponent) {
        if (exponent & 1) result = multiply_mod_u64(result, base, mod
     );
        base = multiply_mod_u64(base, base, mod);
        exponent >>= 1;
    }
    return result;
}

bool is_prime_u64(u64 n) {
    if (n < 2) return false;
    for (u64 prime : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL,
                      19ULL, 23ULL, 29ULL, 31ULL, 37ULL})
        if (n % prime == 0) return n == prime;

    int power_of_two = 0;
    u64 odd = n - 1;
    while ((odd & 1) == 0) odd >>= 1, ++power_of_two;

    for (u64 base : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL,
                     9780504ULL, 1795265022ULL}) {
        if (base % n == 0) continue;
        u64 value = power_mod_u64(base % n, odd, n);
        if (value == 1 || value == n - 1) continue;
        bool witness = true;
        for (int round = 1; round < power_of_two; ++round) {
            value = multiply_mod_u64(value, value, n);
            if (value == n - 1) { witness = false; break; }
        }
        if (witness) return false;
    }
    return true;
}
```

Brent y factorización. Un MCD por bloque; si da n, recuperar el bloque individualmente. Reiniciar si no se obtiene un divisor propio.

```
u64 pollard_rho(u64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    static mt19937_64 rng(chrono::steady_clock::now()
                          .time_since_epoch().count());

    while (true) {
        u64 c = uniform_int_distribution<u64>(1, n - 1)(rng);
        u64 y = uniform_int_distribution<u64>(0, n - 1)(rng);
        u64 x = 0, saved = y, divisor = 1;
        constexpr u64 BLOCK = 128, LIMIT = 1ULL << 18;
        u64 length = 1, batch = 0;
        auto advance = [&](u64 value) {
            return (u64)(((u128)multiply_mod_u64(value, value, n) + c
     ) % n);
        };
        while (divisor == 1 && length <= LIMIT) {
            x = y;
            for (u64 i = 0; i < length; ++i) y = advance(y);
            for (u64 offset = 0; offset < length && divisor == 1;
                 offset += batch) {
                saved = y;
                batch = min(BLOCK, length - offset);
                u64 product = 1;
                for (u64 i = 0; i < batch; ++i) {
                    y = advance(y);
                    u64 difference = x > y ? x - y : y - x;
                    product = multiply_mod_u64(product, difference, n
     );
                }
                divisor = gcd(product, n);
            }
            length <<= 1;
        }
        if (divisor == n) {
            divisor = 1;
            for (u64 i = 0; i < batch && divisor == 1; ++i) {
                saved = advance(saved);
                u64 difference = x > saved ? x - saved : saved - x;
                divisor = gcd(difference, n);
            }
        }
        if (divisor > 1 && divisor < n) return divisor;
    }
}

void factor_u64(u64 n, map<u64,int> &factors) {
    assert(n >= 1);
    if (n == 1) return;
    if (is_prime_u64(n)) { ++factors[n]; return; }
    u64 divisor = pollard_rho(n);
    factor_u64(divisor, factors);
    factor_u64(n / divisor, factors);
}
```

Uso: `factor_u64(n, factores)`, donde `factores` es un `map<u64,int>`. Agrega los exponentes al mapa; para $n = 1$ no agrega nada. Llamar directamente a `pollard_rho` solo con un compuesto. Las siete bases garantizan exactitud de primalidad para 64 bits; la aleatoriedad de Brent afecta el tiempo, no los factores.

6.12 Pollard–Rho con Brent optimizado

Cuándo usar: Muchas factorizaciones o límites ajustados. Esta alternativa completa incluye Montgomery, Miller–Rabin de siete bases, Brent con bloque 256, reinicios, extracción de primos pequeños, cuadrados y acumulación en vector.

Complejidad: Conserva el tiempo heurístico de Brent: $O(\sqrt{p})$ operaciones modulares para hallar un factor p. Ordenar k factores: $O(k \log k)$; memoria de salida y recursión $O(\log n)$. No garantiza un tiempo total máximo.

Copiar **solo este bloque**: no depende de la versión anterior. Se usa un namespace para que ambas opciones puedan convivir en el notebook. La entrada es `rho_optimized::factorize(n)` y requiere $n \ge 1$; devuelve factores primos ordenados con repeticiones, o un vector vacío para 1. La primalidad está disponible como `rho_optimized::is_prime(n)`.

Montgomery representa a como $a2^{64} \bmod n$ y evita dividir en cada producto. La reducción por diferencia de partes altas evita necesitar un acarreo de 129 bits, incluso cuando $n \ge 2^{63}$. Los pares se quitan antes de construir el contexto. Cada contexto guarda su propio módulo.

```
namespace rho_optimized {
using u64 = uint64_t;
using u128 = __uint128_t;

constexpr array<u64, 12> SMALL_PRIMES64 = {
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37
};

struct Montgomery64 {
    u64 mod, inverse = 1, r2;
    explicit Montgomery64(u64 n) : mod(n) {
        assert(n > 1 && (n & 1));
        // Inversa positiva de n modulo 2^64; overflow unsigned
      intencional.
        for (int i = 0; i < 6; ++i)
            inverse *= 2 - mod * inverse;
        u64 r = (u128(1) << 64) % mod;
        r2 = (u128)r * r % mod;
    }
    // Requiere t < mod * 2^64. Todo producto de residuos < mod
      cumple.
    u64 reduce(u128 t) const {
        u64 q = (u64)t * inverse;
        u64 hi = t >> 64;
        u64 subtract = ((u128)q * mod) >> 64;
        return hi >= subtract ? hi - subtract : mod - (subtract - hi)
      ;
    }
    u64 encode(u64 a) const { return mul(a % mod, r2); }
    u64 decode(u64 a) const { return reduce(a); }
    u64 one() const { return reduce(r2); }
    u64 mul(u64 a, u64 b) const { return reduce((u128)a * b); }
    u64 add(u64 a, u64 b) const {
        return a >= mod - b ? a - (mod - b) : a + b;
    }
};

u64 power_mod_encoded(const Montgomery64& ctx, u64 base, u64 exponent
     ) {
    u64 result = ctx.one();
    while (exponent) {
        if (exponent & 1) result = ctx.mul(result, base);
        base = ctx.mul(base, base);
        exponent >>= 1;
    }
    return result;
}

bool is_prime(u64 n) {
    if (n < 2) return false;
    for (u64 p : SMALL_PRIMES64)
        if (n % p == 0) return n == p;

    int s = 0;
    u64 d = n - 1;
```

```
    while ((d & 1) == 0) d >>= 1, ++s;
    Montgomery64 ctx(n);
    u64 one = ctx.one(), minus_one = ctx.encode(n - 1);
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL,
                  9780504ULL, 1795265022ULL}) {
        if (a % n == 0) continue;
        u64 x = power_mod_encoded(ctx, ctx.encode(a), d);
        if (x == one || x == minus_one) continue;
        bool composite = true;
        for (int r = 1; r < s; ++r) {
            x = ctx.mul(x, x);
            if (x == minus_one) { composite = false; break; }
        }
        if (composite) return false;
    }
    return true;
}

// Requiere n compuesto. Bloque 256 y reinicios por intento.
u64 pollard(u64 n) {
    if (n % 2 == 0) return 2;
    if (n % 3 == 0) return 3;
    static mt19937_64 rng(chrono::steady_clock::now()
                          .time_since_epoch().count());
    constexpr u64 BLOCK = 256, LIMIT = 1ULL << 18;
    Montgomery64 ctx(n);
    while (true) {
        u64 c = ctx.encode(uniform_int_distribution<u64>(1, n - 1)(
     rng));
        u64 y = ctx.encode(uniform_int_distribution<u64>(0, n - 1)(
     rng));
        u64 x = 0, saved = y, divisor = 1;
        u64 length = 1, batch = 0;
        auto advance = [&](u64 value) {
            return ctx.add(ctx.mul(value, value), c);
        };
        while (divisor == 1 && length <= LIMIT) {
            x = y;
            for (u64 i = 0; i < length; ++i) y = advance(y);
            for (u64 offset = 0; offset < length && divisor == 1;
                 offset += batch) {
                saved = y;
                batch = min(BLOCK, length - offset);
                u64 product = ctx.one();
                for (u64 i = 0; i < batch; ++i) {
                    y = advance(y);
                    u64 difference = x > y ? x - y : y - x;
                    product = ctx.mul(product, difference);
                }
                divisor = gcd(product, n);
            }
            length <<= 1;
        }
        if (divisor == n) {
            divisor = 1;
            for (u64 i = 0; i < batch && divisor == 1; ++i) {
                saved = advance(saved);
                u64 difference = x > saved ? x - saved : saved - x;
                divisor = gcd(difference, n);
            }
        }
        if (divisor > 1 && divisor < n) return divisor;
    }
}

// sqrtl solo aproxima; las correcciones enteras hacen exacta la raiz
    .
u64 isqrt(u64 n) {
    u64 r = (u64)sqrtl((long double)n);
    while ((u128)r * r > n) --r;
    while ((u128)(r + 1) * (r + 1) <= n) ++r;
    return r;
}

void collect(u64 n, vector<u64>& out) {
    if (n == 1) return;
    u64 r = isqrt(n);
    if ((u128)r * r == n) {
        size_t begin = out.size();
        collect(r, out);
        size_t end = out.size();
        for (size_t i = begin; i < end; ++i) {
            u64 p = out[i];
            out.push_back(p);
        }
        return;
    }
    if (is_prime(n)) { out.push_back(n); return; }
    u64 d = pollard(n);
    collect(d, out);
    collect(n / d, out);
}

vector<u64> factorize(u64 n) {
    assert(n >= 1); // 0 no tiene factorizacion prima finita.
    vector<u64> out;
    out.reserve(64);
    // Quitar factores pequenos una sola vez, antes de la recursion.
    for (u64 p : SMALL_PRIMES64) {
        while (n % p == 0) { out.push_back(p); n /= p; }
        if (n == 1) break;
    }
    collect(n, out);
    sort(out.begin(), out.end());
    return out;
}
} // namespace rho_optimized
```

Detalles importantes. La inversa usa overflow unsigned intencional; los productos usan `u128`. No mezclar residuos normales y codificados; la identidad codificada es `ctx.one()`. Los MCD también sirven sobre residuos codificados porque $\gcd(2^{64}a, n) = \gcd(a, n)$ para n impar. La raíz usa `sqrtl` solo como aproximación y se corrige exactamente. Los cuadrados se factorizan por la raíz una vez y se duplican sus factores. El bloque 256 mejoró los tiempos locales para factores grandes, pero no es un óptimo universal. La versión corta puede pasar el mismo problema y ser preferible por simplicidad. En *Fuzzy Factorization*, conservar diez dígitos deja un número menor que 10^{10}: incluso división por tentativa alcanza. `assert` detecta entradas inválidas durante pruebas y se elimina con `-DNDEBUG`; el llamador debe respetar $n \geq 1$.

6.13 Exponenciación de matrices

Cuándo usar: Acelerar una recurrencia lineal o transiciones repetidas un número enorme de veces, expresando un paso como multiplicación por una matriz.

```
using Matrix = vector<vector<ll>>;

Matrix matrix_multiply(const Matrix &a, const Matrix &b, ll mod) {
    assert(!a.empty() && !b.empty() && !b[0].empty());
    assert(!a[0].empty() && a[0].size() == b.size() && mod > 0);
    int rows = a.size(), inner = b.size(), columns = b[0].size();
    Matrix result(rows, vector<ll>(columns, 0));
    for (int i = 0; i < rows; ++i)
        for (int k = 0; k < inner; ++k)
            for (int j = 0; j < columns; ++j)
                result[i][j] = (result[i][j] +
                    (i128)a[i][k] * b[k][j]) % mod;
    return result;
}

Matrix matrix_power(Matrix base, unsigned long long exponent, ll mod)
    {
    int n = base.size();
    assert(n > 0 && (int)base[0].size() == n && mod > 0);
    Matrix result(n, vector<ll>(n, 0));
    for (int i = 0; i < n; ++i) result[i][i] = 1 % mod;
    while (exponent) {
        if (exponent & 1ULL) result = matrix_multiply(result, base,
     mod);
        base = matrix_multiply(base, base, mod);
        exponent >>= 1;
    }
    return result;
}
```

Complejidad: $O(n^3 \log e)$ tiempo; $O(n^2)$ memoria.

6.14 Eliminación gaussiana y determinante

Cuándo usar: Resolver sistemas lineales, calcular rango o determinante; elegir la versión real con pivoteo o modular según el dominio del problema.

Usar para resolver sistemas lineales, calcular rango o determinante. Esta versión trabaja con reales y pivoteo parcial; para módulo primo reemplazar la división por inverso modular.

Complejidad: tiempo $O(nm \min(n, m))$ para n ecuaciones y m variables; memoria $O(nm)$. Determinante cuadrado: $O(n^3)$.

```
int gaussian_elimination(vector<vector<long double>> matrix,
                         vector<long double> &solution) {
    const long double epsilon = 1e-12L;
    int rows = (int)matrix.size();
    int columns = (int)matrix[0].size() - 1;
    vector<int> pivot_row(columns, -1);
    int row = 0;
    for (int column = 0; column < columns && row < rows; ++column) {
        int selected = row;
        for (int candidate = row; candidate < rows; ++candidate)
            if (fabsl(matrix[candidate][column]) > fabsl(matrix[
     selected][column]))
                selected = candidate;
        if (fabsl(matrix[selected][column]) < epsilon) continue;
        swap(matrix[selected], matrix[row]);
        pivot_row[column] = row;
        for (int candidate = 0; candidate < rows; ++candidate) if (
     candidate != row) {
            long double factor = matrix[candidate][column] / matrix[
     row][column];
            for (int index = column; index <= columns; ++index)
                matrix[candidate][index] -= factor * matrix[row][
     index];
        }
        ++row;
    }
    for (int candidate = 0; candidate < rows; ++candidate) {
        bool all_zero = true;
        for (int column = 0; column < columns; ++column)
            all_zero &= fabsl(matrix[candidate][column]) < epsilon;
        if (all_zero && fabsl(matrix[candidate][columns]) >= epsilon)
```

```
      return 0;
    }
    solution.assign(columns, 0);
    for (int column = 0; column < columns; ++column)
        if (pivot_row[column] != -1)
            solution[column] = matrix[pivot_row[column]][columns]
                             / matrix[pivot_row[column]][column];
    return row == columns ? 1 : 2; // unica / infinitas
}
// matriz cuadrada es invertible:
long double determinant(vector<vector<long double>> matrix) {
    const long double epsilon = 1e-12L;
    int size = (int)matrix.size();
    long double answer = 1;
    for (int column = 0; column < size; ++column) {
        int pivot = column;
        for (int row = column; row < size; ++row)
            if (fabsl(matrix[row][column]) > fabsl(matrix[pivot][
     column])) pivot = row;
        if (fabsl(matrix[pivot][column]) < epsilon) return 0;
        if (pivot != column) swap(matrix[pivot], matrix[column]),
     answer = -answer;
        answer *= matrix[column][column];
        for (int row = column + 1; row < size; ++row) {
            long double factor = matrix[row][column] / matrix[column
     ][column];
            for (int index = column + 1; index < size; ++index)
                matrix[row][index] -= factor * matrix[column][index];
        }
    }
    return answer;
}
```

```
vector<vector<long double>> augmented={
    {1,1,3},
    {2,-1,0}
};
vector<long double> solution;
int type=gaussian_elimination(augmented,solution);

if(type==0) cout<<"Sin solucion\n";
else if(type==1)
    cout<<solution[0]<<" "<<solution[1]<<'\n'; // 1 2
else
    cout<<"Infinitas soluciones\n";
vector<vector<long double>> coefficients={
    {1,1},
    {2,-1}
};
cout<<(double)determinant(coefficients)<<'\n'; // -3
```

6.15 Base XOR de enteros de 64 bits

Cuándo usar: Consultar si un valor es XOR de un subconjunto o maximizar ese XOR. Es eliminación gaussiana sobre $GF(2)$, sin divisiones.

Complejidad: Insertar, consultar pertenencia y maximizar: $O(B)$ con $B = 64$; memoria $O(B)$. El rango indica cuántos vectores son independientes.

```
struct XorBasis {
    array<uint64_t, 64> basis{};
    int rank = 0;
    bool insert(uint64_t value) {
        for (int bit = 63; bit >= 0; --bit) if (value >> bit & 1) {
            if (basis[bit]) value ^= basis[bit];
            else { basis[bit] = value; ++rank; return true; }
        }
        return false;
    }
    bool contains(uint64_t value) const {
        for (int bit = 63; bit >= 0; --bit)
            if (value >> bit & 1) value ^= basis[bit];
        return value == 0;
    }
    uint64_t maximum(uint64_t initial = 0) const {
        for (int bit = 63; bit >= 0; --bit)
            initial = max(initial, initial ^ basis[bit]);
        return initial;
    }
};
void xor_basis_example() {
    XorBasis basis;
    basis.insert(3); basis.insert(5); basis.insert(6);
    cout << basis.rank << ' ' << basis.maximum() << ' ' << basis.
     contains(7)
         << '\n'; // 2 6 0
}
```

6.16 Dependencia XOR de bitsets con reconstrucción

Cuándo usar: Los vectores tienen más de 64 bits y se necesita recuperar un subconjunto no vacío con XOR cero, como Brazilian FootXOR.

Complejidad: Con n vectores de B bits y capacidad $N \geq n$, tiempo $O(nB\lceil(B+N)/w\rceil)$ y memoria $O(B(B+N))$ bits, más la entrada. Las operaciones de bitset procesan palabras de w bits.

```
template<size_t B, size_t N>
optional<vector<int>> xor_dependency(const vector<bitset<B>> &input)
     {
    assert(input.size() <= N);
    vector<bitset<B>> basis(B);
    vector<bitset<N>> combination(B);
    vector<char> used(B);
    for (int id = 0; id < (int)input.size(); ++id) {
        auto value = input[id];
        bitset<N> chosen; chosen[id] = true;
        for (int bit = (int)B - 1; bit >= 0; --bit) if (value[bit]) {
            if (used[bit]) {
                value ^= basis[bit]; chosen ^= combination[bit];
            } else {
                used[bit] = true; basis[bit] = value; combination[bit
     ] = chosen;
                break;
            }
        }
        if (value.none()) {
            vector<int> indices;
            for (int i = 0; i <= id; ++i) if (chosen[i]) indices.
     push_back(i);
            return indices;
        }
    }
    return nullopt;
}
void xor_dependency_example() {
    vector<bitset<4>> input = {bitset<4>(3), bitset<4>(5), bitset
     <4>(6)};
    auto chosen = xor_dependency<4, 3>(input);
    if (chosen) for (int i : *chosen) cout << i << ' '; // 0 1 2
}
```

Para exigir cantidad par de vectores, agregar a cada entrada un bit extra en 1. Toda dependencia resultante tendrá cardinalidad par; dividir sus `indices` en dos mitades da equipos del mismo tamaño y XOR igual. En Brazilian FootXOR (Latin America Championship 2025), $n, k \leq 1500$: usar `xor_dependency<1501,1500>` con el bit 1500 de paridad. El siguiente adaptador lee un caso y escribe los dos equipos:

```
void brazilian_footxor() {
    int n, k; cin >> n >> k;
    assert(1 <= n && n <= 1500 && 1 <= k && k <= 1500);
    vector<bitset<1501>> players(n);
    for (auto &player : players) {
        string s; cin >> s;
        for (int j = 0; j < k; ++j) player[j] = s[j] == '1';
        player[1500] = true; // Paridad de la cantidad de integrantes
         .
    }
    auto chosen = xor_dependency<1501,1500>(players);
    if (!chosen) { cout << "*\n"; return; }
    string teams(n, '0');
    for (int i = 0; i < (int)chosen->size(); ++i)
        teams[(*chosen)[i]] = i < (int)chosen->size() / 2 ? '1' : '2'
     ;
    cout << teams << '\n';
}
```

Cuidado: Una base escalar de 64 bits no sustituye esta variante para cadenas de 1500 bits. `nullopt` significa independencia, no un subconjunto vacío.

6.17 NTT y convolución

Cuándo usar: Multiplicar polinomios o contar combinaciones mediante convolución exacta bajo un primo compatible, evitando errores de punto flotante.

Convolución exacta módulo $998244353 = 119 \cdot 2^{23} + 1$, cuya raíz primitiva es 3. La longitud transformada no puede superar 2^{23}.

Las raíces de cada etapa se precalculan una sola vez y se reutilizan en llamadas posteriores. Para invertir se revierten los índices $1, \ldots, n-1$, se aplica la misma transformada y se divide por n módulo `NTT_MOD`. Se conservan `vector<int>`, la raíz primitiva 3 y la interfaz `number_theoretic_transform(values,inverse)`.

Complejidad: Para una longitud transformada n: $O(n \log n)$ tiempo; $O(n)$ memoria de trabajo y $O(N_{\max})$ memoria persistente para raíces, donde $N_{\max}$ es la mayor longitud transformada hasta el momento.

Cuidado: La caché conserva la memoria de la mayor transformada, incluso si las siguientes son pequeñas. En jueces con `int` de 4 bytes, las raíces ocupan aproximadamente 2 MiB para 2^{19} elementos y 32 MiB para 2^{23}, además de los arreglos de entrada y trabajo.

```
const int NTT_MOD = 998244353;
const int NTT_ROOT = 3;

void number_theoretic_transform(vector<int> &values, bool inverse) {
    int n = values.size();
    assert(n > 0 && (n & (n - 1)) == 0 && n <= (1 << 23));
    static vector<int> roots(2, 1);
    for (int half = roots.size() / 2; half < n / 2; half *= 2) {
        int root = mod_power(NTT_ROOT, (NTT_MOD - 1) / (4 * half),
     NTT_MOD);
        roots.resize(4 * half);
        for (int i = 2 * half; i < 4 * half; ++i)
            roots[i] = (ll)roots[i / 2] * (i & 1 ? root : 1) %
     NTT_MOD;
```

```
    }
    if (inverse) reverse(values.begin() + 1, values.end());
    for (int i = 1, j = 0; i < n; ++i) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(values[i], values[j]);
    }
    for (int half = 1; half < n; half *= 2)
        for (int start = 0; start < n; start += 2 * half)
            for (int offset = 0; offset < half; ++offset) {
                int even = values[start + offset];
                int odd = (ll)roots[half + offset] * values[start +
     half + offset] % NTT_MOD;
                values[start + offset] = even + odd;
                if (values[start + offset] >= NTT_MOD) values[start +
      offset] -= NTT_MOD;
                values[start + half + offset] = even - odd;
                if (values[start + half + offset] < 0) values[start +
     half + offset] += NTT_MOD;
            }
    if (inverse) {
        int inverse_n = mod_power(n, NTT_MOD - 2, NTT_MOD);
        for (int &value : values) value = (ll)value * inverse_n %
     NTT_MOD;
    }
}

vector<int> convolution(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int result_size = a.size() + b.size() - 1;
    int size = 1;
    while (size < result_size) size <<= 1;
    a.resize(size); b.resize(size);
    number_theoretic_transform(a, false);
    number_theoretic_transform(b, false);
    for (int i = 0; i < size; ++i) a[i] = (ll)a[i] * b[i] % NTT_MOD;
    number_theoretic_transform(a, true);
    a.resize(result_size);
    return a;
}
```

Normalizar coeficientes al intervalo $[0, NTT_MOD)$ antes de llamar a `convolution`. Para enteros grandes sin módulo, usar varias NTT y CRT.

```
vector<int> a={1,2}; // 1+2x
vector<int> b={3,4}; // 3+4x
vector<int> c=convolution(a,b);
for(int value:c) cout<<value<<' '; // 3 10 8
```

6.17.1 Ejemplo de aplicación: sumas de pares

Se tienen dos listas A y B. Para cada valor s, contar cuántos pares (a, b) cumplen $a + b = s$.

Construir arreglos de frecuencias:

$$freqA[x] = \#\{a \in A : a = x\}, \qquad freqB[y] = \#\{b \in B : b = y\}.$$

Entonces

$$ways[s] = \sum_{x+y=s} freqA[x] freqB[y],$$

que es exactamente una convolución.

```
vector<int> A={1,2,2},B={0,2};
int maximum=2;
vector<int> frequency_a(maximum+1),frequency_b(maximum+1);
for(int value:A) ++frequency_a[value];
for(int value:B) ++frequency_b[value];

vector<int> ways=convolution(frequency_a,frequency_b);
for(int sum=0;sum<(int)ways.size();++sum)
    cout<<sum<<": "<<ways[sum]<<'\n';
// 0:0, 1:1, 2:2, 3:1, 4:2
```

7. Geometría

7.1 Geometría 2D entera

Cuándo usar: Orientación, colinealidad, intersección de segmentos y distancias al cuadrado con coordenadas enteras y decisiones exactas.

Complejidad: cada operación $O(1)$ tiempo y memoria.

Para coordenadas enteras, usar producto cruzado en `__int128`.

```
struct Point {
    ll x, y;
    Point operator+(Point other) const { return {x + other.x, y +
     other.y}; }
    Point operator-(Point other) const { return {x - other.x, y -
     other.y}; }
    bool operator<(Point other) const { return tie(x, y) < tie(other.
     x, other.y); }
    bool operator==(Point other) const { return x == other.x && y ==
     other.y; }
};

i128 cross(Point a, Point b) { return (i128)a.x * b.y - (i128)a.y * b
    .x; }
i128 cross(Point a, Point b, Point c) { return cross(b - a, c - a); }
i128 dot(Point a, Point b) { return (i128)a.x * b.x + (i128)a.y * b.y
    ; }
i128 squared_distance(Point a, Point b) { return dot(b - a, b - a); }

int orientation(Point a, Point b, Point c) {
    i128 value = cross(a, b, c);
    return (value > 0) - (value < 0); // 1 CCW, -1 CW, 0 collinear
}

bool on_segment(Point a, Point b, Point p) {
    return cross(a, b, p) == 0 && min(a.x, b.x) <= p.x && p.x <= max(
     a.x, b.x)
       && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

bool segments_intersect(Point a, Point b, Point c, Point d) {
    int ab_c = orientation(a, b, c), ab_d = orientation(a, b, d);
    int cd_a = orientation(c, d, a), cd_b = orientation(c, d, b);
    if (ab_c == 0 && on_segment(a, b, c)) return true;
    if (ab_d == 0 && on_segment(a, b, d)) return true;
    if (cd_a == 0 && on_segment(c, d, a)) return true;
    if (cd_b == 0 && on_segment(c, d, b)) return true;
    return ab_c != ab_d && cd_a != cd_b;
}
```

```
Point a{0,0},b{4,0},c{2,2},d{2,-2},p{2,0};
cout<<orientation(a,b,c)<<'\n';       // 1: c esta a la izquierda de
    a->b
cout<<on_segment(a,b,p)<<'\n';        // 1: p pertenece al segmento [
    a,b]
cout<<segments_intersect(a,b,c,d)<<'\n'; // 1: ambos segmentos se
    cruzan
cout<<(ll)squared_distance(a,c)<<'\n';   // 8: (2-0)^2+(2-0)^2
// cross(u,v): signo del giro/area; dot(u,v)=0 si son perpendiculares
    .
```

7.2 Área y convex hull

Cuándo usar: Calcular àrea firmada o obtener el menor polígono convexo que contiene un conjunto de puntos antes de aplicar calipers u otras consultas convexas.

```
i128 polygon_double_area(const vector<Point> &polygon) {
    if (polygon.empty()) return 0;
    i128 area = 0;
    int n = polygon.size();
    for (int i = 0; i < n; ++i)
        area += cross(polygon[i], polygon[(i + 1) % n]);
    return area >= 0 ? area : -area;
}

vector<Point> convex_hull(vector<Point> points) {
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()), points.end());
    if (points.size() <= 1) return points;
    vector<Point> hull;
    for (Point p : points) {
        while (hull.size() >= 2 &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    size_t lower_size = hull.size();
    for (int i = (int)points.size() - 2; i >= 0; --i) {
        Point p = points[i];
        while (hull.size() > lower_size &&
               cross(hull[hull.size() - 2], hull.back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }
    hull.pop_back();
    return hull; // sin puntos collineales intermedios
}
```

```
vector<Point> points={{0,0},{2,0},{2,2},{0,2},{1,1}};
vector<Point> hull=convex_hull(points);
// hull contiene las 4 esquinas en orden antihorario; descarta el
    punto interior.
i128 double_area=polygon_double_area(hull);
cout<<hull.size()<<' '<<(ll)double_area<<'\n'; // 4 8; area real =
    8/2 = 4
```

Complejidad: convex hull $O(n \log n)$ tiempo y $O(n)$ memoria; área $O(n)$ tiempo y $O(1)$ memoria adicional.

Las conversiones, normalización de ángulos, `atan2` y funciones trigonométricas están en "Matemática cotidiana en C++".

7.3 Rectas con punto flotante

Cuándo usar: Proyección, distancia e intersección de rectas cuando el resultado no necesita ser entero y se controla el error con epsilon.

Complejidad: cada operación de rectas $O(1)$ tiempo y memoria.

```
const long double GEOMETRY_EPS = 1e-12L;

struct DPoint {
    long double x, y;
    DPoint operator+(DPoint other) const { return {x + other.x, y +
     other.y}; }
    DPoint operator-(DPoint other) const { return {x - other.x, y -
     other.y}; }
    DPoint operator*(long double scale) const { return {x * scale, y
     * scale}; }
    DPoint operator/(long double scale) const { return {x / scale, y
```

```
    / scale}; }
};

long double dot(DPoint a, DPoint b) { return a.x * b.x + a.y * b.y; }
long double cross(DPoint a, DPoint b) { return a.x * b.y - a.y * b.x;
    }
long double norm(DPoint a) { return sqrtl(dot(a, a)); }

optional<DPoint> line_intersection(DPoint a, DPoint b,
                                   DPoint c, DPoint d) {
    DPoint first = b - a, second = d - c;
    long double denominator = cross(first, second);
    if (fabsl(denominator) <= GEOMETRY_EPS) return nullopt;
    long double parameter = cross(c - a, second) / denominator;
    return a + first * parameter;
}

DPoint projection_on_line(DPoint point, DPoint a, DPoint b) {
    DPoint direction = b - a;
    assert(norm(direction) > GEOMETRY_EPS);
    return a + direction * (dot(point - a, direction) / dot(direction
     , direction));
}

long double distance_to_line(DPoint point, DPoint a, DPoint b) {
    DPoint direction = b - a;
    assert(norm(direction) > GEOMETRY_EPS);
    return fabsl(cross(direction, point - a)) / norm(direction);
}
```

```
DPoint a{0,0},b{4,0},c{2,-2},d{2,2},p{3,3};
auto intersection=line_intersection(a,b,c,d);
cout<<(double)intersection->x<<' '<<(double)intersection->y<<'\n'; //
     2 0
DPoint projected=projection_on_line(p,a,b);
cout<<(double)projected.x<<' '<<(double)projected.y<<'\n'; // 3 0
cout<<(double)distance_to_line(p,a,b)<<'\n'; // 3
// nullopt en line_intersection significa rectas paralelas o
     coincidentes.
```

7.4 Círculos

Cuándo usar: Intersecciones recta–círculo o círculo–círculo, tangencias y distancias asociadas a discos.

Complejidad: cada operación de círculos $O(1)$ tiempo; memoria $O(1)$ más la salida.

```
struct Circle { DPoint center; long double radius; };

// nullopt: circulos coincidentes (infinitas intersecciones).
optional<vector<DPoint>> circle_intersections(Circle first, Circle
     second) {
    DPoint difference = second.center - first.center;
    long double distance = norm(difference);
    if (distance <= GEOMETRY_EPS &&
        fabsl(first.radius - second.radius) <= GEOMETRY_EPS) return
     nullopt;
    if (distance > first.radius + second.radius + GEOMETRY_EPS ||
        distance < fabsl(first.radius - second.radius) - GEOMETRY_EPS
     )
        return vector<DPoint>{};

    long double along = (first.radius * first.radius
                         - second.radius * second.radius
                         + distance * distance) / (2 * distance);
    long double height_squared = first.radius * first.radius - along
     * along;
    if (height_squared < 0 && height_squared > -GEOMETRY_EPS)
     height_squared = 0;
    if (height_squared < 0) return vector<DPoint>{};

    DPoint unit = difference / distance;
    DPoint base = first.center + unit * along;
    long double height = sqrtl(height_squared);
    DPoint perpendicular{-unit.y, unit.x};
    if (height <= GEOMETRY_EPS) return vector<DPoint>{base};
    return vector<DPoint>{base + perpendicular * height,
                          base - perpendicular * height};
}
```

```
Circle first{{0,0},2},second{{2,0},2};
auto intersections=circle_intersections(first,second);
if(!intersections) cout<<"Coincidentes\n"; // Infinitos puntos
     comunes.
else cout<<intersections->size()<<'\n';     // 2: (1,+sqrt(3)), (1,-
     sqrt(3))
// size 0: separados/contenidos; size 1: tangentes; size 2: se cortan
     .
```

7.5 Punto en polígono

Cuándo usar: Decidir si un punto está dentro, fuera o sobre el borde de un polígono simple, incluso no convexo.

Complejidad: $O(n)$ tiempo y $O(1)$ memoria adicional.

Devuelve 0 afuera, 1 adentro y 2 sobre el borde. Funciona para polígonos simples en cualquier orientación.

```
bool on_segment(DPoint a, DPoint b, DPoint point) {
    if (fabsl(cross(b - a, point - a)) > GEOMETRY_EPS) return false;
    return dot(point - a, point - b) <= GEOMETRY_EPS;
}

int point_in_polygon(const vector<DPoint> &polygon, DPoint point) {
    bool inside = false;
    int n = polygon.size();
    for (int i = 0; i < n; ++i) {
        DPoint a = polygon[i], b = polygon[(i + 1) % n];
        if (on_segment(a, b, point)) return 2;
        bool crosses = (a.y > point.y) != (b.y > point.y);
        if (crosses) {
            long double intersection_x = a.x
                + (b.x - a.x) * (point.y - a.y) / (b.y - a.y);
            if (point.x < intersection_x) inside = !inside;
        }
    }
    return inside ? 1 : 0;
}
```

```
vector<DPoint> polygon={{0,0},{4,0},{4,4},{0,4}};
cout<<point_in_polygon(polygon,{2,2})<<'\n'; // 1: adentro
cout<<point_in_polygon(polygon,{0,2})<<'\n'; // 2: sobre el borde
cout<<point_in_polygon(polygon,{5,2})<<'\n'; // 0: afuera
```

7.6 Rotating calipers: diámetro convexo

Cuándo usar: Obtener el par de puntos más alejados de un polígono convexo; aplicar primero convex hull si la entrada es un conjunto desordenado.

Complejidad: $O(n)$ tiempo y $O(1)$ memoria adicional.

El polígono debe ser convexo, sin puntos repetidos y con sus vértices en orden sobre el borde, horario o antihorario. Se detecta el sentido una sola vez y se elige la comparación correspondiente, sin copiar ni invertir el polígono. Eliminar vértices colineales intermedios con `convex_hull` antes de usar calipers. Si todos son colineales, se toman los extremos.

```
template<bool Clockwise>
i128 convex_diameter_oriented(const vector<Point> &polygon) {
    int n = polygon.size();
    i128 answer = 0;
    int opposite = 1;
    for (int i = 0; i < n; ++i) {
        int next = (i + 1) % n;
        while (true) {
            int candidate = (opposite + 1) % n;
            i128 current_area = cross(polygon[next] - polygon[i],
                                      polygon[opposite] - polygon[i])
     ;
            i128 next_area = cross(polygon[next] - polygon[i],
                                   polygon[candidate] - polygon[i]);
            if constexpr (Clockwise) {
                if (next_area >= current_area) break;
            } else {
                if (next_area <= current_area) break;
            }
            opposite = candidate;
        }
        answer = max(answer, squared_distance(polygon[i], polygon[
     opposite]));
        answer = max(answer, squared_distance(polygon[next], polygon[
     opposite]));
    }
    return answer;
}

i128 convex_diameter_squared(const vector<Point> &polygon) {
    int n = polygon.size();
    if (n <= 1) return 0;
    if (n == 2) return squared_distance(polygon[0], polygon[1]);
    for (int i = 0; i < n; ++i) {
        i128 turn = cross(polygon[i], polygon[(i + 1) % n],
                          polygon[(i + 2) % n]);
        if (turn > 0) return convex_diameter_oriented<false>(polygon)
     ;
        if (turn < 0) return convex_diameter_oriented<true>(polygon);
    }
    // Todos colineales: los extremos lexicograficos son los mas
     lejanos.
    auto [first, last] = minmax_element(polygon.begin(), polygon.end
     ());
    return squared_distance(*first, *last);
}
```

```
vector<Point> points={{0,0},{2,0},{2,2},{0,2},{1,1}};
vector<Point> hull=convex_hull(points);
i128 diameter_squared=convex_diameter_squared(hull);
cout<<(ll)diameter_squared<<'\n'; // 8: distancia entre esquinas
     opuestas
reverse(hull.begin(),hull.end()); // El mismo poligono en sentido
     horario.
cout<<(ll)convex_diameter_squared(hull)<<'\n'; // Tambien 8
// La distancia real es sqrt(8); se conserva al cuadrado para evitar
     decimales.
```

7.7 Geometría 3D y planos

Cuándo usar: Problemas espaciales con normales de caras, lado de un plano, proyección o intersección de una recta con un plano.

Usar para normales de caras, orientación respecto de un plano, proyecciones y distancias en el espacio. Un plano por A, B, C tiene normal

$n = (B - A) \times (C - A)$ y ecuación $n \cdot (X - A) = 0$.
Complejidad: cada operación $O(1)$ tiempo y memoria.

```
struct Point3D {
    long double x = 0, y = 0, z = 0;
    Point3D operator+(Point3D other) const { return {x + other.x, y +
      other.y, z + other.z}; }
    Point3D operator-(Point3D other) const { return {x - other.x, y -
      other.y, z - other.z}; }
    Point3D operator*(long double scalar) const { return {x * scalar,
      y * scalar, z * scalar}; }
    Point3D operator/(long double scalar) const { return {x / scalar,
      y / scalar, z / scalar}; }
};

long double dot3d(Point3D a, Point3D b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
Point3D cross3d(Point3D a, Point3D b) {
    return {a.y * b.z - a.z * b.y,
            a.z * b.x - a.x * b.z,
            a.x * b.y - a.y * b.x};
}
long double norm3d(Point3D vector) { return sqrtl(dot3d(vector,
     vector)); }
long double signed_plane_value(Point3D point, Point3D plane_point,
     Point3D normal) {
    return dot3d(point - plane_point, normal);
}
Point3D project_on_plane(Point3D point, Point3D plane_point, Point3D
     normal) {
    long double squared_norm = dot3d(normal, normal);
    assert(squared_norm > 1e-24L);
    return point - normal * (signed_plane_value(point, plane_point,
     normal) / squared_norm);
}
long double distance_to_plane(Point3D point, Point3D plane_point,
     Point3D normal) {
    long double normal_length = norm3d(normal);
    assert(normal_length > 1e-12L);
    return fabsl(signed_plane_value(point, plane_point, normal)) /
     normal_length;
}
optional<Point3D> line_plane_intersection(Point3D point, Point3D
     direction,
                                          Point3D plane_point,
     Point3D normal) {
    assert(norm3d(direction) > 1e-12L && norm3d(normal) > 1e-12L);
    long double denominator = dot3d(direction, normal);
    if (fabsl(denominator) < 1e-12L) return nullopt;
    long double parameter = dot3d(plane_point - point, normal) /
     denominator;
    return point + direction * parameter;
}
```

```
Point3D a{0,0,0},b{1,0,0},c{0,1,0};
Point3D normal=cross3d(b-a,c-a); // (0,0,1): plano z=0
Point3D point{2,3,5};
cout<<(double)signed_plane_value(point,a,normal)<<'\n'; // 5: lado
     positivo
Point3D projected=project_on_plane(point,a,normal);
cout<<(double)projected.x<<' '<<(double)projected.y<<' '
    <<(double)projected.z<<'\n'; // 2 3 0
cout<<(double)distance_to_plane(point,a,normal)<<'\n'; // 5
auto hit=line_plane_intersection({0,0,5},{0,0,-1},a,normal);
cout<<(double)hit->x<<' '<<(double)hit->y<<' '
    <<(double)hit->z<<'\n'; // 0 0 0
// nullopt: la recta es paralela al plano; puede estar fuera o
     contenida.
```

Cuidado: La orientación de la normal depende del orden A, B, C. Normalizar sólo si se necesita una distancia; para signos, evitar la raíz mejora estabilidad.

7.8 Camino mínimo rodeando un polígono convexo

Cuándo usar: El segmento directo está bloqueado por un ùnico obstáculo convexo y se permite recorrer su borde entre puntos tangentes. Usar cuando el segmento directo entre A y B atraviesa el interior de un obstáculo convexo. Si no lo atraviesa, la respuesta es $|AB|$. Si lo atraviesa, un camino òptimo toca dos vértices tangentes y recorre una de las dos cadenas del borde entre ellos.

Procedimiento:

1. hallar desde A y B los dos vértices tangentes al polígono;
2. precomputar longitudes prefijas del perímetro;
3. probar las combinaciones compatibles de tangencias y ambas cadenas;
4. minimizar $|A - u| + \text{borde}(u, v) + |v - B|$.

Complejidad: tangencias $O(\log n)$ con búsqueda binaria, o $O(n)$ en una implementación simple; memoria $O(n)$ para prefijos.

Cuidado: Si se permite tocar el borde, distinguir intersección con el interior de un simple contacto tangencial. Para varios obstáculos construir un grafo de visibilidad; esta receta aislada ya no alcanza.

8. Programación dinámica

8.1 Longest Increasing Subsequence

Cuándo usar: Encontrar la subsecuencia estrictamente creciente más larga; aparece también tras ordenar una dimensión en problemas bidimensionales.

Devuelve una subsecuencia estrictamente creciente y no solamente su longitud.

```
vector<ll> longest_increasing_subsequence(const vector<ll> &a) {
    int n = a.size();
    vector<ll> tails;
    vector<int> tail_index, parent(n, -1);

    for (int i = 0; i < n; ++i) {
        int position = lower_bound(tails.begin(), tails.end(), a[i])
                     - tails.begin();
        if (position == (int)tails.size()) {
            tails.push_back(a[i]);
            tail_index.push_back(i);
        } else {
            tails[position] = a[i];
            tail_index[position] = i;
        }
        if (position > 0) parent[i] = tail_index[position - 1];
    }

    vector<ll> answer;
    if (!tail_index.empty())
        for (int i = tail_index.back(); i != -1; i = parent[i])
            answer.push_back(a[i]);
    reverse(answer.begin(), answer.end());
    return answer;
}
```

Complejidad: $O(n \log n)$ tiempo y $O(n)$ memoria.

Para subsecuencia no decreciente, reemplazar `lower_bound` por `upper_bound`.

8.2 LCS con reconstrucción

Cuándo usar: Encontrar una subsecuencia común de longitud máxima entre dos cadenas. No confundir subsecuencia con substring contiguo.

Complejidad: Tiempo $O(nm)$; memoria $O(nm)$ para reconstruir. Si sólo importa la longitud, usar dos filas y $O(\min(n, m))$ memoria.

```
string longest_common_subsequence(const string &a, const string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            dp[i][j] = a[i - 1] == b[j - 1] ? dp[i - 1][j - 1] + 1
                                            : max(dp[i - 1][j], dp[i][j - 1]);
    string answer;
    for (int i = n, j = m; i && j; ) {
        if (a[i - 1] == b[j - 1]) {
            answer.push_back(a[--i]); --j;
        } else if (dp[i - 1][j] >= dp[i][j - 1]) --i;
        else --j;
    }
    reverse(answer.begin(), answer.end());
    return answer;
}
void lcs_example() {
    cout << longest_common_subsequence("abcde", "ace") << '\n'; //
      ace
}
```

Cuidado: Con $n = m = 5000$, los enteros de la tabla ocupan unos 95.4 MiB; con 10000, unos 381.5 MiB. Esta plantilla no cuenta todas las LCS ni resuelve por sí sola el problema constructivo Many LCS (SEERC 2021).

8.3 Distancia de edición con operaciones

Cuándo usar: Transformar una cadena en otra usando inserción, borrado y reemplazo, cada uno de costo uno. Devuelve costo y una secuencia òptima de operaciones.

Complejidad: Tiempo y memoria $O(nm)$. Sólo la distancia puede calcularse con dos filas; recuperar las operaciones requiere la tabla de esta versión.

Las posiciones de las operaciones son 0-indexadas sobre la cadena *tal como queda tras aplicar las operaciones anteriores.*

```
struct EditStep { char type; int position; char value; }; // I, D, R
pair<int, vector<EditStep>> edit_distance(const string &a, const
     string &b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; ++i) dp[i][m] = n - i;
    for (int j = 0; j <= m; ++j) dp[n][j] = m - j;
    for (int i = n - 1; i >= 0; --i)
        for (int j = m - 1; j >= 0; --j)
            dp[i][j] = a[i] == b[j] ? dp[i + 1][j + 1]
                : 1 + min({dp[i + 1][j], dp[i][j + 1], dp[i + 1][j +
     1]});
    vector<EditStep> steps;
    for (int i = 0, j = 0, position = 0; i < n || j < m; ) {
        if (i < n && j < m && a[i] == b[j]) {
            ++i; ++j; ++position;
        } else if (i < n && j < m && dp[i][j] == 1 + dp[i + 1][j +
```

```
    1]) {
            steps.push_back({'R', position++, b[j++]}); ++i;
        } else if (i < n && dp[i][j] == 1 + dp[i + 1][j]) {
            steps.push_back({'D', position, 0}); ++i;
        } else {
            steps.push_back({'I', position++, b[j++]});
        }
    }
    return {dp[0][0], steps};
}
void edit_distance_example() {
    auto [cost, steps] = edit_distance("kitten", "sitting");
    cout << cost << '\n'; // 3
    for (auto step : steps)
        cout << step.type << ' ' << step.position << ' ' << step.
     value << '\n';
    // R 0 s; R 4 i; I 6 g. En D, value no se utiliza.
}
```

8.4 Distancia al palíndromo más cercano

Cuándo usar: Ediciones de costo uno para convertir una cadena en algún palíndromo, como Palindromic Distance (Taiwan Online Programming Contest 2025).

Complejidad: Tiempo y memoria $O(n^2)$; reconstrucción $O(n)$. Con suma de longitudes ≤ 3000, hay a lo sumo $9 \cdot 10^6$ celdas entre todos los casos.

```
pair<int, string> nearest_palindrome(const string &s) {
    int n = s.size();
    if (n == 0) return {0, ""};
    vector<vector<int>> dp(n, vector<int>(n));
    auto cost = [&](int l, int r) { return l >= r ? 0 : dp[l][r]; };
    for (int l = n - 1; l >= 0; --l)
        for (int r = l + 1; r < n; ++r)
            dp[l][r] = s[l] == s[r] ? cost(l + 1, r - 1)
                : 1 + min({cost(l + 1, r), cost(l, r - 1), cost(l +
     1, r - 1)});
    string left, right;
    for (int l = 0, r = n - 1; l <= r; ) {
        if (l == r) { left.push_back(s[l]); break; }
        if (s[l] == s[r] || dp[l][r] == 1 + cost(l + 1, r - 1)) {
            left.push_back(s[l]); right.push_back(s[l]); ++l; --r;
        } else if (dp[l][r] == 1 + cost(l + 1, r)) ++l;
        else --r;
    }
    reverse(right.begin(), right.end());
    return {dp[0][n - 1], left + right};
}
void palindrome_edit_example() {
    auto [cost, palindrome] = nearest_palindrome("hello");
    cout << cost << '\n'; // 2; palindrome es una solucion valida.
}
```

Una inserción que empareja un extremo puede sustituirse por borrar ese extremo con igual costo. Por eso alcanza reconstruir con borrados y reemplazos. Los tres algoritmos trabajan con bytes de `string`, no con caracteres Unicode decodificados.

8.5 Knapsack 0/1

Cuándo usar: Elegir cada objeto como máximo una vez, con capacidad moderada, para maximizar valor o decidir qué sumas son alcanzables.

Cada objeto puede elegirse a lo sumo una vez; por eso la capacidad se recorre en orden descendente. Para recuperar sus *índices*, activar los comentarios: las decisiones usan $O(nW)$ bits adicionales y la recuperación cuesta $O(n)$. Admite peso cero; pesos no negativos y sumas de valores que entren en `long long`.

```
ll knapsack_01(const vector<int> &weight, const vector<ll> &value,
               int capacity
    // , vector<int>& chosen // Reconstruccion: indices elegidos.
) {
    assert(weight.size() == value.size() && capacity >= 0);
    assert(all_of(weight.begin(), weight.end(), [](int w) { return w
     >= 0; }));
    vector<ll> dp(capacity + 1, 0);
    // vector<vector<bool>> take(weight.size(), vector<bool>(capacity
       + 1, false));
    for (int item = 0; item < (int)weight.size(); ++item)
        for (int current = capacity; current >= weight[item]; --
     current) {
            // if (dp[current - weight[item]] + value[item] > dp[
     current]) take[item][current] = true;
            dp[current] = max(dp[current],
                              dp[current - weight[item]] + value[item
     ]);
        }
    // chosen.clear();
    // for (int item = (int)weight.size() - 1, current = capacity;
     item >= 0; --item)
    //     if (take[item][current]) { chosen.push_back(item); current
       -= weight[item]; }
    // reverse(chosen.begin(), chosen.end());
    return *max_element(dp.begin(), dp.end());
}
```

Complejidad: $O(nW)$ tiempo y $O(W)$ memoria.

Ejemplo opcional, tras activar los comentarios anteriores:

```
// void knapsack_recovery_example() {
//     vector<int> weight = {2,3,4}, chosen;
//     vector<ll> value = {4,5,7};
//     cout << knapsack_01(weight, value, 5, chosen) << '\n'; // 9
//     for (int item : chosen) cout << item << ' '; // 0 1
// }
```

8.6 Bitmask DP: camino hamiltoniano mínimo

Cuándo usar: El estado depende del subconjunto exacto de elementos ya usados y n es pequeño, normalmente $n \leq 20$.

Encuentra el costo mínimo de un camino que visita cada vértice exactamente una vez, con inicio y fin libres; no exige volver al inicio. La matriz cuadrada `cost[u][v]` representa un grafo dirigido: usar `LINF` si no existe la arista $u \to v$. Para un grafo no dirigido, cargar ambos sentidos. Admite costos negativos y devuelve `LINF` si no existe un camino completo; con un solo vértice devuelve cero. Devuelve el costo, no la secuencia de nodos. Se precalculan máscaras de aristas existentes para recorrer sólo los vecinos que aún no fueron visitados. La reconstrucción comentada reutiliza la DP: agrega $O(n)$ memoria para la secuencia y cuesta $O(n^2)$ al terminar. Si no hay camino, queda vacía.

```
ll hamiltonian_path(const vector<vector<ll>> &cost
    // , vector<int>& path // Reconstruccion: vertices en orden.
) {
    int n = cost.size();
    assert(1 <= n && n < 31);
    assert(all_of(cost.begin(), cost.end(), [n](const auto &row) {
        return (int)row.size() == n;
    }));
    int states = 1 << n;
    vector<int> outgoing(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v = 0; v < n; ++v)
            if (cost[u][v] != LINF) outgoing[u] |= 1 << v;
    vector<vector<ll>> dp(states, vector<ll>(n, LINF));
    for (int v = 0; v < n; ++v) dp[1 << v][v] = 0;
    for (int mask = 1; mask < states; ++mask)
        for (int last = 0; last < n; ++last) if (dp[mask][last] !=
     LINF)
            for (int nexts = outgoing[last] & ~mask; nexts; nexts &=
     nexts - 1) {
                int next = __builtin_ctz((unsigned)nexts); // nexts
     != 0
                dp[mask | (1 << next)][next] = min(
                    dp[mask | (1 << next)][next],
                    dp[mask][last] + cost[last][next]);
            }
    // path.clear();
    // int last = min_element(dp.back().begin(), dp.back().end()) -
     dp.back().begin();
    // if (dp.back()[last] != LINF) for (int mask = states - 1; ; ) {
    //     path.push_back(last);
    //     int previous_mask = mask ^ (1 << last);
    //     if (previous_mask == 0) break;
    //     int previous = -1;
    //     for (int v = 0; v < n; ++v)
    //         if ((previous_mask >> v & 1) && cost[v][last] != LINF
     &&
    //             dp[previous_mask][v] != LINF &&
    //             dp[previous_mask][v] + cost[v][last] == dp[mask][
     last]) {
    //             previous = v; break;
    //         }
    //     assert(previous != -1);
    //     last = previous; mask = previous_mask;
    // }
    // reverse(path.begin(), path.end());
    return *min_element(dp.back().begin(), dp.back().end());
}
```

Complejidad: $O(2^n n^2)$ tiempo y $O(2^n n)$ memoria; usar para n pequeño.

Cuidado: Requiere $1 \leq n < 31$ para la máscara, pero tiempo y memoria suelen limitarlo mucho antes: con $n = 20$, sólo los valores de la DP ocupan 160 MiB, más el costo de los vectores. Los costos finitos y todas las sumas parciales deben estar estrictamente entre $-$`LINF` y `LINF`. Una cota suficiente, para $n > 1$, es $(n-1)\max|w| <$ `LINF` sobre las aristas existentes. Nunca sumar la marca de arista inexistente.

```
// void hamiltonian_recovery_example() {
//     vector<vector<ll>> cost = {{0,4,5}, {LINF,0,-2}, {LINF,LINF
     ,0}};
//     vector<int> path;
//     ll answer = hamiltonian_path(cost, path);
//     if (answer == LINF) cout << "Sin camino\n";
//     else { cout << answer << '\n'; // 2
//         for (int v : path) cout << v << ' '; // 0 1 2
//     }
// }
```

8.7 Mochila ilimitada

Cuándo usar: Cada tipo de objeto puede repetirse, sus pesos son enteros positivos y se maximiza valor sin superar una capacidad W.

Complejidad: Tiempo $O(nW)$; memoria $O(W)$ más los objetos recuperados. La capacidad se recorre en orden ascendente para permitir

repeticiones.

```
pair<ll, vector<int>> unbounded_knapsack(const vector<int> &weight,
                                         const vector<ll> &value, int
     capacity) {
    assert(capacity >= 0 && weight.size() == value.size());
    vector<ll> dp(capacity + 1);
    vector<int> last(capacity + 1, -1);
    for (int i = 0; i < (int)weight.size(); ++i) {
        assert(weight[i] > 0);
        for (int c = weight[i]; c <= capacity; ++c)
            if (dp[c - weight[i]] + value[i] > dp[c]) {
                dp[c] = dp[c - weight[i]] + value[i]; last[c] = i;
            }
    }
    vector<int> chosen;
    for (int c = capacity; last[c] != -1; c -= weight[last[c]])
        chosen.push_back(last[c]);
    return {dp[capacity], chosen};
}
void unbounded_example() {
    auto [value, items] = unbounded_knapsack({3,5}, {4,7}, 10);
    cout << value << '\n'; // 14; items = {1,1}.
}
```

Cuidado: Peso cero y valor positivo darían un òptimo infinito. Los valores y sumas deben entrar en `long long`. Se permite elegir cero objetos.

8.8 Suma exacta 0/1 con mínima cantidad de objetos

Cuándo usar: Cada objeto se usa a lo sumo una vez y se necesita sumar exactamente W, minimizando la cantidad elegida. Sirve como componente de Nine Packs.

Complejidad: Tiempo $O(nW)$; memoria $O(W)$ enteros y $O(nW)$ bits de decisiones para reconstruir. Para sólo obtener el costo, omitir `take`.

```
optional<vector<int>> minimum_items_exact(const vector<int> &weight,
     int target,
                                          vector<int> *all_costs =
     nullptr) {
    assert(target >= 0);
    int n = weight.size(), infinity = n + 1;
    vector<int> dp(target + 1, infinity);
    vector<vector<bool>> take(n, vector<bool>(target + 1));
    dp[0] = 0;
    for (int i = 0; i < n; ++i) {
        assert(weight[i] >= 0);
        for (int s = target; s >= weight[i]; --s)
            if (dp[s - weight[i]] + 1 < dp[s]) {
                dp[s] = dp[s - weight[i]] + 1; take[i][s] = true;
            }
    }
    if (all_costs) *all_costs = dp;
    if (dp[target] == infinity) return nullopt;
    vector<int> chosen;
    for (int i = n - 1, s = target; i >= 0; --i)
        if (take[i][s]) { chosen.push_back(i); s -= weight[i]; }
    reverse(chosen.begin(), chosen.end());
    return chosen;
}
void exact_sum_example() {
    auto items = minimum_items_exact({8,10,12}, 20);
    if (items) for (int i : *items) cout << i << ' '; // 0 2
    // nullopt: imposible; vector vacio: target == 0.
}
```

Pasar `&costs` como tercer argumento permite consultar el mínimo para *todas* las sumas, con `weight.size()+1` como marca de imposible. En Nine Packs se calcula una tabla por lado y se busca la mejor suma positiva común; no llamar a la función de nuevo para cada suma. Copiar esa tabla agrega $O(W)$ tiempo y memoria, sin cambiar la cota total.

8.9 Subset sum con bitset

Cuándo usar: Sólo importa qué sumas son alcanzables con objetos 0/1 de peso no negativo. Procesar muchas sumas en paralelo reduce el costo de la DP booleana.

Complejidad: Para n objetos y S posiciones, $O(n\lceil S/w\rceil)$ operaciones de palabra, con w bits por palabra (usualmente 64); memoria $O(S)$ bits. La aceleración concreta depende de la implementación de bitset.

```
template<size_t S>
bitset<S> subset_sums(const vector<int> &weight) {
    static_assert(S > 0);
    bitset<S> reachable;
    reachable[0] = true;
    for (int w : weight) {
        assert(w >= 0);
        reachable |= reachable << (size_t)w;
    }
    return reachable;
}
void subset_sum_example() {
    auto reachable = subset_sums<21>({8,10,12}); // S debe ser >
      target.
    cout << reachable[20] << ' ' << reachable[19] << '\n'; // 1 0
}
```

Cuidado: Esta variante no devuelve los objetos ni la mínima cantidad. Una suma fuera de $[0, S)$ se descarta. Evitar bitsets locales gigantes si el juez tiene poco stack; se puede dar almacenamiento `static` a la estructura. Para muchos pesos repetidos, agrupar cantidades puede ser necesario para pasar.

8.10 Digit DP: contar enteros por sus dígitos

Cuándo usar: Contar números de un intervalo enorme cuya validez depende de los dígitos, no recorrer uno por uno todos los valores hasta 10^{18}.

Complejidad: Esta plantilla: $O(d\cdot 11\cdot 10)$ tiempo y $O(d\cdot 11)$ memoria para d dígitos. Otros estados, como un residuo módulo k, agregan su propio factor.

Ejemplo: contar enteros no negativos sin dos dígitos adyacentes iguales. `last=10` significa que aún sólo se leyeron ceros iniciales. El cero se cuenta como un número válido.

```
ll count_no_equal_neighbors(ll upper) {
    if (upper < 0) return 0;
    string digits = to_string(upper);
    ll memo[20][11];
    for (auto &row : memo) fill(begin(row), end(row), -1);
    function<ll(int,int,bool)> dfs = [&](int position, int last, bool
      tight) -> ll {
        if (position == (int)digits.size()) return 1;
        ll &cached = memo[position][last];
        if (!tight && cached != -1) return cached;
        ll answer = 0;
        int limit = tight ? digits[position] - '0' : 9;
        for (int digit = 0; digit <= limit; ++digit) {
            if (digit == last) continue;
            int next = last == 10 && digit == 0 ? 10 : digit;
            answer += dfs(position + 1, next, tight && digit == limit
     );
        }
        if (!tight) cached = answer;
        return answer;
    };
    return dfs(0, 10, true);
}
void digit_dp_example() {
    ll left = 0, right = 20;
    cout << count_no_equal_neighbors(right) -
     count_no_equal_neighbors(left - 1)
        << '\n'; // 20: solo 11 queda excluido.
}
```

Cuidado: Requiere $0 \le L \le R \le LLONG_MAX$ para la resta de prefijos del ejemplo. Los ceros iniciales no deben confundirse con un dígito real. Adaptar el estado y el caso base al problema: Score Values (World Finals 2025) necesita residuos y maximizar frecuencias; esta función no lo resuelve directamente.

8.11 Divide and Conquer DP Optimization

Cuándo usar: DP por particiones $dp[k][i] = \min_{j<i}(dp[k-1][j] + cost(j,i))$ cuando el punto òptimo es monótono en i.

Para transiciones

$$dp_{new}[m] = \min_{0\le k\le m}\{dp_{old}[k] + cost(k,m)\},$$

cuando el índice óptimo es monótono: $opt[m] \le opt[m+1]$.

```
template<class Cost>
void compute_divide_conquer(int left, int right,
                            int option_left, int option_right,
                            const vector<ll> &previous,
                            vector<ll> &current, Cost cost) {
    if (left > right) return;
    while (option_left <= option_right && previous[option_left] ==
     LINF)
        ++option_left;
    while (option_left <= option_right && previous[option_right] ==
     LINF)
        --option_right;
    if (option_left > option_right || right < option_left) {
        fill(current.begin() + left, current.begin() + right + 1,
     LINF);
        return;
    }
    int middle = left + (right - left) / 2;
    pair<ll,int> best{LINF, -1};
    int last_option = min(middle, option_right);
    for (int option = option_left; option <= last_option; ++option) {
        if (previous[option] == LINF) continue;
        ll transition = cost(option, middle);
        if (transition == LINF) continue;
        i128 raw = (i128)previous[option] + transition;
        ll candidate = raw > LINF ? LINF : raw < -LINF ? -LINF : (ll)
     raw;
        best = min(best, pair<ll,int>{candidate, option});
    }
    current[middle] = best.first;
    // Sin optimo finito no hay informacion para podar a los vecinos.
    int left_end = best.second == -1 ? option_right : best.second;
    int right_begin = best.second == -1 ? option_left : best.second;
    compute_divide_conquer(left, middle - 1, option_left, left_end,
```

```
                        previous, current, cost);
    compute_divide_conquer(middle + 1, right, right_begin,
     option_right,
                        previous, current, cost);
}
```

La monotonía debe demostrarse (por ejemplo con desigualdad cuadrangular o matriz Monge); no alcanza con que el costo "parezca convexo". `LINF` representa estados o transiciones imposibles; no se suma a un costo finito. Si no hay candidato finito, el estado queda en `LINF` y ambos lados conservan los límites de candidatos. Los costos y sumas finitos deben estar estrictamente entre $-$`LINF` y `LINF`.

Complejidad: $O(kn \log n)$ para k capas si evaluar `cost` cuesta $O(1)$ y cada estado procesado tiene candidato finito; memoria $O(n)$ con dos capas. Si hay estados imposibles por restricciones de `cost`, el fallback sin poda puede costar $O(kn^2)$; conviene acotar el dominio a los estados factibles cuando el problema lo permita.

8.11.1 Acotar los estados factibles

Para dividir los primeros i elementos en exactamente g segmentos no vacíos, los estados $i < g$ son imposibles. Si el último segmento es $[j, i)$, el corte cumple $g - 1 \le j < i$. Inicializar los demás estados en `LINF` y calcular sólo $i \in [g, n]$:

```
template<class SegmentCost>
ll partition_nonempty(int n, int groups, SegmentCost segment_cost) {
    assert(n >= 0);
    if (groups < 0 || groups > n) return LINF;
    vector<ll> previous(n + 1, LINF), current(n + 1, LINF);
    previous[0] = 0; // Cero elementos y cero grupos.
    for (int g = 1; g <= groups; ++g) {
        fill(current.begin(), current.end(), LINF);
        auto cost = [&](int j, int i) -> ll {
            if (j >= i) return LINF; // Segmento vacio o invertido.
            return segment_cost(j, i); // Costo de [j,i).
        };
        compute_divide_conquer(g, n, g - 1, n - 1,
                               previous, current, cost);
        previous.swap(current);
    }
    return previous[n];
}
```

Si todo segmento no vacío tiene costo finito, cada estado procesado tiene al menos un candidato finito: el prefijo de longitud $g - 1$ puede formar $g - 1$ grupos y dejar el resto como último segmento. Con óptimos monótonos y costo $O(1)$, se conserva $O(\texttt{groups} \cdot n \log n)$ tiempo y $O(n)$ memoria. No hace falta que todas las transiciones sean válidas.

Cuidado: Que existan exactamente g grupos no garantiza la monotonía del corte óptimo; hay que demostrarla para el costo del problema. Si hay otras restricciones sobre los segmentos, pueden quedar estados imposibles dentro de $[g, n]$ y esta llamada por sí sola no elimina el fallback cuadrático.

Si se conocen estados factibles con huecos, se puede adaptar la recursión para recorrer una lista ordenada de ellos. Conservar los índices originales al evaluar costos y acotar candidatos; la posición dentro de la lista no es el extremo del segmento. Si la factibilidad sólo se descubre probando todos los candidatos, esta plantilla no garantiza evitar $O(n^2)$ por capa.

8.12 DP $O(n\sqrt{n})$ por cota triangular

Cuándo usar: Un parámetro del estado crece o decrece estrictamente en cada salto, por lo que la suma mínima de los saltos limita su cantidad a $O(\sqrt{n})$.

Usar cuando un estado guarda una longitud/salto que cambia estrictamente en cada transición. No pueden existir demasiados cambios: $1 + 2 + \cdots + t \le n$ implica $t = O(\sqrt{n})$. Guardar sólo estados hasta ese límite suele bajar una DP aparentemente cuadrática a $O(n\sqrt{n})$.

Complejidad: tiempo $O(n\sqrt{n})$; memoria $O(n\sqrt{n})$ o $O(\sqrt{n})$ con rolling arrays.

```
int triangular_state_limit(int n) {
    int length = 0;
    while (1LL * (length + 1) * (length + 2) / 2 <= n) ++length;
    return length;
}

// Esqueleto: adaptar la transicion al problema.
vector<vector<ll>> triangular_bound_dp(int n) {
    int limit = triangular_state_limit(n);
    const ll infinity = (1LL << 60);
    vector<vector<ll>> dp(n + 1, vector<ll>(limit + 1, infinity));
    dp[0][0] = 0;
    for (int position = 0; position < n; ++position)
        for (int length = 0; length <= limit; ++length) if (dp[
     position][length] < infinity) {
            dp[position + 1][length] = min(dp[position + 1][length],
     dp[position][length]);
            int jump = length + 1;
            if (position + jump <= n && jump <= limit)
                dp[position + jump][jump] = min(dp[position + jump][
     jump],
                                                dp[position][length]
     + 1);
        }
    return dp;
}
```

Cuidado: La cota debe demostrarse con la suma mínima de los saltos; no basta con observar que aparece $\sqrt{n}$ en los límites.

9. Algoritmos offline

9.1 Mo's Algorithm

Cuándo usar: Muchas consultas offline sobre subarreglos, sin updates o con una extensión controlada, cuando agregar/quitar un extremo es barato.

Reordena consultas estáticas de rango para mover dos punteros pocas veces. Los rangos son semiabiertos $[l, r)$. El estado debe admitir agregar y quitar un elemento en $O(1)$ o costo pequeño.

Complejidad: aproximadamente $O((n+q)\sqrt{n})$ movimientos; memoria $O(n+q)$.

```
struct MoQuery { int left, right, index; };

template<class Add, class Remove, class GetAnswer>
vector<ll> mos_algorithm(int n, vector<MoQuery> queries,
                         Add add, Remove remove, GetAnswer get_answer
     ) {
    int block_size = max(1, (int)sqrt(max(1, n)));
    sort(queries.begin(), queries.end(), [&](const MoQuery &a, const
     MoQuery &b) {
        int block_a = a.left / block_size, block_b = b.left /
     block_size;
        if (block_a != block_b) return block_a < block_b;
        return block_a & 1 ? a.right > b.right : a.right < b.right;
    });

    vector<ll> answers(queries.size());
    int current_left = 0, current_right = 0;
    for (const MoQuery &query : queries) {
        while (current_left > query.left) add(--current_left);
        while (current_right < query.right) add(current_right++);
        while (current_left < query.left) remove(current_left++);
        while (current_right > query.right) remove(--current_right);
        answers[query.index] = get_answer();
    }
    return answers;
}
```

Ejemplos: cantidad de valores distintos, suma de frecuencias al cuadrado y consultas offline sobre frecuencias. Para actualizaciones existe Mo con tiempo.

Ejemplo: cantidad de valores distintos en cada rango. `add(index)` agrega `values[index]` a la ventana actual, `remove(index)` lo quita y `get_answer()` devuelve el estado mantenido.

```
vector<int> values={1,2,1,3,2};
vector<MoQuery> queries={{0,3,0},{2,5,1}}; // [0,3), [2,5)
unordered_map<int,int> frequency;
ll distinct=0;
auto add=[&](int index) {
    if(frequency[values[index]]++==0) ++distinct;
};
auto remove=[&](int index) {
    if(--frequency[values[index]]==0) --distinct;
};
auto get_answer=[&]() { return distinct; };
vector<ll> answer=mos_algorithm(values.size(),queries,
                                add,remove,get_answer);
cout<<answer[0]<<' '<<answer[1]<<'\n'; // 2 3
```

Las consultas se procesan reordenadas, pero `query.index` restaura el orden original de las respuestas. "Offline" significa que todas las consultas deben conocerse antes de comenzar; Mo no acelera una sola consulta ni admite updates comunes en esta versión.